Contents

- Secure Development Documentation
- Concepts
  - Best practices
    - Overview
    - Design secure apps
    - Develop secure apps
    - Deploy secure apps
- Resources
  - Microsoft Security Code Analysis
    - Overview
    - Onboarding and installation
    - Configuration
    - FAQs
    - YAML Configuration
    - Releases and roadmap
  - Microsoft Threat Modeling tool
    - Getting started
    - Feature overview
    - Threats
    - Releases
      - Threat Modeling Tool GA release 7.1.5091.2 - September 12 2018
      - Threat Modeling Tool update release 7.1.51023.1 - November 1 2018
      - Threat Modeling Tool update release 7.1.60126.1 - January 29 2019
      - Threat Modeling Tool update release 7.1.60408.1 - April 9 2019
      - Threat Modeling Tool update release 7.1.60702.1 - July 2 2019
      - Threat Modeling Tool update release 7.1.61015.1 - October 16 2019
      - Threat Modeling Tool update release 7.3.00206.1 - February 11 2020
      - Threat Modeling Tool update release 7.3.00316.1 - March 22 2020
      - Threat Modeling Tool update release 7.3.00714.2 - July 14 2020

Threat Modeling Tool update release 7.3.00729.1 - July 29 2020
Mitigations
Auditing and logging
Authentication
Authorization
Communication security
Configuration management
Cryptography
Exception management
Input validation
Sensitive data
Session management

Secure development best practices on Azure

10/12/2021 • 4 minutes to read • Edit Online

This series of articles presents security activities and controls to consider when you develop applications for the cloud. The phases of the Microsoft Security Development Lifecycle (SDL) and security questions and concepts to consider during each phase of the lifecycle are covered. The goal is to help you define activities and Azure services that you can use in each phase of the lifecycle to design, develop, and deploy a more secure application.

The recommendations in the articles come from our experience with Azure security and from the experiences of our customers. You can use these articles as a reference for what you should consider during a specific phase of your development project, but we suggest that you also read through all of the articles from beginning to end at least once. Reading all articles introduces you to concepts that you might have missed in earlier phases of your project. Implementing these concepts before you release your product can help you build secure software, address security compliance requirements, and reduce development costs.

These articles are intended to be a resource for software designers, developers, and testers at all levels who build and deploy secure Azure applications.

Overview

Security is one of the most important aspects of any application, and it's not a simple thing to get right. Fortunately, Azure provides many services that can help you secure your application in the cloud. These articles address activities and Azure services you can implement at each stage of your software development lifecycle to help you develop more secure code and deploy a more secure application in the cloud.

Security development lifecycle

Following best practices for secure software development requires integrating security into each phase of the software development lifecycle, from requirement analysis to maintenance, regardless of the project methodology (waterfall, agile, or DevOps). In the wake of high-profile data breaches and the exploitation of operational security flaws, more developers are understanding that security needs to be addressed throughout the development process.

The later you fix a problem in your development lifecycle, the more that fix will cost you. Security issues are no exception. If you disregard security issues in the early phases of your software development, each phase that follows might inherit the vulnerabilities of the preceding phase. Your final product will have accumulated multiple security issues and the possibility of a breach. Building security into each phase of the development lifecycle helps you catch issues early, and it helps you reduce your development costs.

We follow the phases of the Microsoft Security Development Lifecycle (SDL) to introduce activities and Azure services that you can use to fulfill secure software development practices in each phase of the lifecycle.

The SDL phases are:

- Training
- Requirements
- Design
- Implementation
- Verification
- Release
- Response



In these articles we group the SDL phases into design, develop, and deploy.

Engage your organization's security team

Your organization might have a formal application security program that assists you with security activities from start to finish during the development lifecycle. If your organization has security and compliance teams, be sure to engage them before you begin developing your application. Ask them at each phase of the SDL whether there are any tasks you missed.

We understand that many readers might not have a security or compliance team to engage. These articles can help guide you in the security questions and decisions you need to consider at each phase of the SDL.

Resources

Use the following resources to learn more about developing secure applications and to help secure your applications on Azure:

Microsoft Security Development Lifecycle (SDL) – The SDL is a software development process from Microsoft that helps developers build more secure software. It helps you address security compliance requirements while reducing development costs.

Open Web Application Security Project (OWASP) – OWASP is an online community that produces freely available articles, methodologies, documentation, tools, and technologies in the field of web application security.

Pushing Left, Like a Boss – A series of online articles that outlines different types of application security activities that developers should complete to create more secure code.

Microsoft identity platform – The Microsoft identity platform is an evolution of the Azure AD identity service and developer platform. It's a full-featured platform that consists of an authentication service, open-source libraries, application registration and configuration, full developer documentation, code samples, and other developer content. The Microsoft identity platform supports industry-standard protocols like OAuth 2.0 and OpenID Connect.

Security best practices for Azure solutions – A collection of security best practices to use when you design, deploy, and manage cloud solutions by using Azure. This paper is intended to be a resource for IT pros. This might include designers, architects, developers, and testers who build and deploy secure Azure solutions.

Security and Compliance Blueprints on Azure – Azure Security and Compliance Blueprints are resources that can help you build and launch cloud-powered applications that comply with stringent regulations and standards.

Next steps

In the following articles, we recommend security controls and activities that can help you design, develop, and deploy secure applications.

- Design secure applications
- Develop secure applications
- Deploy secure applications

Design secure applications on Azure

6/21/2021 • 21 minutes to read • Edit Online

In this article we present security activities and controls to consider when you design applications for the cloud. Training resources along with security questions and concepts to consider during the requirements and design phases of the Microsoft Security Development Lifecycle (SDL) are covered. The goal is to help you define activities and Azure services that you can use to design a more secure application.

The following SDL phases are covered in this article:

- Training
- Requirements
- Design

Training

Before you begin developing your cloud application, take time to understand security and privacy on Azure. By taking this step, you can reduce the number and severity of exploitable vulnerabilities in your application. You'll be more prepared to react appropriately to the ever-changing threat landscape.

Use the following resources during the training stage to familiarize yourself with the Azure services that are available to developers and with security best practices on Azure:

- Developer's guide to Azure shows you how to get started with Azure. The guide shows you which services you can use to run your applications, store your data, incorporate intelligence, build IoT apps, and deploy your solutions in a more efficient and secure way.
- Get started guide for Azure developers provides essential information for developers who are looking to get started using the Azure platform for their development needs.
- SDKs and tools describes the tools that are available on Azure.
- Azure DevOps Services provides development collaboration tools. The tools include high-performance pipelines, free Git repositories, configurable Kanban boards, and extensive automated and cloud-based load testing. The DevOps Resource Center combines our resources for learning DevOps practices, Git version control, agile methods, how we work with DevOps at Microsoft, and how you can assess your own DevOps progression.
- Top 5 security items to consider before pushing to production shows you how to help secure your web applications on Azure and protect your apps against the most common and dangerous web application attacks.
- Secure DevOps Kit for Azure is a collection of scripts, tools, extensions, and automations that caters to the comprehensive Azure subscription and resource security needs of DevOps teams that use extensive automation. The Secure DevOps Kit for Azure can show you how to smoothly integrate security into your native DevOps workflows. The kit addresses tools like security verification tests (SVTs), which can help developers write secure code and test the secure configuration of their cloud applications in the coding and early development stages.
- Security best practices for Azure solutions provides a collection of security best practices to use as you design, deploy, and manage your cloud solutions by using Azure.

Requirements

The requirements definition phase is a crucial step in defining what your application is and what it will do when it's released. The requirements phase is also a time to think about the security controls that you will build into your application. During this phase, you also begin the steps that you will take throughout the SDL to ensure that you release and deploy a secure application.

Consider security and privacy issues

This phase is the best time to consider foundational security and privacy issues. Defining acceptable levels of security and privacy at the start of a project helps a team:

- Understand risks associated with security issues.
- Identify and fix security bugs during development.
- Apply established levels of security and privacy throughout the entire project.

When you write the requirements for your application, be sure to consider security controls that can help keep your application and data safe.

Ask security questions

Ask security questions like:

- Does my application contain sensitive data?
- Does my application collect or store data that requires me to adhere to industry standards and compliance programs like the Federal Financial Institution Examination Council (FFIEC) or the Payment Card Industry Data Security Standards (PCI DSS)?
- Does my application collect or contain sensitive personal or customer data that can be used, either on its own or with other information, to identify, contact, or locate a single person?
- Does my application collect or contain data that can be used to access an individual's medical, educational, financial, or employment information? Identifying the sensitivity of your data during the requirements phase helps you classify your data and identify the data protection method you will use for your application.
- Where and how is my data stored? Consider how you will monitor the storage services that your application uses for any unexpected changes (such as slower response times). Will you be able to influence logging to collect more detailed data and analyze a problem in depth?
- Will my application be available to the public (on the internet) or internally only? If your application is available to the public, how do you protect the data that might be collected from being used in the wrong way? If your application is available internally only, consider who in your organization should have access to the application and how long they should have access.
- Do you understand your identity model before you begin designing your application? How will you determine that users are who they say they are and what a user is authorized to do?
- Does my application perform sensitive or important tasks (such as transferring money, unlocking doors, or delivering medicine)? Consider how you will validate that the user performing a sensitive task is authorized to perform the task and how you will authenticate that the person is who they say they are. Authorization (AuthZ) is the act of granting an authenticated security principal permission to do something. Authentication (AuthN) is the act of challenging a party for legitimate credentials.
- Does my application perform any risky software activities, like allowing users to upload or download files or other data? If your application does perform risky activities, consider how your application will protect users from handling malicious files or data.

Review OWASP top 10

Consider reviewing the OWASP Top 10 Application Security Risks. The OWASP Top 10 addresses critical security risks to web applications. Awareness of these security risks can help you make requirement and design decisions that minimize these risks in your application.

Thinking about security controls to prevent breaches is important. However, you also want to assume a breach will occur. Assuming a breach helps answer some important questions about security in advance, so they don't have to be answered in an emergency:

- How will I detect an attack?
- What will I do if there is an attack or breach?
- How am I going to recover from the attack like data leaking or tampering?

Design

The design phase is critical for establishing best practices for design and functional specifications. It also is critical for performing risk analysis that helps mitigate security and privacy issues throughout a project.

When you have security requirements in place and use secure design concepts, you can avoid or minimize opportunities for a security flaw. A security flaw is an oversight in the design of the application that might allow a user to perform malicious or unexpected actions after your application is released.

During the design phase, also think about how you can apply security in layers; one level of defense isn't necessarily enough. What happens if an attacker gets past your web application firewall (WAF)? You want another security control in place to defend against that attack.

With this in mind, we discuss the following secure design concepts and the security controls you should address when you design secure applications:

- Use a secure coding library and a software framework.
- Scan for vulnerable components.
- Use threat modeling during application design.
- Reduce your attack surface.
- Adopt a policy of identity as the primary security perimeter.
- Require re-authentication for important transactions.
- Use a key management solution to secure keys, credentials, and other secrets.
- Protect sensitive data.
- Implement fail-safe measures.
- Take advantage of error and exception handling.
- Use logging and alerting.

Use a secure coding library and a software framework

For development, use a secure coding library and a software framework that has embedded security. Developers can use existing, proven features (encryption, input sanitation, output encoding, keys or connection strings, and anything else that would be considered a security control) instead of developing security controls from scratch. This helps guard against security-related design and implementation flaws.

Be sure that you're using the latest version of your framework and all the security features that are available in the framework. Microsoft offers a comprehensive set of development tools for all developers, working on any platform or language, to deliver cloud applications. You can code with the language of your choice by choosing from various SDKs. You can take advantage of full-featured integrated development environments (IDEs) and editors that have advanced debugging capabilities and built-in Azure support.

Microsoft offers a variety of languages, frameworks, and tools that you can use to develop applications on Azure. An example is Azure for .NET and .NET Core developers. For each language and framework that we offer,

you'll find quickstarts, tutorials, and API references to help you get started fast.

Azure offers a variety of services you can use to host websites and web applications. These services let you develop in your favorite language, whether that's .NET, .NET Core, Java, Ruby, Node.js, PHP, or Python. Azure App Service Web Apps (Web Apps) is one of these services.

Web Apps adds the power of Microsoft Azure to your application. It includes security, load balancing, autoscaling, and automated management. You can also take advantage of DevOps capabilities in Web Apps, like package management, staging environments, custom domains, SSL/TLS certificates, and continuous deployment from Azure DevOps, GitHub, Docker Hub, and other sources.

Azure offers other services that you can use to host websites and web applications. For most scenarios, Web Apps is the best choice. For a micro service architecture, consider Azure Service Fabric. If you need more control over the VMs that your code runs on, consider Azure Virtual Machines. For more information about how to choose between these Azure services, see a comparison of Azure App Service, Virtual Machines, Service Fabric, and Cloud Services.

Apply updates to components

To prevent vulnerabilities, you should continuously inventory both your client-side and server-side components (for example, frameworks and libraries) and their dependencies for updates. New vulnerabilities and updated software versions are released continuously. Ensure that you have an ongoing plan to monitor, triage, and apply updates or configuration changes to the libraries and components you use.

See the Open Web Application Security Project (OWASP) page on using components with known vulnerabilities for tool suggestions. You can also subscribe to email alerts for security vulnerabilities that are related to components you use.

Use threat modeling during application design

Threat modeling is the process of identifying potential security threats to your business and application, and then ensuring that proper mitigations are in place. The SDL specifies that teams should engage in threat modeling during the design phase, when resolving potential issues is relatively easy and cost-effective. Using threat modeling in the design phase can greatly reduce your total cost of development.

To help facilitate the threat modeling process, we designed the SDL Threat Modeling Tool with non-security experts in mind. This tool makes threat modeling easier for all developers by providing clear guidance about how to create and analyze threat models.

Modeling the application design and enumerating STRIDE threats—Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege—across all trust boundaries has proven an effective way to catch design errors early on. The following table lists the STRIDE threats and gives some example mitigations that use features provided by Azure. These mitigations won't work in every situation.

THREAT	SECURITY PROPERTY	POTENTIAL AZURE PLATFORM MITIGATION
Spoofing	Authentication	Require HTTPS connections.
Tampering	Integrity	Validate SSL/TLS certificates. Applications that use SSL/TLS must fully verify the X.509 certificates of the entities they connect to. Use Azure Key Vault certificates to manage your x509 certificates.
Repudiation	Non-repudiation	Enable Azure monitoring and diagnostics.

THREAT	SECURITY PROPERTY	POTENTIAL AZURE PLATFORM MITIGATION
Information Disclosure	Confidentiality	Encrypt sensitive data at rest and in transit.
Denial of Service	Availability	Monitor performance metrics for potential denial of service conditions. Implement connection filters. Azure DDoS protection, combined with application design best practices, provides defense against DDoS attacks.
Elevation of Privilege	Authorization	Use Azure Active Directory Privileged Identity Management.

Reduce your attack surface

An attack surface is the total sum of where potential vulnerabilities might occur. In this paper, we focus on an application's attack surface. The focus is on protecting an application from attack. A simple and quick way to minimize your attack surface is to remove unused resources and code from your application. The smaller your application, the smaller your attack surface. For example, remove:

- Code for features you haven't released yet.
- Debugging support code.
- Network interfaces and protocols that aren't used or which have been deprecated.
- Virtual machines and other resources that you aren't using.

Doing regular cleanup of your resources and ensuring that you remove unused code are great ways to ensure that there are fewer opportunities for malicious actors to attack.

A more detailed and in-depth way to reduce your attack surface is to complete an attack surface analysis. An attack surface analysis helps you map the parts of a system that need to be reviewed and tested for security vulnerabilities.

The purpose of an attack surface analysis is to understand the risk areas in an application so developers and security specialists are aware of what parts of the application are open to attack. Then, you can find ways to minimize this potential, track when and how the attack surface changes, and what this means from a risk perspective.

An attack surface analysis helps you identify:

- Functions and parts of the system you need to review and test for security vulnerabilities.
- High-risk areas of code that require defense-in-depth protection (parts of the system that you need to defend).
- When you alter the attack surface and need to refresh a threat assessment.

Reducing opportunities for attackers to exploit a potential weak spot or vulnerability requires you to thoroughly analyze your application's overall attack surface. It also includes disabling or restricting access to system services, applying the principle of least privilege, and employing layered defenses wherever possible.

We discuss conducting an attack surface review during the verification phase of the SDL.

> **NOTE**
>
> **What's the difference between threat modeling and attack surface analysis?** Threat modeling is the process of identifying potential security threats to your application and ensuring that proper mitigations against the threats are in place. Attack surface analysis identifies high-risk areas of code that are open to attack. It involves finding ways to defend high-risk areas of your application and reviewing and testing those areas of code before you deploy the application.

Adopt a policy of identity as the primary security perimeter

When you design cloud applications, it's important to expand your security perimeter focus from a network-centric approach to an identity-centric approach. Historically, the primary on-premises security perimeter was an organization's network. Most on-premises security designs use the network as the primary security pivot. For cloud applications, you are better served by considering identity as the primary security perimeter.

Things you can do to develop an identity-centric approach to developing web applications:

- Enforce multi-factor authentication for users.
- Use strong authentication and authorization platforms.
- Apply the principle of least privilege.
- Implement just-in-time access.

Enforce multi-factor authentication for users

Use two-factor authentication. Two-factor authentication is the current standard for authentication and authorization because it avoids the security weaknesses that are inherent in username and password types of authentication. Access to the Azure management interfaces (Azure portal/remote PowerShell) and to customer-facing services should be designed and configured to use Azure Multi-Factor Authentication.

Use strong authentication and authorization platforms

Use platform-supplied authentication and authorization mechanisms instead of custom code. This is because developing custom authentication code can be prone to error. Commercial code (for example, from Microsoft) often is extensively reviewed for security. Azure Active Directory (Azure AD) is the Azure solution for identity and access management. These Azure AD tools and services help with secure development:

- Microsoft identity platform is a set of components that developers use to build apps that securely sign in users. The platform assists developers who are building single-tenant, line-of-business (LOB) apps and developers who are looking to develop multi-tenant apps. In addition to basic sign-in, apps built by using the Microsoft identity platform can call Microsoft APIs and custom APIs. The Microsoft identity platform supports industry-standard protocols like OAuth 2.0 and OpenID Connect.

- Azure Active Directory B2C (Azure AD B2C) is an identity management service you can use to customize and control how customers sign up, sign in, and manage their profiles when they use your applications. This includes applications that are developed for iOS, Android, and .NET, among others. Azure AD B2C enables these actions while protecting customer identities.

Apply the principle of least privilege

The concept of least privilege means giving users the precise level of access and control they need to do their jobs and nothing more.

Would a software developer need domain admin rights? Would an administrative assistant need access to administrative controls on their personal computer? Evaluating access to software is no different. If you use Azure role-based access control (Azure RBAC) to give users different abilities and authority in your application, you wouldn't give everyone access to everything. By limiting access to what is required for each role, you limit the risk of a security issue occurring.

Ensure that your application enforces least privilege throughout its access patterns.

> **NOTE**
> The rules of least privilege need to apply to the software and to the people creating the software. Software developers can be a huge risk to IT security if they are given too much access. The consequences can be severe if a developer has malicious intent or is given too much access. We recommend that the rules of least privilege be applied to developers throughout the development lifecycle.

Implement just-in-time access

Implement *just-in-time* (JIT) access to further lower the exposure time of privileges. Use Azure AD Privileged Identity Management to:

- Give users the permissions they need only JIT.
- Assign roles for a shortened duration with confidence that the privileges are revoked automatically.

Require re-authentication for important transactions

Cross-site request forgery (also known as *XSRF* or *CSRF*) is an attack against web-hosted apps in which a malicious web app influences the interaction between a client browser and a web app that trusts that browser. Cross-site request forgery attacks are possible because web browsers send some types of authentication tokens automatically with every request to a website. This form of exploitation is also known as a *one-click attack* or *session riding* because the attack takes advantage of the user's previously authenticated session.

The best way to defend against this kind of attack is to ask the user for something that only the user can provide before every important transaction, such as a purchase, account deactivation, or a password change. You might ask the user to reenter their password, complete a captcha, or submit a secret token that only the user would have. The most common approach is the secret token.

Use a key management solution to secure keys, credentials, and other secrets

Losing keys and credentials is a common problem. The only thing worse than losing your keys and credentials is having an unauthorized party gain access to them. Attackers can take advantage of automated and manual techniques to find keys and secrets that are stored in code repositories like GitHub. Don't put keys and secrets in these public code repositories or on any other server.

Always put your keys, certificates, secrets, and connection strings in a key management solution. You can use a centralized solution in which keys and secrets are stored in hardware security modules (HSMs). Azure provides you with an HSM in the cloud with Azure Key Vault.

Key Vault is a *secret store*: it's a centralized cloud service for storing application secrets. Key Vault keeps your confidential data safe by keeping application secrets in a single, central location and providing secure access, permissions control, and access logging.

Secrets are stored in individual *vaults*. Each vault has its own configuration and security policies to control access. You get to your data through a REST API or through a client SDK that's available for most programming languages.

> **IMPORTANT**
> Azure Key Vault is designed to store configuration secrets for server applications. It's not intended for storing data that belongs to app users. This is reflected in its performance characteristics, API, and cost model.
>
> User data should be stored elsewhere, like in an Azure SQL Database instance that has Transparent Data Encryption (TDE) or in a storage account that uses Azure Storage Service Encryption. Secrets that are used by your application to access these data stores can be kept in Azure Key Vault.

Protect sensitive data

Protecting data is an essential part of your security strategy. Classifying your data and identifying your data

protection needs helps you design your app with data security in mind. Classifying (categorizing) stored data by sensitivity and business impact helps developers determine the risks that are associated with data.

Label all applicable data as sensitive when you design your data formats. Ensure that the application treats the applicable data as sensitive. These practices can help you protect your sensitive data:

- Use encryption.
- Avoid hard-coding secrets like keys and passwords.
- Ensure that access controls and auditing are in place.

Use encryption

Protecting data should be an essential part of your security strategy. If your data is stored in a database or if it moves back and forth between locations, use encryption of data at rest (while in the database) and encryption of data in transit (on its way to and from the user, the database, an API, or service endpoint). We recommend that you always use SSL/TLS protocols to exchange data. Ensure that you use the latest version of TLS for encryption (currently, this is version 1.2).

Avoid hard-coding

Some things should never be hard-coded in your software. Some examples are hostnames or IP addresses, URLs, email addresses, usernames, passwords, storage account keys, and other cryptographic keys. Consider implementing requirements around what can or can't be hard-coded in your code, including in the comment sections of your code.

When you put comments in your code, ensure that you don't save any sensitive information. This includes your email address, passwords, connection strings, information about your application that would only be known by someone in your organization, and anything else that might give an attacker an advantage in attacking your application or organization.

Basically, assume that everything in your development project will be public knowledge when it is deployed. Avoid including sensitive data of any kind in the project.

Earlier, we discussed Azure Key Vault. You can use Key Vault to store secrets like keys and passwords instead of hard-coding them. When you use Key Vault in combination with managed identities for Azure resources, your Azure web app can access secret configuration values easily and securely without storing any secrets in your source control or configuration. To learn more, see Manage secrets in your server apps with Azure Key Vault.

Implement fail-safe measures

Your application must be able to handle errors that occur during execution in a consistent manner. The application should catch all errors and either fail safe or closed.

You should also ensure that errors are logged with sufficient user context to identify suspicious or malicious activity. Logs should be retained for a sufficient time to allow delayed forensic analysis. Logs should be in a format that can be easily consumed by a log management solution. Ensure that alerts for errors that are related to security are triggered. Insufficient logging and monitoring allows attackers to further attack systems and maintain persistence.

Take advantage of error and exception handling

Implementing correct error and exception handling is an important part of defensive coding. Error and exception handling are critical to making a system reliable and secure. Mistakes in error handling can lead to different kinds of security vulnerabilities, such as leaking information to attackers and helping attackers understand more about your platform and design.

Ensure that:

- You handle exceptions in a centralized manner to avoid duplicated try/catch blocks in the code.
- All unexpected behaviors are handled inside the application.

- Messages that are displayed to users don't leak critical data but do provide enough information to explain the issue.
- Exceptions are logged and that they provide enough information for forensics or incident response teams to investigate.

Azure Logic Apps provides a first-class experience for handling errors and exceptions that are caused by dependent systems. You can use Logic Apps to create workflows to automate tasks and processes that integrate apps, data, systems, and services across enterprises and organizations.

Use logging and alerting

Log your security issues for security investigations and trigger alerts about issues to ensure that people know about problems in a timely manner. Enable auditing and logging on all components. Audit logs should capture user context and identify all important events.

Check that you don't log any sensitive data that a user submits to your site. Examples of sensitive data include:

- User credentials
- Social Security numbers or other identifying information
- Credit card numbers or other financial information
- Health information
- Private keys or other data that can be used to decrypt encrypted information
- System or application information that can be used to more effectively attack the application

Ensure that the application monitors user management events such as successful and failed user logins, password resets, password changes, account lockout, and user registration. Logging for these events helps you detect and react to potentially suspicious behavior. It also allows you to gather operations data, like who is accessing the application.

Next steps

In the following articles, we recommend security controls and activities that can help you develop and deploy secure applications.

- Develop secure applications
- Deploy secure applications

Develop secure applications on Azure

6/8/2021 • 9 minutes to read • Edit Online

In this article, we present security activities and controls to consider when you develop applications for the cloud. Security questions and concepts to consider during the implementation and verification phases of the Microsoft Security Development Lifecycle (SDL) are covered. The goal is to help you define activities and Azure services that you can use to develop a more secure application.

The following SDL phases are covered in this article:

- Implementation
- Verification

Implementation

The focus of the implementation phase is to establish best practices for early prevention and to detect and remove security issues from the code. Assume that your application will be used in ways that you didn't intend it to be used. This helps you guard against accidental or intentional misuse of your application.

Perform code reviews

Before you check in code, conduct code reviews to increase overall code quality and reduce the risk of creating bugs. You can use Visual Studio to manage the code review process.

Perform static code analysis

Static code analysis (also known as *source code analysis*) is usually performed as part of a code review. Static code analysis commonly refers to running static code analysis tools to find potential vulnerabilities in non-running code by using techniques like taint checking and data flow analysis.

Azure Marketplace offers developer tools that perform static code analysis and assist with code reviews.

Validate and sanitize every input for your application

Treat all input as untrusted to protect your application from the most common web application vulnerabilities. Untrusted data is a vehicle for injection attacks. Input for your application includes parameters in the URL, input from the user, data from the database or from an API, and anything that is passed in that a user could potentially manipulate. An application should validate that data is syntactically and semantically valid before the application uses the data in any way (including displaying it back to the user).

Validate input early in the data flow to ensure that only properly formed data enters the workflow. You don't want malformed data persisting in your database or triggering a malfunction in a downstream component.

Blocklisting and allowlisting are two general approaches to performing input syntax validation:

- Blocklisting attempts to check that a given user input doesn't contain "known to be malicious" content.
- Allowlisting attempts to check that a given user input matches a set of "known good" inputs. Character-based allowlisting is a form of allowlisting where an application checks that user input contains only "known good" characters or that input matches a known format. For example, this might involve checking that a username contains only alphanumeric characters or that it contains exactly two numbers.

Allowlisting is the preferred approach for building secure software. Blocklisting is prone to error because it's impossible to think of a complete list of potentially bad input.

Do this work on the server, not on the client side (or on the server and on the client side).

Verify your application's outputs

Any output that you present either visually or within a document should always be encoded and escaped. Escaping, also known as *output encoding*, is used to help ensure that untrusted data isn't a vehicle for an injection attack. Escaping, combined with data validation, provides layered defenses to increase security of the system as a whole.

Escaping makes sure that everything is displayed as *output*. Escaping also lets the interpreter know that the data isn't intended to be executed, and this prevents attacks from working. This is another common attack technique called *cross-site scripting* (XSS).

If you are using a web framework from a third party, you can verify your options for output encoding on websites by using the OWASP XSS prevention cheat sheet.

Use parameterized queries when you contact the database

Never create an inline database query "on the fly" in your code and send it directly to the database. Malicious code inserted into your application could potentially cause your database to be stolen, wiped, or modified. Your application could also be used to run malicious operating system commands on the operating system that hosts your database.

Instead, use parameterized queries or stored procedures. When you use parameterized queries, you can invoke the procedure from your code safely and pass it a string without worrying that it will be treated as part of the query statement.

Remove standard server headers

Headers like Server, X-Powered-By, and X-AspNet-Version reveal information about the server and underlying technologies. We recommend that you suppress these headers to avoid fingerprinting the application. See removing standard server headers on Azure websites.

Segregate your production data

Your production data, or "real" data, should not be used for development, testing, or any other purpose than what the business intended. A masked (anonymized) dataset should be used for all development and testing.

This means fewer people have access to your real data, which reduces your attack surface. It also means fewer employees see personal data, which eliminates a potential breach in confidentiality.

Implement a strong password policy

To defend against brute-force and dictionary-based guessing, you must implement a strong password policy to ensure that users create a complex password (for example, 12 characters minimum length and requiring alphanumeric and special characters).

Azure Active Directory B2C helps you with password management, by providing self-service password reset, force password reset, and more.

To defend against attacks on default accounts, verify that all keys and passwords are replaceable and that they're generated or replaced after you install resources.

If the application must autogenerate passwords, ensure that the generated passwords are random and that they have high entropy.

Validate file uploads

If your application allows file uploads, consider precautions that you can take for this risky activity. The first step in many attacks is to get some malicious code into a system that is under attack. Using a file upload helps the attacker accomplish this. OWASP offers solutions for validating a file to ensure that the file you're uploading is safe.

Antimalware protection helps identify and remove viruses, spyware, and other malicious software. You can

install Microsoft Antimalware or a Microsoft partner's endpoint protection solution (Trend Micro, Broadcom, McAfee, Windows Defender, and Endpoint Protection).

Microsoft Antimalware includes features like real-time protection, scheduled scanning, malware remediation, signature updates, engine updates, samples reporting, and exclusion event collection. You can integrate Microsoft Antimalware and partner solutions with Azure Security Center for ease of deployment and built-in detections (alerts and incidents).

Don't cache sensitive content

Don't cache sensitive content on the browser. Browsers can store information for caching and history. Cached files are stored in a folder like the Temporary Internet Files folder, in the case of Internet Explorer. When these pages are referred to again, the browser displays the pages from its cache. If sensitive information (address, credit card details, Social Security number, username) is displayed to the user, the information might be stored in the browser's cache and be retrievable by examining the browser's cache or by simply pressing the browser's **Back** button.

Verification

The verification phase involves a comprehensive effort to ensure that the code meets the security and privacy tenets that were established in the preceding phases.

Find and fix vulnerabilities in your application dependencies

You scan your application and its dependent libraries to identify any known vulnerable components. Products that are available to perform this scan include OWASP Dependency Check,Snyk, and Black Duck.

Vulnerability scanning powered by Tinfoil Security is available for Azure App Service Web Apps. Tinfoil Security scanning through App Service offers developers and administrators a fast, integrated, and economical means of discovering and addressing vulnerabilities before a malicious actor can take advantage of them.

> **NOTE**
>
> You can also integrate Tinfoil Security with Azure AD. Integrating Tinfoil Security with Azure AD provides you with the following benefits:
>
> - In Azure AD, you can control who has access to Tinfoil Security.
> - Your users can be automatically signed in to Tinfoil Security (single sign-on) by using their Azure AD accounts.
> - You can manage your accounts in a single, central location, the Azure portal.

Test your application in an operating state

Dynamic application security testing (DAST) is a process of testing an application in an operating state to find security vulnerabilities. DAST tools analyze programs while they are executing to find security vulnerabilities such as memory corruption, insecure server configuration, cross-site scripting, user privilege issues, SQL injection, and other critical security concerns.

DAST is different from static application security testing (SAST). SAST tools analyze source code or compiled versions of code when the code is not executing in order to find security flaws.

Perform DAST, preferably with the assistance of a security professional (a penetration tester or vulnerability assessor). If a security professional isn't available, you can perform DAST yourself with a web proxy scanner and some training. Plug in a DAST scanner early on to ensure that you don't introduce obvious security issues into your code. See the OWASP site for a list of web application vulnerability scanners.

Perform fuzz testing

In fuzz testing, you induce program failure by deliberately introducing malformed or random data to an application. Inducing program failure helps reveal potential security issues before the application is released.

Security Risk Detection is the Microsoft unique fuzz testing service for finding security-critical bugs in software.

Conduct attack surface review

Reviewing the attack surface after code completion helps ensure that any design or implementation changes to an application or system has been considered. It helps ensure that any new attack vectors that were created as a result of the changes, including threat models, has been reviewed and mitigated.

You can build a picture of the attack surface by scanning the application. Microsoft offers an attack surface analysis tool called Attack Surface Analyzer. You can choose from many commercial dynamic testing and vulnerability scanning tools or services, including OWASP Zed Attack Proxy Project, Arachni, Skipfish, and w3af. These scanning tools crawl your app and map the parts of the application that are accessible over the web. You can also search the Azure Marketplace for similar developer tools.

Perform security penetration testing

Ensuring that your application is secure is as important as testing any other functionality. Make penetration testing a standard part of the build and deployment process. Schedule regular security tests and vulnerability scanning on deployed applications, and monitor for open ports, endpoints, and attacks.

Run security verification tests

Secure DevOps Kit for Azure (AzSK) contains SVTs for multiple services of the Azure platform. You run these SVTs periodically to ensure that your Azure subscription and the different resources that comprise your application are in a secure state. You can also automate these tests by using the continuous integration/continuous deployment (CI/CD) extensions feature of AzSK, which makes SVTs available as a Visual Studio extension.

Next steps

In the following articles, we recommend security controls and activities that can help you design and deploy secure applications.

- Design secure applications
- Deploy secure applications

Deploy secure applications on Azure

3/5/2021 • 3 minutes to read • Edit Online

In this article we present security activities and controls to consider when you deploy applications for the cloud. Security questions and concepts to consider during the release and response phases of the Microsoft Security Development Lifecycle (SDL) are covered. The goal is to help you define activities and Azure services that you can use to deploy a more secure application.

The following SDL phases are covered in this article:

- Release
- Response

Release

The focus of the release phase is readying a project for public release. This includes planning ways to effectively perform post-release servicing tasks and address security vulnerabilities that might occur later.

Check your application's performance before you launch

Check your application's performance before you launch it or deploy updates to production. Run cloud-based load tests by using Visual Studio to find performance problems in your application, improve deployment quality, make sure that your application is always up or available, and that your application can handle traffic for your launch.

Install a web application firewall

Web applications are increasingly targets of malicious attacks that exploit common known vulnerabilities. Common among these exploits are SQL injection attacks and cross-site scripting attacks. Preventing these attacks in application code can be challenging. It might require rigorous maintenance, patching, and monitoring at many layers of the application topology. A centralized WAF helps make security management simpler. A WAF solution can also react to a security threat by patching a known vulnerability at a central location versus securing each individual web application.

The Azure Application Gateway WAF provides centralized protection of your web applications from common exploits and vulnerabilities. The WAF is based on rules from the OWASP core rule sets 3.0 or 2.2.9.

Create an incident response plan

Preparing an incident response plan is crucial to help you address new threats that might emerge over time. Preparing an incident response plan includes identifying appropriate security emergency contacts and establishing security servicing plans for code that's inherited from other groups in the organization and for licensed third-party code.

Conduct a final security review

Deliberately reviewing all security activities that were performed helps ensure readiness for your software release or application. The final security review (FSR) usually includes examining threat models, tools outputs, and performance against the quality gates and bug bars that were defined in the requirements phase.

Certify release and archive

Certifying software before a release helps ensure that security and privacy requirements are met. Archiving all pertinent data is essential for performing post-release servicing tasks. Archiving also helps lower the long-term costs associated with sustained software engineering.

Response

The response post-release phase centers on the development team being able and available to respond appropriately to any reports of emerging software threats and vulnerabilities.

Execute the incident response plan

Being able to implement the incident response plan instituted in the release phase is essential to helping protect customers from software security or privacy vulnerabilities that emerge.

Monitor application performance

Ongoing monitoring of your application after it's deployed potentially helps you detect performance issues as well as security vulnerabilities. Azure services that assist with application monitoring are:

- Azure Application Insights
- Azure Security Center

Application Insights

Application Insights is an extensible Application Performance Management (APM) service for web developers on multiple platforms. Use it to monitor your live web application. Application Insights automatically detects performance anomalies. It includes powerful analytics tools to help you diagnose issues and understand what users actually do with your app. It's designed to help you continuously improve performance and usability.

Azure Security Center

Azure Security Center helps you prevent, detect, and respond to threats with increased visibility into (and control over) the security of your Azure resources, including web applications. Azure Security Center helps detect threats that might otherwise go unnoticed. It works with various security solutions.

Security Center's Free tier offers limited security for your Azure resources only. The Security Center Standard tier extends these capabilities to on-premises resources and other clouds. Security Center Standard helps you:

- Find and fix security vulnerabilities.
- Apply access and application controls to block malicious activity.
- Detect threats by using analytics and intelligence.
- Respond quickly when under attack.

Next steps

In the following articles, we recommend security controls and activities that can help you design and develop secure applications.

- Design secure applications
- Develop secure applications

About Microsoft Security Code Analysis

6/15/2021 • 4 minutes to read • Edit Online

> **NOTE**
> Effective March 1, 2022, the Microsoft Security Code Analysis (MSCA) extension will be retired. Existing MSCA customers will retain their access to MSCA through March 1, 2022. Please refer to the OWASP Source Code Analysis Tools for alternative options in Azure DevOps. For customers planning to migrate to GitHub, you can check out GitHub Advanced Security.

With the Microsoft Security Code Analysis extension, teams can add security code analysis to their Azure DevOps continuous integration and delivery (CI/CD) pipelines. This analysis is recommended by the Secure Development Lifecycle (SDL) experts at Microsoft.

A consistent UX simplifies security by hiding the complexity of running tools. With NuGet-based delivery of the tools, teams no longer need to manage the installation or update of tooling. With both command-line and basic interfaces for build tasks, all users can have as much control over the tools as they want.

Teams can also use powerful postprocessing capabilities such as:

- Publishing logs for retention.
- Generating actionable, developer-focused reports.
- Configuring build breaks on regression tests.

Why should I use Microsoft Security Code Analysis?

Security simplified

Adding Microsoft Security Code Analysis tools to your Azure DevOps pipeline is as simple as adding new tasks. Customize the tasks or use their default behavior. Tasks run as part of your Azure DevOps pipeline and produce logs that detail many kinds of results.

Clean builds

After addressing the initial issues reported by the tools, you can configure the extension to break builds on new issues. Setting up continuous integration builds on every pull request is easy.

Set it and forget it

By default, the build tasks and tools stay up-to-date. If there's an updated version of a tool, you don't need to download and install it. The extension takes care of the updating for you.

Under the hood

The extension's build tasks hide the complexities of:

- Running security static-analysis tools.
- Processing the results from log files to create a summary report or break the build.

Microsoft Security Code Analysis tool set

The Microsoft Security Code Analysis extension makes the latest versions of important analysis tools readily available to you. The extension includes both Microsoft-managed tools and open-source tools.

These tools are automatically downloaded to the cloud-hosted agent after you use the corresponding build task

to configure and run the pipeline.

This section lists the set of tools that are currently available in the extension. Watch for the addition of more tools. Also, send us your suggestions for tools that you want us to add.

Anti-Malware Scanner

The Anti-Malware Scanner build task is now included in the Microsoft Security Code Analysis extension. This task must be run on a build agent that has Windows Defender already installed. For more information, see the Windows Defender website.

BinSkim

BinSkim is a Portable Executable (PE) lightweight scanner that validates compiler settings, linker settings, and other security-relevant characteristics of binary files. This build task provides a command-line wrapper around the binskim.exe console application. BinSkim is an open-source tool. For more information, see BinSkim on GitHub.

Credential Scanner

Passwords and other secrets stored in source code are a significant problem. Credential Scanner is a proprietary static-analysis tool that helps solve this problem. The tool detects credentials, secrets, certificates, and other sensitive content in your source code and your build output.

Roslyn Analyzers

Roslyn Analyzers is Microsoft's compiler-integrated tool for statically analyzing managed C# and Visual Basic code. For more information, see Roslyn-based analyzers.

TSLint

TSLint is an extensible static-analysis tool that checks TypeScript code for readability, maintainability, and errors in functionality. It's widely supported by modern editors and build systems. You can customize it with your own lint rules, configurations, and formatters. TSLint is an open-source tool. For more information, see TSLint on GitHub.

Analysis and post-processing of results

The Microsoft Security Code Analysis extension also has three postprocessing tasks. These tasks help you analyze the results found by the security-tool tasks. When added to a pipeline, these tasks usually follow all other tool tasks.

Publish Security Analysis Logs

The Publish Security Analysis Logs build task preserves the log files of the security tools that are run during the build. You can read these logs for investigation and follow-up.

You can publish the log files to Azure Artifacts as a .zip file. You can also copy them to an accessible file share from your private build agent.

Security Report

The Security Report build task parses the log files. These files are created by the security tools that run during the build. The build task then creates a single summary report file. This file shows all issues found by the analysis tools.

You can configure this task to report results for specific tools or for all tools. You can also choose what issue level to report, like errors only or both errors and warnings.

Post-Analysis (build break)

With the Post-Analysis build task, you can inject a build break that purposely causes a build to fail. You inject a build break if one or more analysis tools report issues in the code.

You can configure this task to break the build for issues found by specific tools or all tools. You can also configure it based on the severity of issues found, such as errors or warnings.

> **NOTE**
>
> By design, each build task succeeds if the task finishes successfully. This is true whether or not a tool finds issues, so that the build can run to completion by allowing all tools to run.

Next steps

For instructions on how to onboard and install Microsoft Security Code Analysis, refer to our Onboarding and installation guide.

For more information about configuring the build tasks, see our Configuration guide or YAML Configuration guide.

If you have further questions about the extension and the tools offered, check out our FAQ page.

Onboarding and installing

6/15/2021 • 3 minutes to read • Edit Online

> **NOTE**
> Effective March 1, 2022, the Microsoft Security Code Analysis (MSCA) extension will be retired. Existing MSCA customers will retain their access to MSCA through March 1, 2022. Please refer to the OWASP Source Code Analysis Tools for alternative options in Azure DevOps. For customers planning to migrate to GitHub, you can check out GitHub Advanced Security.

Prerequisites to getting started with Microsoft Security Code Analysis:

- An eligible Microsoft Unified Support offering, as detailed in the following section.
- An Azure DevOps organization.
- Permission to install extensions to the Azure DevOps organization.
- Source code that can be synced to a cloud-hosted Azure DevOps pipeline.

Onboarding the Microsoft Security Code Analysis extension

Interested in purchasing the Microsoft Security Code Analysis extension?

If you have one of the following support offerings, contact your Technical Account Manager to purchase or swap existing hours to get access to the extension:

- Unified Support Advanced tier
- Unified Support Performance tier
- Premier Support for Developers
- Premier Support for Partners
- Premier Support for Enterprise

If you do not have one of the mentioned above support agreements, you can purchase the extension from one of our Partners.

Next Steps:

If you meet the above qualifications, contact a partner from the below list to purchase the Microsoft Security Code Analysis extension. Otherwise, contact Microsoft Security Code Analysis Support.

> **Partners:**

- Zones – Contact Details: cloudsupport@zones.com
- Wortell – Contact Details: info@wortell.nl
- Logicalis – Contact Details: logicalisleads@us.logicalis.com

Become a Partner

The Microsoft Security Code Analysis team is looking to onboard partners with a Premier Support for Partners agreement. Partners will help empower Azure DevOps customers to develop more securely by selling the extension to customers who wish to purchase it, but don't have an Enterprise Support agreement with Microsoft. Interested partners can register here.

Installing the Microsoft Security Code Analysis extension

1. After the extension is shared with your Azure DevOps organization, go to your Azure DevOps organization page. An example URL for such a page is `https://dev.azure.com/contoso`.
2. Select the shopping bag icon in the upper-right corner next to your name, then select **Manage extensions**.
3. Select **Shared**.
4. Select the Microsoft Security Code Analysis extension, select **install**.
5. From the drop-down list, choose the Azure DevOps organization to install the extension on.
6. Select **Install**. After installation is complete, you can start using the extension.

> **NOTE**
> Even if you don't have access for installing the extension, continue with the installation steps. You can request access from your Azure DevOps organization admin during the installation process.

After you install the extension, the secure development build tasks are visible and available to add to your Azure Pipelines.

Adding specific build tasks to your Azure DevOps pipeline

1. From your Azure DevOps organization, open your team project.
2. Select **Pipelines** > **Builds**.
3. Select the pipeline into which you want to add the extension build tasks:
   - New pipeline: Select **New** and follow the steps detailed to create a new pipeline.
   - Edit pipeline: Select an existing pipeline and then select **Edit** to begin editing the pipeline.
4. Select **+** and go to the **Add Tasks** pane.
5. From either the list or by using the search box, find the build task you want to add. Select **Add**.
6. Specify the parameters needed for the task.
7. Queue a new build.

> **NOTE**
> File and folder paths are relative to the root of your source repository. If you specify the output files and folders as parameters, they are replaced with the common location that we have defined on the build agent.

> **TIP**
> - To run an analysis after your build, place the Microsoft Security Code Analysis build tasks after the Publish Build Artifacts step of your build. That way, your build can finish and post results before running static analysis tools.
> - Always select **Continue on Error** for secure-development build tasks. Even if one tool fails, the others can run. There are no interdependencies among tools.
> - Microsoft Security Code Analysis build tasks fail only if a tool fails to run successfully. But they succeed even if a tool identifies issues in the code. By using the Post-Analysis build task, you can configure your build to fail when a tool identifies issues in the code.
> - Some Azure DevOps build tasks are not supported when run via a release pipeline. More specifically, Azure DevOps doesn't support tasks that publish artifacts from within a release pipeline.
> - For a list of predefined variables in Azure DevOps Team Build that you can specify as parameters, see Azure DevOps Build Variables.

Next steps

For more information about configuring the build tasks, see our Configuration guide or YAML Configuration guide.

If you have more questions about the extension and the tools offered, check out our FAQ page.

Configure and customize the build tasks

6/15/2021 • 8 minutes to read • Edit Online

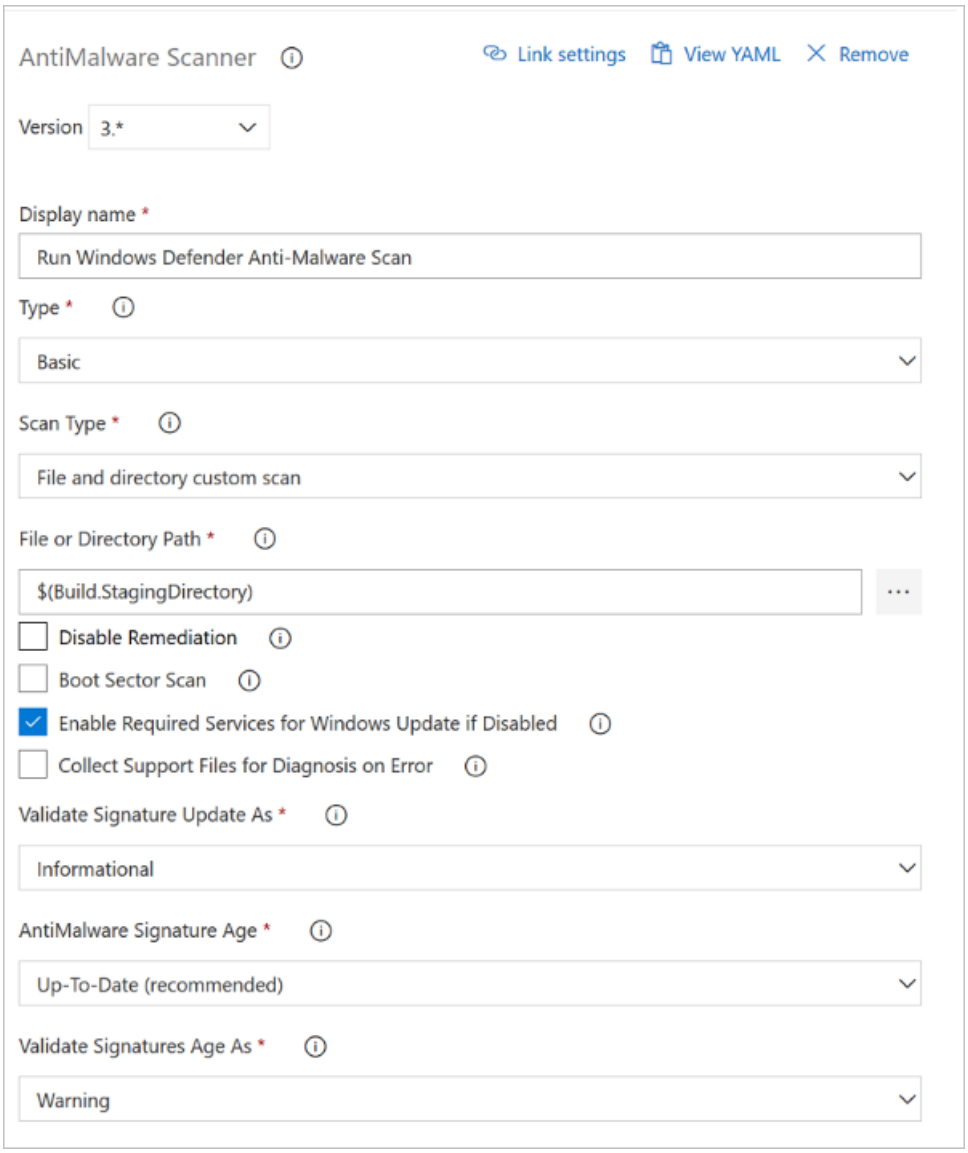
> **NOTE**
> Effective March 1, 2022, the Microsoft Security Code Analysis (MSCA) extension will be retired. Existing MSCA customers will retain their access to MSCA through March 1, 2022. Please refer to the OWASP Source Code Analysis Tools for alternative options in Azure DevOps. For customers planning to migrate to GitHub, you can check out GitHub Advanced Security.

This article describes in detail the configuration options available in each of the build tasks. The article starts with the tasks for security code analysis tools. It ends with the post-processing tasks.

Anti-Malware Scanner task

> **NOTE**
> The Anti-Malware Scanner build task requires a build agent with Windows Defender enabled. Hosted Visual Studio 2017 and later provide such an agent. The build task won't run on the Visual Studio 2015 hosted agent.
>
> Although signatures can't be updated on these agents, signatures should always be less than three hours old.

Details of task configuration are shown in the following screenshot and text.



In the **Type** list box of the screenshot, **Basic** is selected. Select **Custom** to provide command-line arguments that customize the scan.

Windows Defender uses the Windows Update client to download and install signatures. If signature update fails on your build agent, the **HRESULT** error code is likely coming from Windows Update.

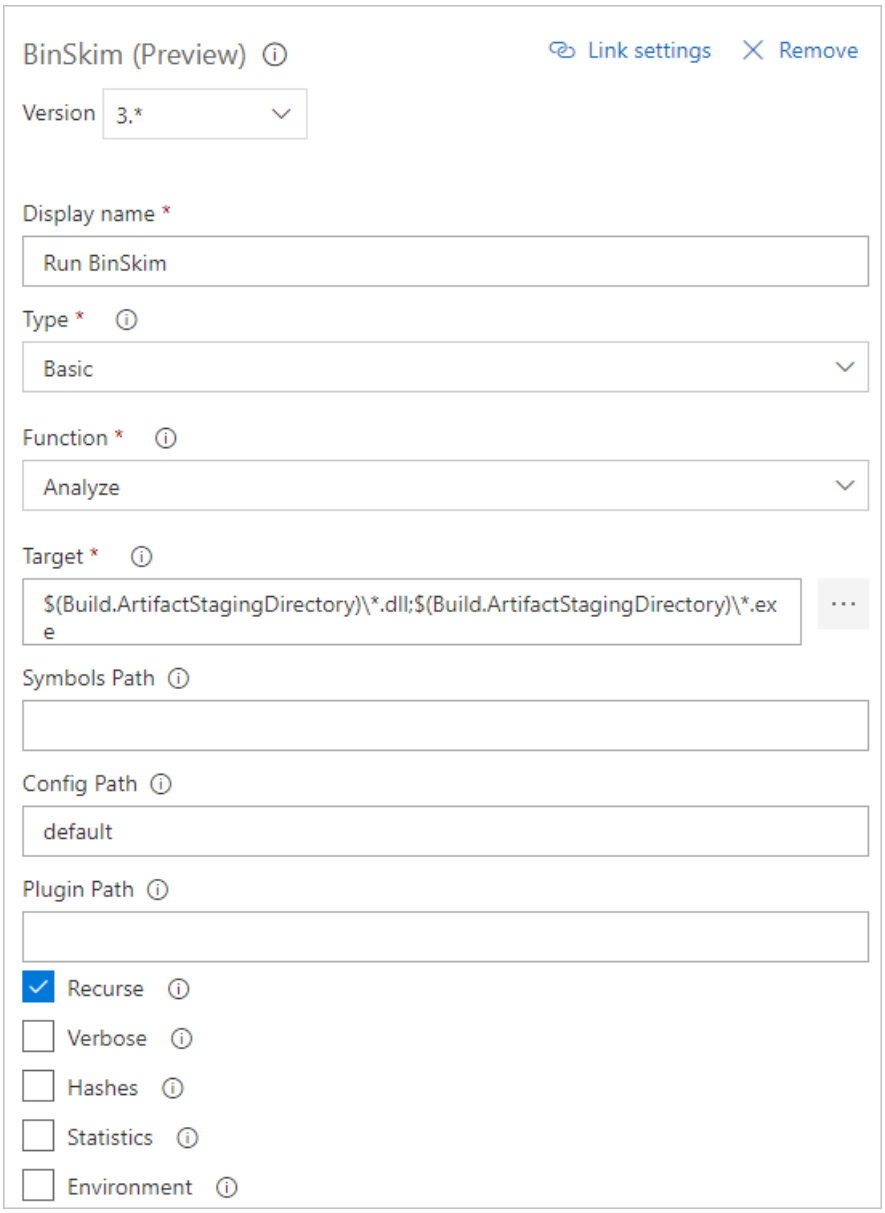
For more information on Windows Update errors and their mitigation, see Windows Update error codes by component and the TechNet article Windows Update Agent - Error Codes.

For information about YAML configuration for this task, check our Anti-Malware YAML options

BinSkim task

> **NOTE**
>
> Before you can run the BinSkim task, your build must meet one of these conditions:
>
> - Your build produces binary artifacts from managed code.
> - You have binary artifacts committed that you want to analyze with BinSkim.

Details of task configuration are shown in the following screenshot and list.



- Set the build configuration to Debug so that .pdb debug files are produced. BinSkim uses these files to map issues in the output binaries back to source code.
- To avoid researching and creating your own command line:
  - In the **Type** list, select **Basic**.
  - In the **Function** list, select **Analyze**.
- In **Target**, enter one or more specifiers for a file, directory, or filter pattern. These specifiers resolve to one or more binaries to be analyzed:
  - Multiple specified targets must be separated by a semicolon (;).
  - A specifier can be a single file or contain wildcards.
  - Directory specifications must always end with *.
  - Examples:

```
*.dll;*.exe
$(BUILD_STAGINGDIRECTORY)\*
$(BUILD_STAGINGDIRECTORY)\*.dll;$(BUILD_STAGINGDIRECTORY)\*.exe;
```

- If you select **Command Line** in the **Type** list, you need to run binskim.exe:
  - Make sure the first arguments to binskim.exe are the verb **analyze** followed by one or more path specifications. Each path can be either a full path or a path relative to the source directory.
  - Multiple target paths must be separated by a space.

- You can omit the **/o** or **/output** option. The output value is added for you or replaced.
- Standard command-line configurations are shown as follows.

```
analyze $(Build.StagingDirectory)\* --recurse --verbose
analyze *.dll *.exe --recurse --verbose
```

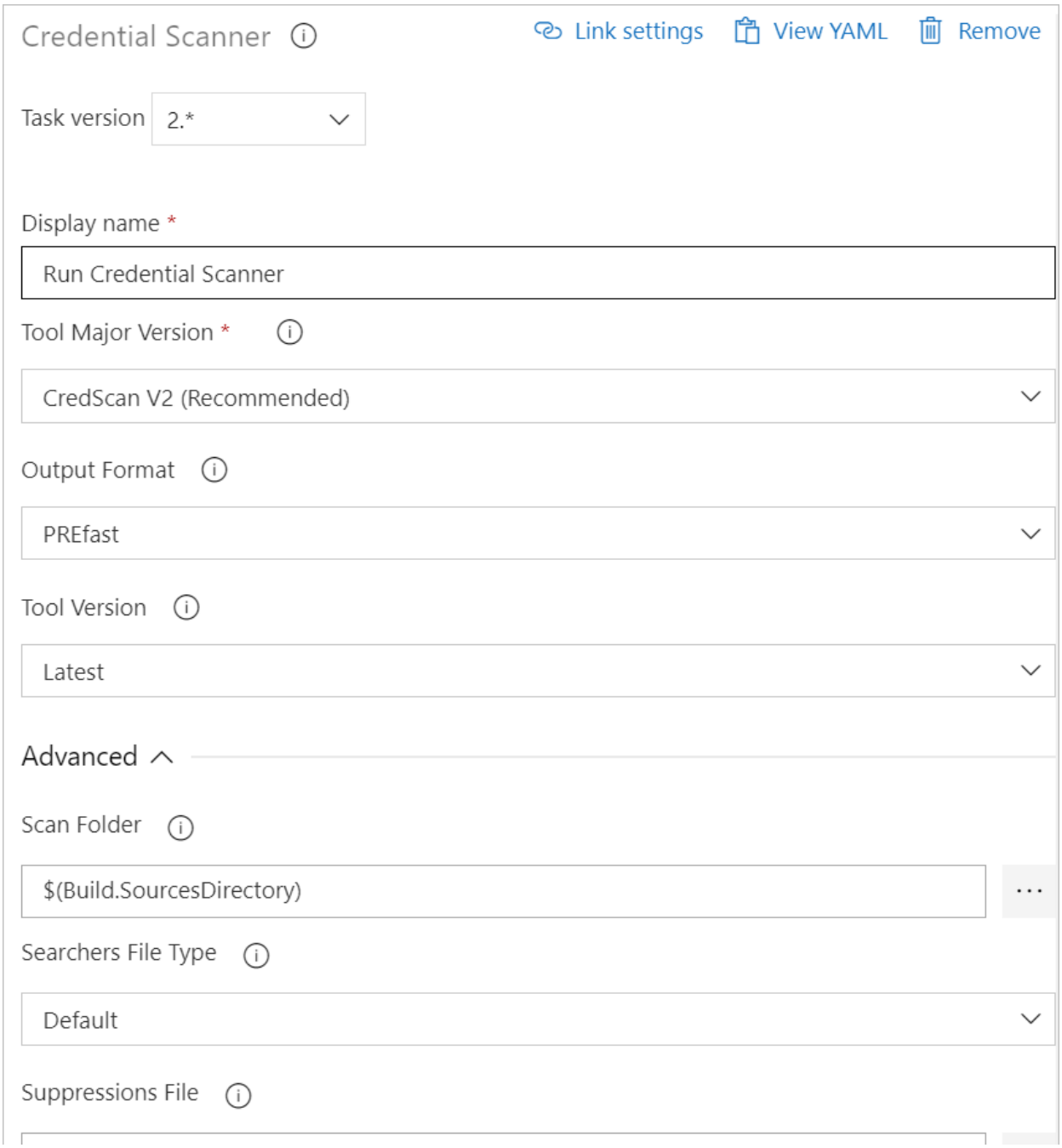
> **NOTE**
> The trailing * is important if you specify directories for the target.

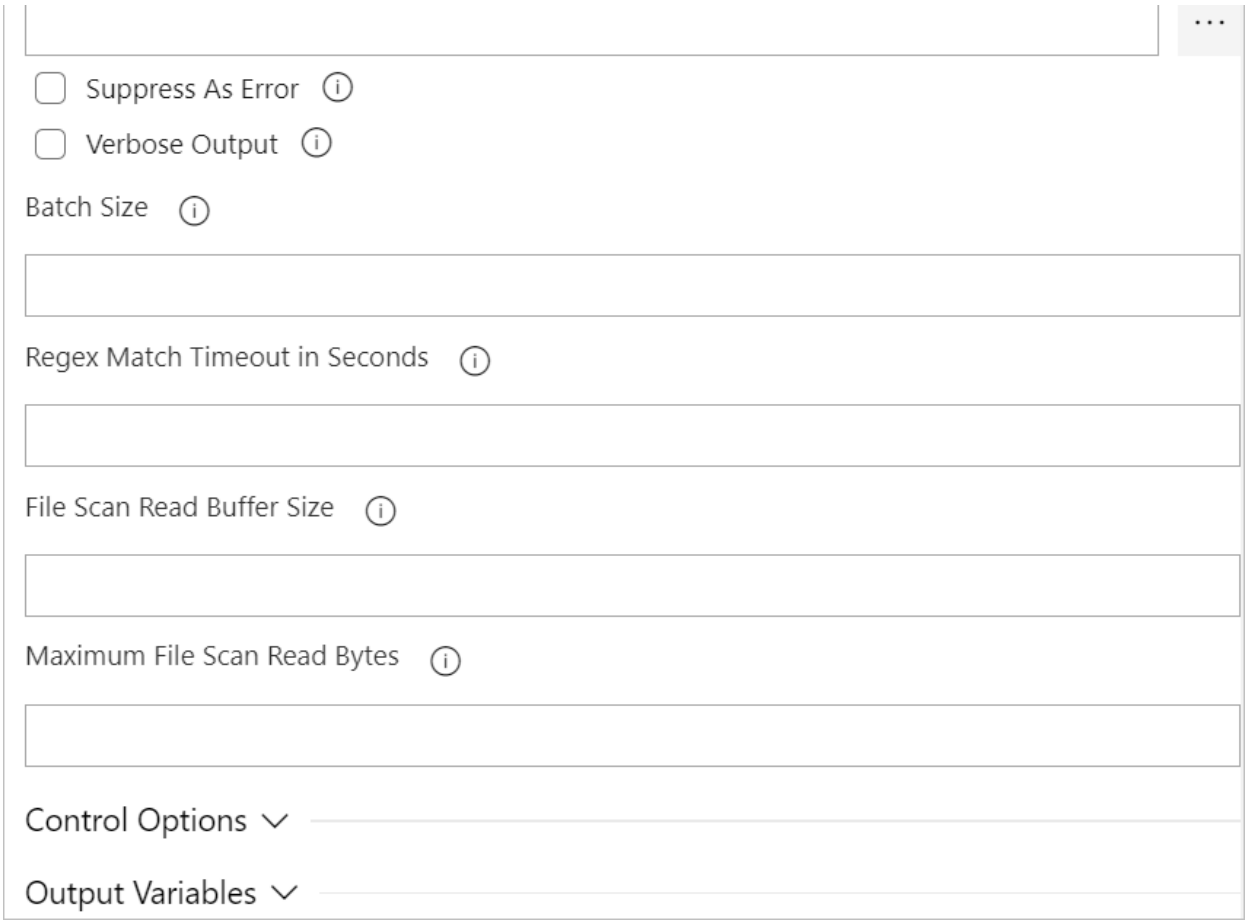
For more information on BinSkim command-line arguments, rules by ID, or exit codes, see the BinSkim User Guide.

For information about YAML configuration for this task, check our BinSkim YAML options

Credential Scanner task

Details of task configuration are shown in the following screenshot and list.





Available options include:

- **Display Name**: Name of the Azure DevOps Task. The default value is Run Credential Scanner
- **Tool Major Version**: Available values include **CredScan V2**, **CredScan V1**. We recommend customers to use the **CredScan V2** version.
- **Output Format**: Available values include **TSV**, **CSV**, **SARIF**, and **PREfast**.
- **Tool Version**: We recommend you select **Latest**.
- **Scan Folder**: The repository folder to be scanned.
- **Searchers File Type**: The options for locating the searchers file that is used for scanning.
- **Suppressions File**: A JSON file can suppress issues in the output log. For more information about suppression scenarios, see the FAQ section of this article.
- **Verbose Output**: Self-explanatory.
- **Batch Size**: The number of concurrent threads used to run Credential Scanner. The default value is 20. Possible values range from 1 through 2,147,483,647.
- **Match Timeout**: The amount of time in seconds to spend attempting a searcher match before abandoning the check.
- **File Scan Read Buffer Size**: The size in bytes of the buffer used while content is read. The default value is 524,288.
- **Maximum File Scan Read Bytes**: The maximum number of bytes to read from a file during content analysis. The default value is 104,857,600.
- **Control Options** > **Run this task**: Specifies when the task will run. Select **Custom conditions** to specify more complex conditions.
- **Version**: The build task version within Azure DevOps. This option isn't frequently used.

For information about YAML configuration for this task, check our Credential Scanner YAML options

Roslyn Analyzers task

NOTE

Before you can run the Roslyn Analyzers task, your build needs to meet these conditions:

- Your build definition includes the built-in MSBuild or VSBuild build task to compile C# or Visual Basic code. The analyzers task relies on the input and output of the built-in task to run the MSBuild compilation with Roslyn analyzers enabled.
- The build agent running this build task has Visual Studio 2017 version 15.5 or later installed, so that it uses compiler version 2.6 or later.

Details of task configuration are shown in the following list and note.

Available options include:

- **Ruleset**: Values are **SDL Required**, **SDL Recommended**, or your own custom rule set.
- **Analyzers Version**: We recommend you select **Latest**.
- **Compiler Warnings Suppressions File**: A text file with a list of warnings IDs that are suppressed.
- **Control Options** > **Run this task**: Specifies when the task will run. Choose **Custom conditions** to specify more complex conditions.

NOTE

- Roslyn Analyzers are integrated with the compiler and can be run only as part of csc.exe compilation. Hence, this task requires the compiler command that ran earlier in the build to be replayed or run again. This replay or run is done by querying Azure DevOps (formerly Visual Studio Team Services) for the MSBuild build task logs.

  There is no other way for the task to reliably get the MSBuild compilation command line from the build definition. We considered adding a freeform text box to allow users to enter their command lines. But then it would be hard to keep those command lines up-to-date and in sync with the main build.

  Custom builds require replaying the entire set of commands, not just compiler commands. In these cases, enabling Roslyn Analyzers isn't trivial or reliable.

- Roslyn Analyzers are integrated with the compiler. To be invoked, Roslyn Analyzers require compilation.

  This new build task is implemented by recompiling C# projects that were already built. The new task uses only the MSBuild and VSBuild build tasks in the same build or build definition as the original task. However, in this case the new task uses them with Roslyn Analyzers enabled.

  If the new task runs on the same agent as the original task, the new task's output overwrites the original task's output in the *s* sources folder. Although the build output is the same, we advise that you run MSBuild, copy output to the the artifacts staging directory, and then run Roslyn Analyzers.

For additional resources for the Roslyn Analyzers task, check out The Roslyn-based Analyzers on Microsoft Docs.

You can find the analyzer package installed and used by this build task on the NuGet page Microsoft.CodeAnalysis.FxCopAnalyzers.

For information about YAML configuration for this task, check our Roslyn Analyzers YAML options

TSLint task

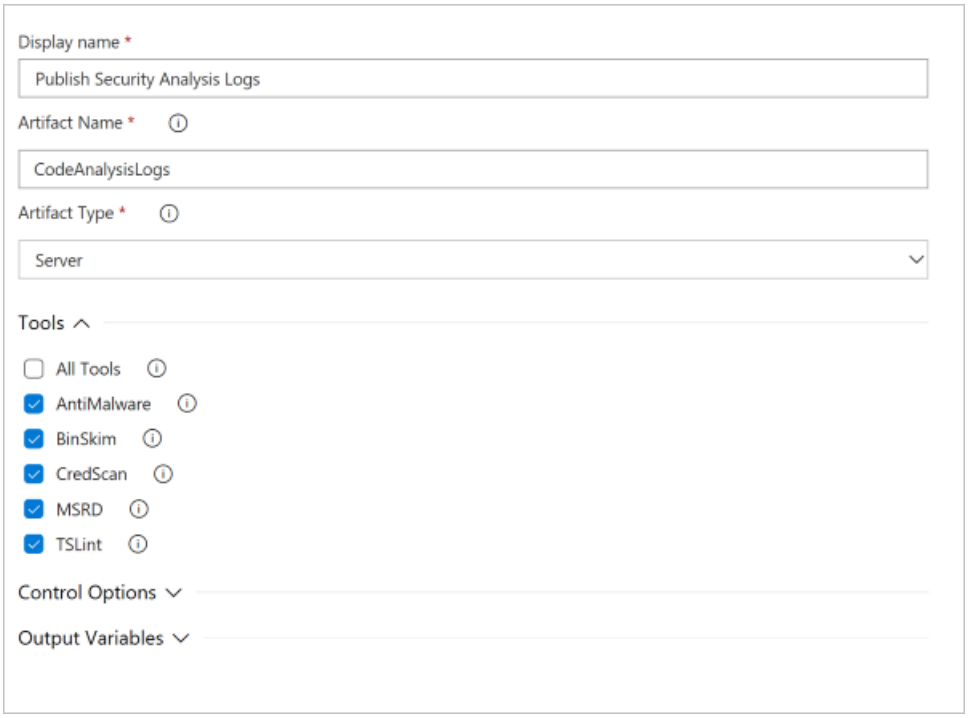
For More information about TSLint, go to the TSLint GitHub repo.

> **NOTE**
>
> As you might be aware, the TSLint GitHub repo home page says that TSLint will be deprecated sometime in 2019. Microsoft is investigating ESLint as an alternative task.

For information about YAML configuration for this task, check our TSLint YAML options

Publish Security Analysis Logs task

Details of task configuration are shown in the following screenshot and list.

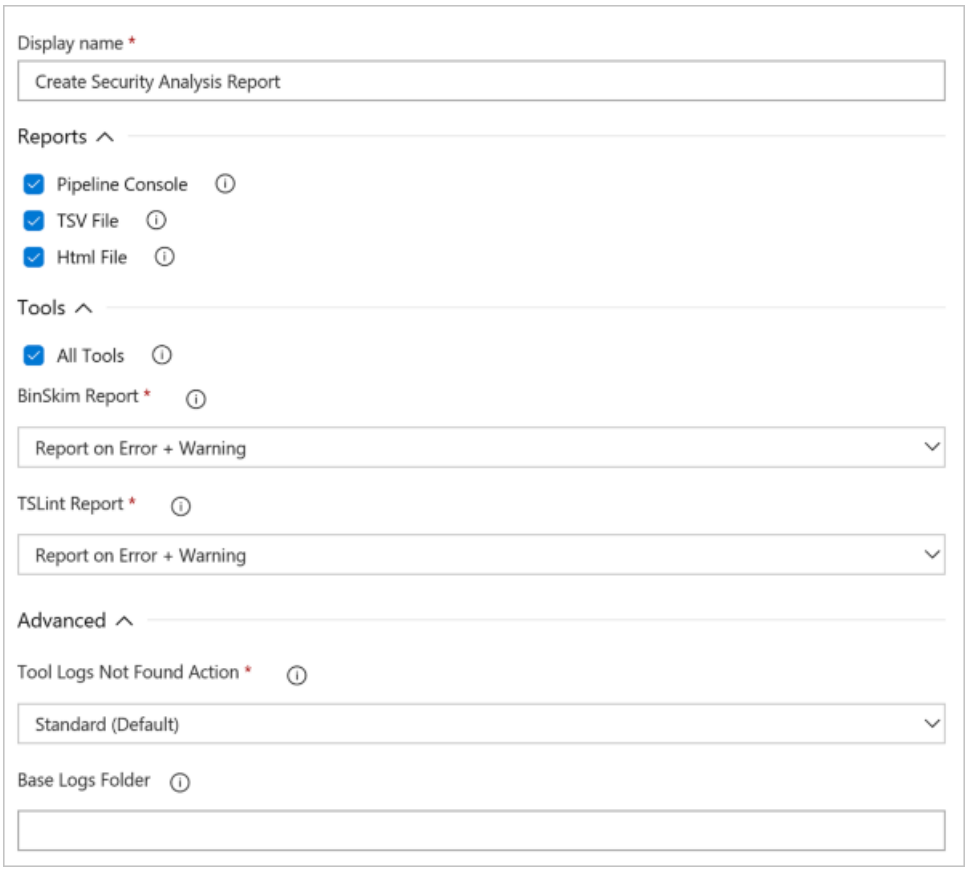


- **Artifact Name**: Any string identifier.
- **Artifact Type**: Depending on your selection, you can publish logs to your Azure DevOps Server or to a shared file that is accessible to the build agent.
- **Tools**: You can choose to preserve logs for specific tools, or you can select **All Tools** to preserve all logs.

For information about YAML configuration for this task, check our Publish Security Logs YAML options

Security Report task

Details of Security Report configuration are shown in the following screenshot and list.

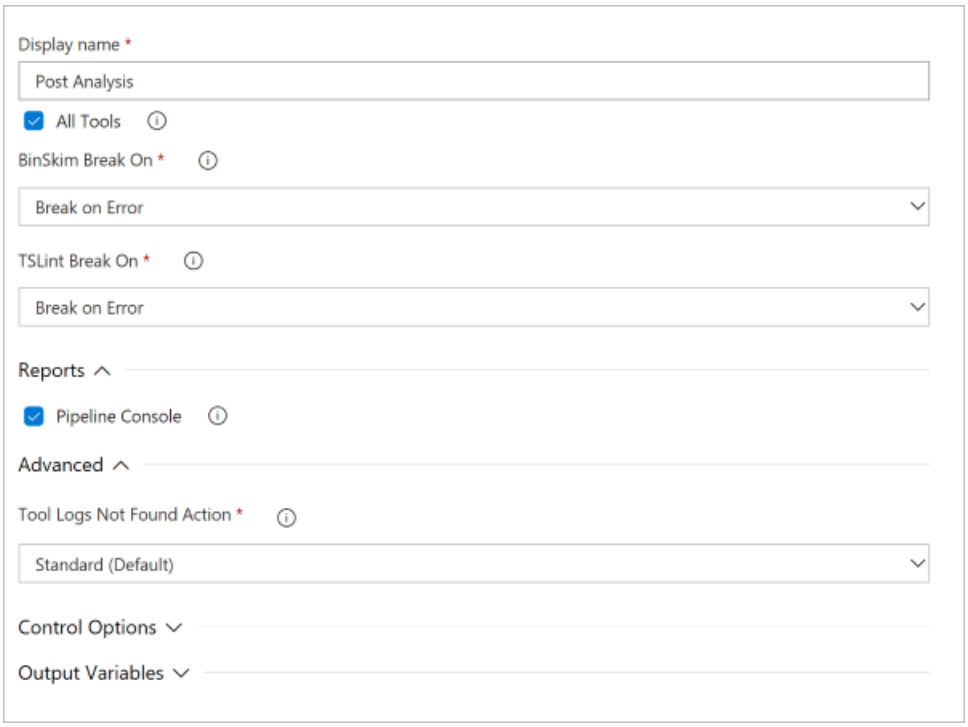


- **Reports**: Select any of the **Pipeline Console**, **TSV File**, and **Html File** formats. One report file is created for each selected format.
- **Tools**: Select the tools in your build definition for which you want a summary of detected issues. For each tool selected, there might be an option to select whether you see errors only or see both errors and warnings in the summary report.
- **Advanced Options**: If there are no logs for one of the tools selected, you can choose to log a warning or an error. If you log an error, the task fails.
- **Base Logs Folder**: You can customize the base logs folder where logs are to be found. But this option is typically not used.

For information about YAML configuration for this task, check our Security report YAML options

Post-Analysis task

Details of task configuration are shown in the following screenshot and list.



- **Tools**: Select the tools in your build definition for which you want to conditionally inject a build break. For each tool selected, there might be an option to select whether you want to break on errors only or on both errors and warnings.
- **Report**: You can optionally write the results that are causing the build break. The results are written to the Azure DevOps console window and log file.
- **Advanced Options**: If there are no logs for one of the tools selected, you can choose to log a warning or an error. If you log an error, the task fails.

For information about YAML configuration for this task, check our Post Analysis YAML options

Next steps

For information about YAML based configuration, refer to our YAML Configuration guide.

If you have further questions about the Security Code Analysis extension and the tools offered, check out our FAQ page.

YAML configuration options to customize the build tasks

6/15/2021 • 17 minutes to read • Edit Online

NOTE

Effective March 1, 2022, the Microsoft Security Code Analysis (MSCA) extension will be retired. Existing MSCA customers will retain their access to MSCA through March 1, 2022. Please refer to the OWASP Source Code Analysis Tools for alternative options in Azure DevOps. For customers planning to migrate to GitHub, you can check out GitHub Advanced Security.

This article lists all YAML configuration options available in each of the build tasks. The article starts with the tasks for security code analysis tools. It ends with the post-processing tasks.

Anti-Malware Scanner task

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
InputType	pickList	always	True	Basic	Basic, Custom	
ScanType	pickList	InputType = Basic	True	CustomScan	CustomScan, FullSystemScan, QuickScan, YourConfiguredScan	The scan type to use for the AntiMalware scan.
FileDirPath	filePath	ScanType = CustomScan	True	$(Build.StagingDirectory)		Indicates the file or directory to be scanned.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
DisableRemediation	boolean	ScanType = CustomScan	False	true		When checked: 1) File exclusions are ignored. 2) Archive files are scanned. 3) Actions are not applied after detection. 4) Event log entries are not written after detection. 5) Detections from the custom scan are not displayed in the user interface. 6) The console output will show the list of detections from the custom scan.
BootSectorScan	boolean	ScanType = CustomScan	False	false		If checked, it enables boot sector scanning.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
Arguments	string	InputType = Custom	True	-Scan -ScanType 3 -DisableRemediation -File $(Build.StagingDirectory)		The command line arguments, where the argument for -File is an absolute path, or a relative path to the $(Build.StagingDirectory) predefined on your build agent. Note: If you do not provide an argument for -File as the last argument, it defaults to $(Build.StagingDirectory). You can also provide your own arguments allowed by the MpCmdRun.exe tool. For more details about the command line arguments for this tool, please enter **-h** or **-?** in the Arguments field and execute the build task.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
EnableServices	boolean	always	True	false		If checked, it will try enabling the required services for Windows Update in case they are disabled. **NOTE**: Please make sure that the group policy does not disable the services and the account this build is running under has admin privileges.
SupportLogOnError	boolean	always	True	false		If checked, it will collect the support files for diagnosis when an error happened. This may take several minutes. **NOTE**: Please make sure that the account this build is running under has admin privileges.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
TreatSignatureUpdateFailureAs	pickList	always	True	Warning	Error, Standard, Warning	The log level used if the signature cannot be updated at run-time. When set to **Error**, a failure to update the signature will fail the build task. Note that it is common for the signature update to fail on hosted build agents, even though the signature may be relatively current (less than 3 hours old).
SignatureFreshness	pickList	always	True	UpToDate	OneDay, ThreeDays, TwoDays, UpToDate	The maximum allowed age for the AntiMalware signature. If the signature cannot be updated and is older than this value, the build task will behave according to the selected value in the **Validate Signatures Age As** field. Note: If you choose **Up-To-Date**, the signatures are allowed to be up to 3 hours old.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
TreatStaleSignatureAs	pickList	always	True	Error	Error, Standard, Warning	The log level used if the signature age is older than the selected **AntiMalware Signature Age**. An outdated signature may be treated as a **Warning** or **Informational** to continue with the AntiMalware scan, but this is not recommended.

BinSkim task

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
InputType	pickList	always	True	Basic	Basic, CommandLine	
arguments	string	InputType = CommandLine	True			Standard BinSkim command line arguments to execute. The output path will be removed and replaced. For more details about the command line arguments for this tool, please enter **help** in the Arguments field and execute the build task.
Function	pickList	InputType = Basic	True	analyze	analyze, dump, exportConfig, exportRules	

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
AnalyzeTarget	filePath	InputType = Basic && Function = analyze	True	$(Build.ArtifactStagingDirectory)*.dll; $(Build.ArtifactStagingDirectory)*.exe		One or more specifiers to a file, directory, or filter pattern that resolves to one or more binaries to analyze. (';' separated list)
AnalyzeSymPath	string	InputType = Basic && Function = analyze	False			Path to the symbols file for the target.
AnalyzeConfigPath	string	InputType = Basic && Function = analyze	False	default		Path to a policy file that will be used to configure analysis. Pass value of 'default' to use built-in settings.
AnalyzePluginPath	string	InputType = Basic && Function = analyze	False			Path to a plug-in that will be invoked against all targets in the analysis set.
AnalyzeRecurse	boolean	InputType = Basic && Function = analyze	False	true		Recurse into subdirectories when evaluating file specifier arguments.
AnalyzeVerbose	boolean	InputType = Basic && Function = analyze	False	false		Emit verbose output. The resulting comprehensive report is designed to provide appropriate evidence for compliance scenarios.
AnalyzeHashes	boolean	InputType = Basic && Function = analyze	False	false		Output SHA-256 hash of analysis targets when emitting SARIF reports.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
AnalyzeStatistics	boolean	InputType = Basic && Function = analyze	False	false		Generate timing and other statistics for analysis session.
AnalyzeEnvironment	boolean	InputType = Basic && Function = analyze	False	false		Log machine environment details of run to output file. WARNING: This option records potentially sensitive information (such as all environment variable values) to any emitted log.
ExportRulesOutputType	pickList	InputType = Basic && Function = exportRules	False	SARIF	SARIF, SonarQube	The type of rules descriptor file to output. This will be included in the BinSkim logs folder published by the Publish Security Analysis Logs build task.
DumpTarget	filePath	InputType = Basic && Function = dump	True	$(Build.ArtifactStagingDirectory)		One or more specifiers to a file, directory, or filter pattern that resolves to one or more binaries to analyze. (';' separated list)
DumpRecurse	boolean	InputType = Basic && Function = dump	False	true		Recurse into subdirectories when evaluating file specifier arguments.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
DumpVerbose	boolean	InputType = Basic && Function = dump	False	true		Emit verbose output. The resulting comprehensive report is designed to provide appropriate evidence for compliance scenarios.
toolVersion	pickList	always	False	Latest	1.5.0, Latest, LatestPreRelease	The version of the tool to run.

Credential Scanner task

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
outputFormat	pickList	always	False	pre	csv, pre, tsv	The output format of the Credential Scanner results file.
toolVersion	pickList	always	False	Latest	1.27.7, Latest, LatestPreRelease	The version of the tool to run.
scanFolder	filePath	always	False	$(Build.SourcesDirectory)		The folder in your repository to scan for credentials.
searchersFileType	pickList	always	False	Default	Custom, Default, DefaultAndCustom	Options to locate the searchers file used for scanning.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
searchersFile	filePath	searchersFileType == Custom OR searchersFileType == DefaultAndCustom	False			The Credential Scanner searchers configuration file of checks to execute. Multiple values can be included and used by providing a comma-separated list of paths to Credential Scanner searchers files.
suppressionsFile	filePath	always	False			The Credential Scanner suppressions file to use for suppressing issues in the output log.
suppressAsError	boolean	always	False	false		Suppressed matches will be output to the output file [-O]-matches.[-f] rather than the default suppressed output file [-O]-suppressed.[-f]. (Defaults to 'False')
verboseOutput	boolean	always	False	false		Output verbose information.
batchSize	string	always	False			The number of concurrent threads used to run Credential Scanners in parallel. (Default to 20) Value must be within the range of 1-2147483647.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
regexMatchTimeoutInSeconds	string	always	False			The amount of time in seconds to spend attempting a searcher match before abandoning the check. Adds `-Co RegexMatchTimeoutInSecon <Value>` to the command line.
fileScanReadBufferSize	string	always	False			Buffer size while reading content in bytes. (Defaults to 524288) Adds `-Co FileScanReadBufferSize= <Value>` to the command line.
maxFileScanReadBytes	string	always	False			Maximum number of bytes to read from a given file during content analysis. (Defaults to 104857600) Adds `-Co MaxFileScanReadBytes= <Value>` to the command line.

Roslyn Analyzers task

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
userProvideBuildInfo	pickList	always	True	auto	auto, msBuildInfo	Options for a user to provide the MSBuild version, MSBuild architecture, and build commandline for Roslyn analysis. If **Auto** is selected, this task will retrieve the build info from the previous **MSBuild**, **VSBuild**, and/or **.NET Core** (for build) tasks in the same pipeline.
msBuildVersion	pickList	userProvideBuildInfo == msBuildInfo	True	16.0	15.0, 16.0	The MSBuild version.
msBuildArchitecture	pickList	userProvideBuildInfo == msBuildInfo	True	x86	DotNetCore, x64, x86	The MSBuild architecture. Note: If the build commandline calls **dotnet.exe build**, choose the **Via .NET Core** option.
msBuildCommandline	string	userProvideBuildInfo == msBuildInfo	True			The complete build commandline for compiling your solution or projects. Notes: The commandline should begin with a full path to **MSBuild.exe** or **dotnet.exe**. The command will run with $(Build.SourcesDirectory) as the working directory.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
rulesetName	pickList	always	False	Recommended	Custom, None, Recommended, Required	A named ruleset to use. If `Ruleset Configured In Your Visual Studio Project File(s)` is chosen, the ruleset preconfigured in your VS project file(s) will be used. If `Custom` is chosen, a custom ruleset path option can be set.
rulesetVersion	pickList	rulesetName == Required OR rulesetName == Recommended	False	Latest	8.0, 8.1, 8.2, Latest, LatestPreRelease	The version of the chosen SDL ruleset.
customRuleset	string	rulesetName = Custom	False			An accessible path to a ruleset to use. Relative paths will be normalized to the root of the source repository (`$(Build.SourcesDirectory`). If the ruleset specifies `Rules` with `Actions` set to `Error`, the build task will fail. To use a ruleset that does this, please check `Continue on error` in the build task's `Control Options` .

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
microsoftAnalyzersVersion	pickList	always	False	Latest	2.9.3, 2.9.4, 2.9.6, Latest, LatestPreRelease	The version of the Microsoft.CodeAnalysis.FxCopAnalyzers package to run.
suppressionFileForCompilerWarnings	filePath	always	False			A suppressions file to suppress C# and VB compiler warnings. A plain-text file with each warning ID listed one a separate line. For compiler warnings, specify only the numeric part of the warning identifier. For example 1018 will suppress CS1018, and CA1501 will suppress CA1501. A relative file path will be appended to the root of the source repository (`$(Build.SourcesDirectory`).

TSLint task

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
RuleLibrary	pickList	always	True	tslint	custom, microsoft, tslint	All results include the rules shipped with the selected version of TSLint (**Base Only**). **Base Only** - Only the rules shipped with

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
						TSLint. **Include Microsoft Rules** - Downloads tslint-microsoft-contrib and includes its rules to be available for use in the TSLint run. Choosing this option hides the `Type Checking` checkbox, as it is required by Microsoft's rules and will automatically be used. It also unhides the `Microsoft Contribution Version` field, allowing a version of the `tslint-microsoft-contrib` from npm to be selected. **Include Custom Rules** - Unhides the `Rules Directory` field, which accepts an accessible path to a directory of TSLint rules to be available for use in the TSLint run. **Note:** The default value has changed to tslint, as many users have experienced issues configuring the Microsoft ruleset. For specific version configuration, please see

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
RulesDirectory	string	RuleLibrary == custom	True			An accessible directory containing additional TSLint rules to be available for use in the TSLint run.

tslint-microsoft-contrib on GitHub.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
Ruleset	pickList	RuleLibrary != microsoft	True	tsrecommended	custom, tslatest, tsrecommended	Defines the rules to run against TypeScript files. **tslint:latest** - Extends `tslint:recommended` and is continuously updated to include configuration for the latest rules in every TSLint release. Using this config may introduce breaking changes across minor releases as a new rules are enabled which cause lint failures in your code. When TSLint reaches a major version bump, `tslint:recommended` will be updated to be identical to `tslint:latest`. **tslint:recommended** - A stable, somewhat opinionated set of rules which TSLint encourages for general TypeScript programming. This configuration follows `semver`, so it will *not* have breaking changes across minor or patch releases.
RulesetMicrosoft	pickList	RuleLibrary == microsoft	True	mssdlrequired	custom,	Defines the

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
					msrecommended, mssdlrecommended, mssdlrequired, tslatest, tsrecommended	rules to run against TypeScript files. **microsoft:sdl-required** - Run all of the available checks provided by tslint and the tslint-microsoft-contrib rules that satisfy the *required* Security Development Lifecycle (SDL) policies. **microsoft:sdl-recommended** - Run all of the available checks provided by tslint and the tslint-microsoft-contrib rules that satisfy the *required and recommended* Security Development Lifecycle (SDL) policies. **microsoft:recommended** All checks that are recommended by the creators of the tslint-microsoft-contrib rules. This includes security and non-security checks. **tslint:latest** - Extends `tslint:recommended` and is continuously updated to include configuration for the latest rules in every TSLint release.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
						Using this config may introduce breaking changes across minor releases as a new rules are enabled which cause lint failures in your code. When TSLint reaches a major version bump, `tslint:recommended` will be updated to be identical to `tslint:latest`. **tslint:recommended** - A stable, somewhat opinionated set of rules which TSLint encourages for general TypeScript programming. This configuration follows `semver`, so it will *not* have breaking changes across minor or patch releases.
RulesetFile	string	Ruleset == custom OR RulesetMicrosoft == custom	True			A configuration file specifying which rules to run. The path to the config will be added as the path for custom rules.
FileSelectionType	pickList	always	True	fileGlob	fileGlob, projectFile	

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
Files	string	FileSelectionType == fileGlob	True	***.ts		A file glob that determines which file(s) to process. Paths are relative to the `Build.SourcesDirectory` value. Microsoft's Contribution library requires the use of a project file. If you are using Microsoft's Contribution library with the `File Glob Pattern` option, a project file will be generated for you.
ECMAScriptVersion	pickList	FileSelectionType == fileGlob && RuleLibrary == microsoft	True	ES3	ES2015, ES2016, ES2017, ES3, ES5, ES6, ESNext	The target version of ECMAScript configured with your TypeScript compiler. When using a project file, this is the compilerOptions.target field of your TypeScript tsconfig.json file.
Project	string	FileSelectionType == projectFile	True			Path to a tsconfig.json file that specifies TypeScript files to run TSLint on. Paths are relative to the `Build.SourcesDirectory` value.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
TypeCheck	boolean	RuleLibrary != microsoft && FileSelectionType == projectFile	False	true		Enables the type checker when running linting rules.
ExcludeFiles	string	always	False			A glob which indicates files to exclude from linting. Paths are relative to the `Build.SourcesDirectory` value. Multiple values can be specified separated by semicolon.
OutputFormat	pickList	always	True	json	checkstyle, codeFrame, filesList, json, msbuild, pmd, prose, stylish, verbose, vso	The formatter to use to generate output. Note that the JSON format is compatible with Post Analysis.
NodeMemory	string	always	False			An explicit amount of memory in MBs to allocate to node for running TSLint. Example: 8000 Maps to the `--max_old_space=<value>` CLI option for node, which is a `v8 option`.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
ToolVersion	pickList	RuleLibrary != microsoft	True	latest	4.0.0, 4.0.1, 4.0.2, 4.1.0, 4.1.1, 4.2.0, 4.3.0, 4.3.1, 4.4.0, 4.4.1, 4.4.2, 4.5.0, 4.5.1, 5.0.0, 5.1.0, 5.2.0, 5.3.0, 5.3.2, 5.4.0, 5.4.1, 5.4.2, 5.4.3, 5.5.0, latest	The version of TSLint to download and run.
TypeScriptVersion	pickList	always	True	latest	0.8.0, 0.8.1, 0.8.2, 0.8.3, 0.9.0, 0.9.1, 0.9.5, 0.9.7, 1.0.0, 1.0.1, 1.3.0, 1.4.1, 1.5.3, 1.6.2, 1.7.3, 1.7.5, 1.8.0, 1.8.10, 1.8.2, 1.8.5, 1.8.6, 1.8.7, 1.8.9, 1.9.0, 2.0.0, 2.0.10, 2.0.2, 2.0.3, 2.0.6, 2.0.7, 2.0.8, 2.0.9, 2.1.1, 2.1.4, 2.1.5, 2.1.6, 2.2.0, 2.2.1, custom, latest	The version of typescript to download and use. **Note:** This needs to be the same version of TypeScript as is used to compile your code.
TypeScriptVersionCustom	string	TypeScriptVersion == custom	True	latest		The version of typescript to download and use. **Note:** This needs to be the same version of TypeScript as is used to compile your code.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
MicrosoftContribVersion	pickList	RuleLibrary == microsoft		latest	4.0.0, 4.0.1, 5.0.0, 5.0.1, latest	The version of tslint-microsoft-contrib (SDL Rules) to download and use. **Note:** The version of tslint will be chosen that is compatible with the version chosen for tslint-microsoft-contrib. Updates to tslint-microsoft-contrib will be gated by this build task, until a period of testing can occur.

Publish Security Analysis Logs task

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
ArtifactName	string	always	True	CodeAnalysis Logs		The name of the artifact to create.
ArtifactType	pickList	always	True	Container	Container, FilePath	The type of the artifact to create.
TargetPath	string	ArtifactType = FilePath	False	\my\share$(Build.DefinitionName) $(Build.BuildNumber)		The file share to which to copy the files
AllTools	boolean	always	True	true		Publish results generated by all Secure Development Tools build tasks.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
AntiMalware	boolean	AllTools = false	True	true		Publish results generated by AntiMalware build tasks.
BinSkim	boolean	AllTools = false	True	true		Publish results generated by BinSkim build tasks.
CredScan	boolean	AllTools = false	True	true		Publish results generated by Credential Scanner build tasks.
RoslynAnalyzers	boolean	AllTools = false	True	false		Publish results generated by Roslyn Analyzers build tasks.
TSLint	boolean	AllTools = false	True	true		Publish results generated by TSLint build tasks. Note that only TSLint logs in the JSON format are supported for reports. If you have chosen a different format, please update your TSLint build task accordingly.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
ToolLogsNotFoundAction	picklist	always	True	Standard	Error, None, Standard, Warning	The action to take when logs for a selected tool (or any tool if All Tools is checked) are not found, implying the tool was not run. **Options:** **None:** Message is written to the verbose output stream accessible only by setting the VSTS variable **system.debug** to **true**. **Standard:** (Default) Writes a standard output message that no logs were found for the tool. **Warning:** Writes a yellow warning message that no logs were found for the tool, which shows up on the build summary page as a warning. **Error:** Writes a red error message and throws an exception, breaking the build. Use this option to ensure with individual tool choices to ensure which tools ran.

Security Report task

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
VstsConsole	boolean	always	False	true		Write results to the Pipeline Console.
TsvFile	boolean	always	False	true		Generate a tsv file (tab separated values) with one line per found result and tabs separating info for the result.
HtmlFile	boolean	always	False	true		Generate an html report file.
AllTools	boolean	always	True	false		Report results generated by all Secure Development Tools build tasks.
BinSkim	boolean	AllTools = false	True	false		Report results generated by BinSkim build tasks.
BinSkimBreakOn	pickList	AllTools = true OR BinSkim = true	True	Error	Error, WarningAbove	The level of results to report.
CredScan	boolean	AllTools = false	True	false		Report results generated by Credential Scanner build tasks.
RoslynAnalyzers	boolean	AllTools = false	True	false		Report results generated by Roslyn Analyzer build tasks.
RoslynAnalyzersBreakOn	pickList	AllTools = true OR RoslynAnalyzers = true	True	Error	Error, WarningAbove	The level of results to report.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
TSLint	boolean	AllTools = false	True	false		Report results generated by TSLint build tasks. Note that only TSLint logs in the JSON format are supported for reports. If you have chosen a different format, please update your TSLint build task accordingly.
TSLintBreakOn	pickList	AllTools = true OR TSLint = true	True	Error	Error, WarningAbove	The level of results to report.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
ToolLogsNotFoundAction	picklist	always	True	Standard	Error, None, Standard, Warning	The action to take when logs for a selected tool (or any tool if All Tools is checked) are not found, implying the tool was not run. **Options:** **None:** Message is written to the verbose output stream accessible only by setting the VSTS variable **system.debug** to **true**. **Standard:** (Default) Writes a standard output message that no logs were found for the tool. **Warning:** Writes a yellow warning message that no logs were found for the tool, which shows up on the build summary page as a warning. **Error:** Writes a red error message and throws an exception, breaking the build. Use this option to ensure with individual tool choices to ensure which tools ran.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
CustomLogsFolder	string	always	False			The base folder where analysis tool logs are located; the individual log files will be in subfolders named after each tool, under this path.

Post-Analysis task

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
AllTools	boolean	always	True	false		Break the build if any issues are found by any Microsoft Security Code Analysis build task.
BinSkim	boolean	AllTools = false	True	false		Break the build if any BinSkim issues are found, according to the Break On option you selected.
BinSkimBreakOn	pickList	AllTools = true OR BinSkim = true	True	Error	Error, WarningAbove	The level of issues to break the build.
CredScan	boolean	AllTools = false	True	false		Break the build if any Credential Scanner issues are found.
RoslynAnalyzers	boolean	AllTools = false	True	false		Break the build if any Roslyn Analyzers issues are found.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
RoslynAnalyzersBreakOn	pickList	AllTools = true OR RoslynAnalyzers = true	True	Error	Error, WarningAbove	The level of issues to break the build.
TSLint	boolean	AllTools = false	True	false		Break the build if any TSLint issues are found. Note that only TSLint logs in the JSON format are supported for post analysis. If you have chosen a different format, please update your TSLint build task accordingly.
TSLintBreakOn	pickList	AllTools = true OR TSLint = true	True	Error	Error, WarningAbove	The level of issues to break the build.
VstsConsole	boolean	always	False	true		Write results to the Pipeline Console.

INPUTTYPE	TYPE	APPLICABLE	REQUIRED	DEFAULT VALUE	OPTIONS (FOR PICKLISTS)	DESCRIPTION
ToolLogsNotFoundAction	picklist	always	True	Standard	Error, None, Standard, Warning	The action to take when logs for a selected tool (or any tool if All Tools is checked) are not found, implying the tool was not run. **Options:** **None:** Message is written to the verbose output stream accessible only by setting the VSTS variable **system.debug** to **true**. **Standard:** (Default) Writes a standard output message that no logs were found for the tool. **Warning:** Writes a yellow warning message that no logs were found for the tool, which shows up on the build summary page as a warning. **Error:** Writes a red error message and throws an exception, breaking the build. Use this option to ensure with individual tool choices to ensure which tools ran.

Next steps

If you have further questions about the Security Code Analysis extension and the tools offered, check out our FAQ page.

Microsoft Security Code Analysis releases and roadmap

6/15/2021 • 2 minutes to read • Edit Online

NOTE

Effective March 1, 2022, the Microsoft Security Code Analysis (MSCA) extension will be retired. Existing MSCA customers will retain their access to MSCA through March 1, 2022. Please refer to the OWASP Source Code Analysis Tools for alternative options in Azure DevOps. For customers planning to migrate to GitHub, you can check out GitHub Advanced Security.

Microsoft Security Code Analysis team in partnership with Developer Support is proud to announce recent and upcoming enhancements to our MSCA extension.

Credential Scanner v2.0: Released in April 2020

Innovations & Improvements

- **Core Engine**
  - Average performance upgrade of 25% with near linear run times
  - Context/evidence based searching and ranking for increased accuracy
  - Improvements to general password detections and matching logic for obvious placeholders (for example, fakePassword)
- **Coverage** - Support for 25+ secret types including the following top requested:
  - Fabric account certificate Passphrase
  - Client Secret/API Key
  - HTTP authorization header
  - Amazon S3 Client Secret Access Key
  - Azure Active Directory Client Access Token
  - Azure Function Master/API Key
  - Power BI Access Key
  - Azure Resource Manager template password pattern
- **Outputs**
  - Support for SARIF 2.1 and CSV file output file formats

BinSkim v1.6.0: Released in April 2020

Improvements

- FEATURE: Update to final SARIF v2 (version 2.1.16). This update enables results caching when passing --hashes on the command-line, a significant performance improvement when recursively analyzing directories with multiple copies of scan targets.
- BUG FIX: Fix typo in BA2021.DoNotMarkWritableSectionsAsExecutable output.
- PERFORMANCE: Eliminate PDB loading for all non-mixed-mode for managed assemblies, including IL Library (ahead of time compiled) binaries.
- FALSE NEGATIVE FIX: Verify that a PDB placed alongside a binary actually matches the binary under analysis

- FEATURE: Provide --local-symbol-directories argument to specify additional (local, non-symbol-server) PDB look-up locations
- FALSE POSITIVE FIX: Skip PDB-driven analysis for the generated .NET core native bootstrap exe (which is not user-controllable code).

What's next in Q3 CY20?

- Java Security Analysis tool
- Python Security Analysis tool
- ES Lint to replace TS Lint for TypeScript and JavaScript
- Resource Manager Templates Analysis tool

Tool Deprecation Notification

Microsoft Security Risk Detection (MSRD) is deprecated on June 26 2020.

The deprecated MSRD fuzzing service will be replaced with an open source self-hosted developer fuzzing platform for Azure. This platform is currently being developed and tested in partnership with many of Microsoft's core product teams. This fuzzing platform will integrate sanitizers and allow for adaptive, learning fuzz tests built into CI/CD pipelines that grow over time with software projects. The Open Source release of this platform is scheduled for the latter half of 2020.

Next steps

For instructions on how to onboard and install Microsoft Security Code Analysis, refer to our Onboarding and installation guide.

If you have more questions about the extension and the tools offered, check out our FAQ page.

Microsoft Threat Modeling Tool

3/5/2021 • 2 minutes to read • Edit Online

The Threat Modeling Tool is a core element of the Microsoft Security Development Lifecycle (SDL). It allows software architects to identify and mitigate potential security issues early, when they are relatively easy and cost-effective to resolve. As a result, it greatly reduces the total cost of development. Also, we designed the tool with non-security experts in mind, making threat modeling easier for all developers by providing clear guidance on creating and analyzing threat models.

The tool enables anyone to:

- Communicate about the security design of their systems
- Analyze those designs for potential security issues using a proven methodology
- Suggest and manage mitigations for security issues

Here are some tooling capabilities and innovations, just to name a few:

- **Automation:** Guidance and feedback in drawing a model
- **STRIDE per Element:** Guided analysis of threats and mitigations
- **Reporting:** Security activities and testing in the verification phase
- **Unique Methodology:** Enables users to better visualize and understand threats
- **Designed for Developers and Centered on Software:** many approaches are centered on assets or attackers. We are centered on software. We build on activities that all software developers and architects are familiar with -- such as drawing pictures for their software architecture
- **Focused on Design Analysis:** The term "threat modeling" can refer to either a requirements or a design analysis technique. Sometimes, it refers to a complex blend of the two. The Microsoft SDL approach to threat modeling is a focused design analysis technique

Next steps

The table below contains important links to get you started with the Threat Modeling Tool: See also: System requirements

STEP	DESCRIPTION
1	Download the Threat Modeling Tool
2	Read Our getting started guide
3	Get familiar with the features
4	Learn about generated threat categories
5	Find mitigations to generated threats

Resources

Here are a few older articles still relevant to threat modeling today:

- Article on the Importance of Threat Modeling

- Training Published by Trustworthy Computing

Check out what a few Threat Modeling Tool experts have done:

- Threats Manager
- Simone Curzi Security Blog

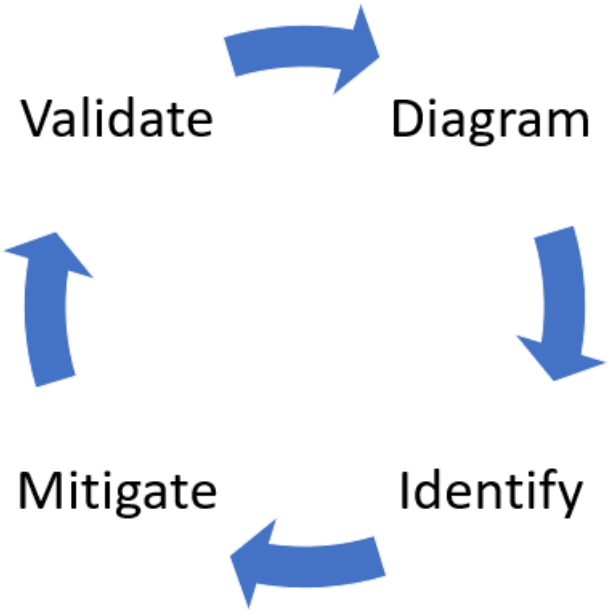
Getting started with the Threat Modeling Tool

3/5/2021 • 7 minutes to read • Edit Online

The Microsoft Threat Modeling Tool 2018 was released as GA in September 2018 as a free **click-to-download**. The change in delivery mechanism allows us to push the latest improvements and bug fixes to customers each time they open the tool, making it easier to maintain and use. This article takes you through the process of getting started with the Microsoft SDL threat modeling approach and shows you how to use the tool to develop great threat models as a backbone of your security process.

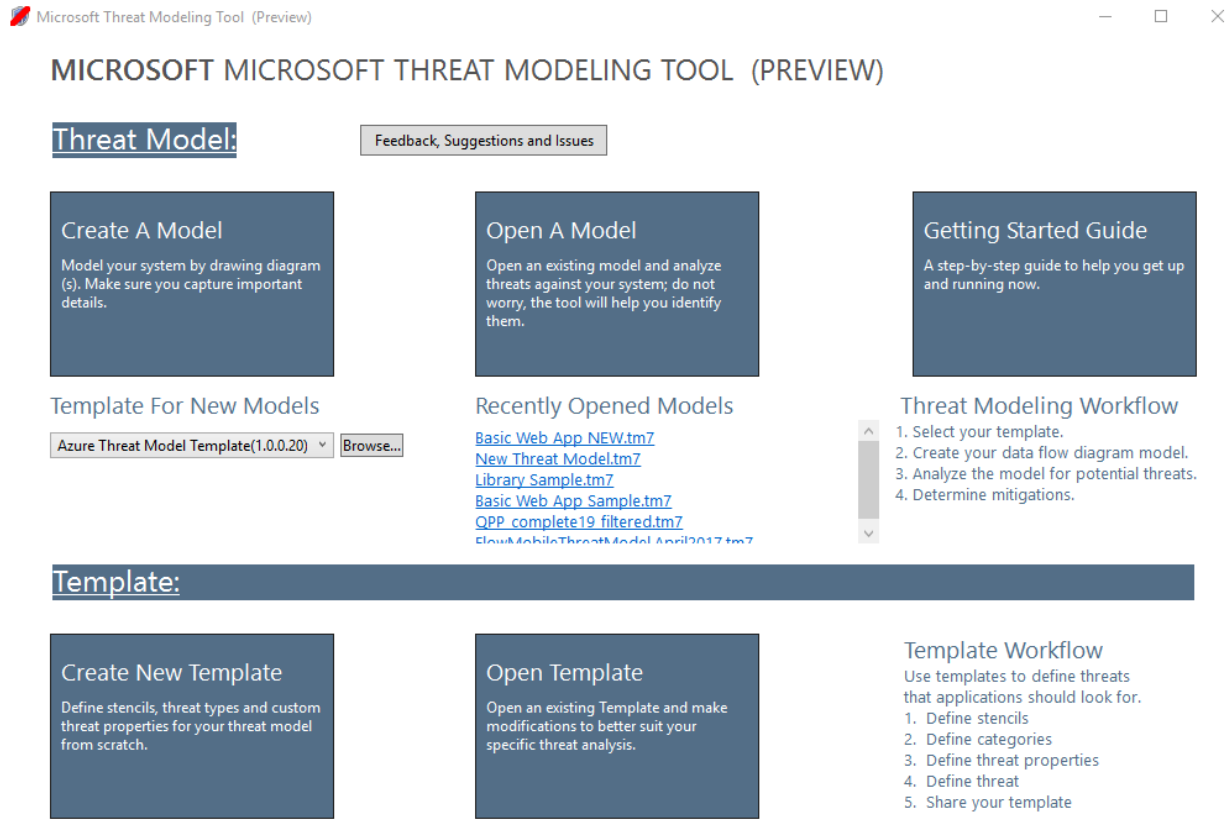
This article builds on existing knowledge of the SDL threat modeling approach. For a quick review, refer to **Threat Modeling Web Applications** and an archived version of **Uncover Security Flaws Using the STRIDE Approach** MSDN article published in 2006.

To quickly summarize, the approach involves creating a diagram, identifying threats, mitigating them and validating each mitigation. Here's a diagram that highlights this process:



Starting the threat modeling process

When you launch the Threat Modeling Tool, you'll notice a few things, as seen in the picture:



Threat model section

COMPONENT	DETAILS
Feedback, Suggestions and Issues Button	Takes you the **MSDN Forum** for all things SDL. It gives you an opportunity to read through what other users are doing, along with workarounds and recommendations. If you still can't find what you're looking for, email tmtextsupport@microsoft.com for our support team to help you
Create a Model	Opens a blank canvas for you to draw your diagram. Make sure to select which template you'd like to use for your model
Template for New Models	You must select which template to use before creating a model. Our main template is the Azure Threat Model Template, which contains Azure-specific stencils, threats and mitigations. For generic models, select the SDL TM Knowledge Base from the drop-down menu. Want to create your own template or submit a new one for all users? Check out our **Template Repository** GitHub Page to learn more
Open a Model	Opens previously saved threat models. The Recently Opened Models feature is great if you need to open your most recent files. When you hover over the selection, you'll find 2 ways to open models: • Open From this Computer – classic way of opening a file using local storage • Open from OneDrive – teams can use folders in OneDrive to save and share all their threat models in a single location to help increase productivity and collaboration

COMPONENT	DETAILS
Getting Started Guide	Opens the **Microsoft Threat Modeling Tool** main page

Template section

COMPONENT	DETAILS
Create New Template	Opens a blank template for you to build on. Unless you have extensive knowledge in building templates from scratch, we recommend you to build from existing ones
Open Template	Opens existing templates for you to make changes to

The Threat Modeling Tool team is constantly working to improve tool functionality and experience. A few minor changes might take place over the course of the year, but all major changes require rewrites in the guide. Refer to it often to ensure you get the latest announcements.

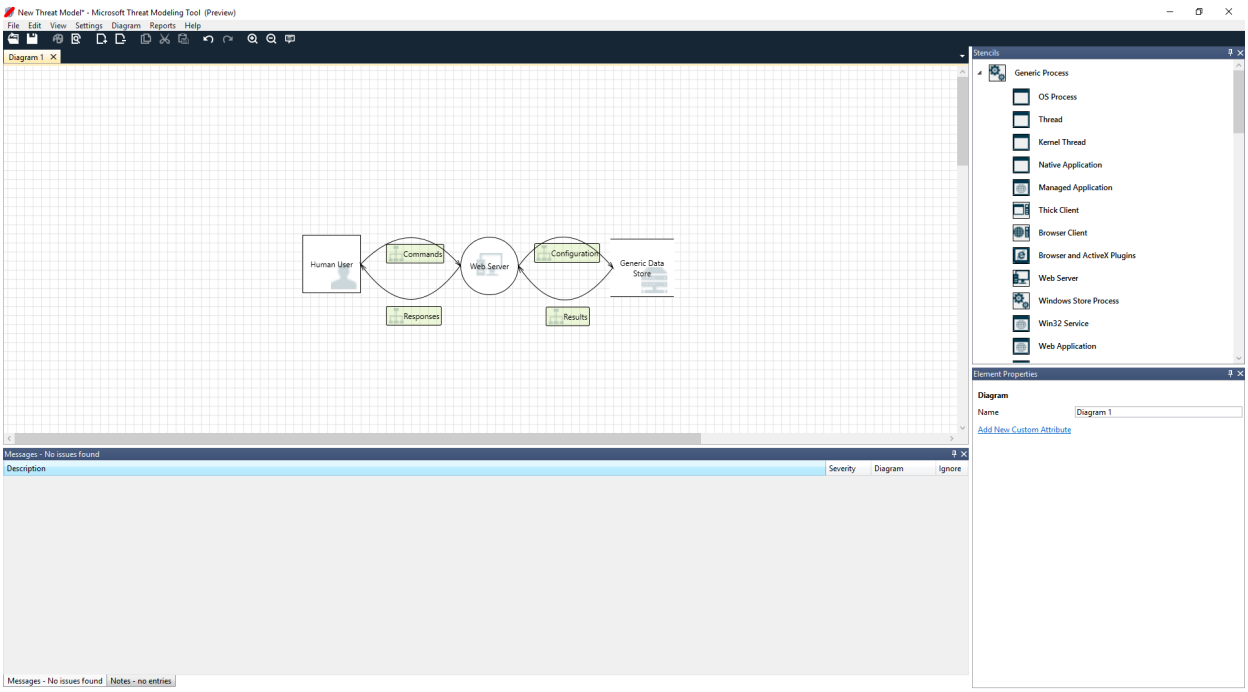
Building a model

In this section, we follow:

- Cristina (a developer)
- Ricardo (a program manager) and
- Ashish (a tester)

They are going through the process of developing their first threat model.

> Ricardo: Hi Cristina, I worked on the threat model diagram and wanted to make sure we got the details right. Can you help me look it over? Cristina: Absolutely. Let's take a look. Ricardo opens the tool and shares his screen with Cristina.



> Cristina: Ok, looks straightforward, but can you walk me through it? Ricardo: Sure! Here is the breakdown:
>
> - Our human user is drawn as an outside entity—a square

- They're sending commands to our Web server—the circle
- The Web server is consulting a database (two parallel lines)

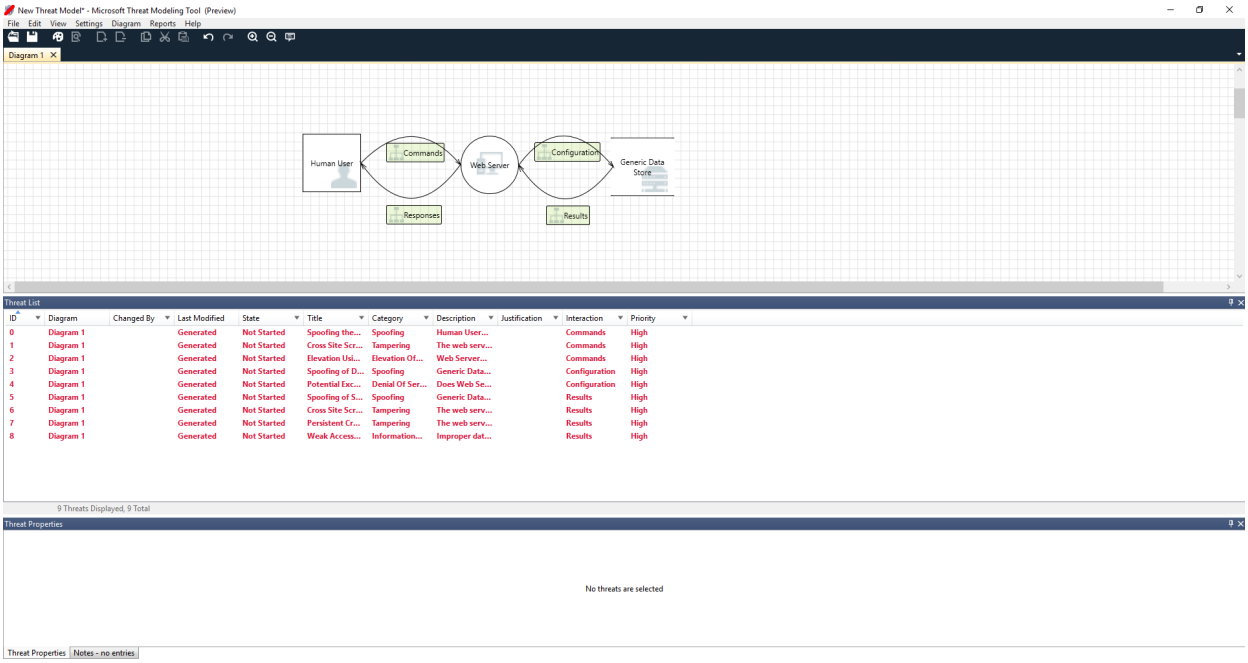
What Ricardo just showed Cristina is a DFD, short for **Data Flow Diagram**. The Threat Modeling Tool allows users to specify trust boundaries, indicated by the red dotted lines, to show where different entities are in control. For example, IT administrators require an Active Directory system for authentication purposes, so the Active Directory is outside of their control.

> Cristina: Looks right to me. What about the threats? Ricardo: Let me show you.

Analyzing threats

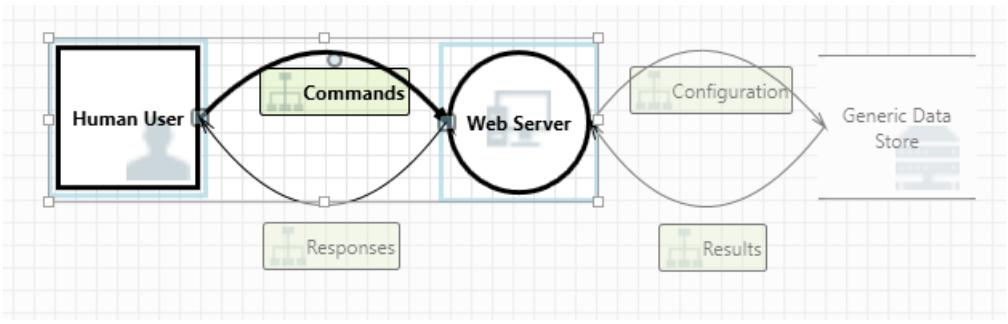
Once he clicks on the analysis view from the icon menu selection (file with magnifying glass), he is taken to a list of generated threats the Threat Modeling Tool found based on the default template, which uses the SDL approach called **STRIDE (Spoofing, Tampering, Info Disclosure, Repudiation, Denial of Service and Elevation of Privilege)**. The idea is that software comes under a predictable set of threats, which can be found using these 6 categories.

This approach is like securing your house by ensuring each door and window has a locking mechanism in place before adding an alarm system or chasing after the thief.

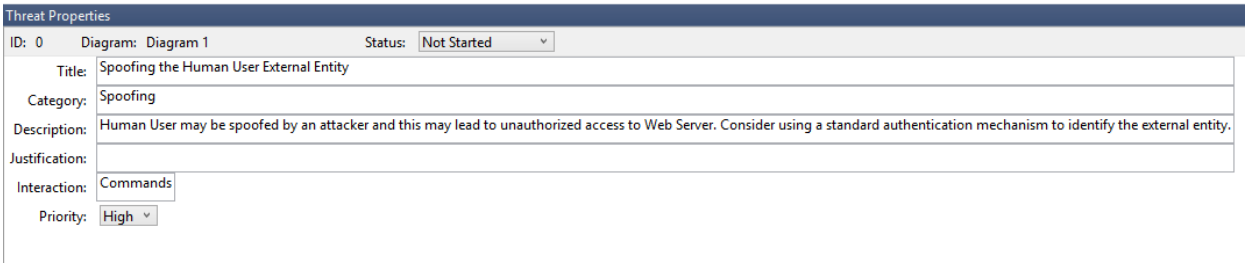


Ricardo begins by selecting the first item on the list. Here's what happens:

First, the interaction between the two stencils is enhanced



Second, additional information about the threat appears in the Threat Properties window



The generated threat helps him understand potential design flaws. The STRIDE categorization gives him an idea on potential attack vectors, while the additional description tells him exactly what's wrong, along with potential ways to mitigate it. He can use editable fields to write notes in the justification details or change priority ratings depending on his organization's bug bar.

Azure templates have additional details to help users understand not only what's wrong, but also how to fix it by adding descriptions, examples and hyperlinks to Azure-specific documentation.

The description made him realize the importance of adding an authentication mechanism to prevent users from being spoofed, revealing the first threat to be worked on. A few minutes into the discussion with Cristina, they understood the importance of implementing access control and roles. Ricardo filled in some quick notes to make sure these were implemented.

As Ricardo went into the threats under Information Disclosure, he realized the access control plan required some read-only accounts for audit and report generation. He wondered whether this should be a new threat, but the mitigations were the same, so he noted the threat accordingly. He also thought about information disclosure a bit more and realized that the backup tapes were going to need encryption, a job for the operations team.

Threats not applicable to the design due to existing mitigations or security guarantees can be changed to "Not Applicable" from the Status drop-down. There are three other choices: Not Started – default selection, Needs Investigation – used to follow up on items and Mitigated – once it's fully worked on.

Reports & sharing

Once Ricardo goes through the list with Cristina and adds important notes, mitigations/justifications, priority and status changes, he selects Reports -> Create Full Report -> Save Report, which prints out a nice report for him to go through with colleagues to ensure the proper security work is implemented.

Threat Modeling Report

Created on 7/31/2017 12:35:42 PM
Threat Model Name:
Owner:
Reviewer:
Contributors:
Description:
Assumptions:
External Dependencies:

Threat Model Summary:

Not Started	9
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	0
Total	9
Total Migrated	0

Diagram: Diagram 1

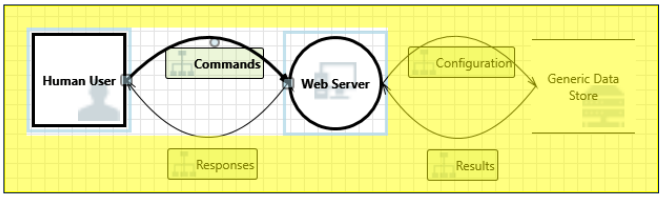
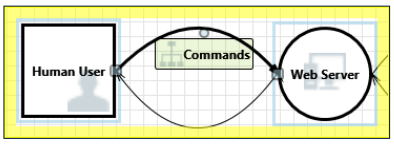


Diagram 1 Diagram Summary:

Not Started	9
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	0
Total	9
Total Migrated	0

Interaction: Commands



1. Spoofing the Human User External Entity [State: Not Started] [Priority: High]

Category:	Spoofing
Description:	Human User may be spoofed by an attacker and this may lead to unauthorized access to Web Server. Consider using a standard authentication mechanism to identify the external entity.
Justification:	<no mitigation provided>
Possible Mitigation(s):	
SDL Phase:	Design

2. Cross Site Scripting [State: Not Started] [Priority: High]

Category:	Tampering
Description:	The web server 'Web Server' could be a subject to a cross-site scripting attack because it does not sanitize untrusted input.
Justification:	<no mitigation provided>
Possible Mitigation(s):	
SDL Phase:	Design

If Ricardo wants to share the file instead, he can easily do so by saving in his organization's OneDrive account. Once he does that, he can copy the document link and share it with his colleagues.

Threat modeling meetings

When Ricardo sent his threat model to his colleague using OneDrive, Ashish, the tester, was underwhelmed. Seemed like Ricardo and Cristina missed quite a few important corner cases, which could be easily compromised. His skepticism is a complement to threat models.

In this scenario, after Ashish took over the threat model, he called for two threat modeling meetings: one meeting to synchronize on the process and walk through the diagrams and then a second meeting for threat review and sign-off.

In the first meeting, Ashish spent 10 minutes walking everyone through the SDL threat modeling process. He then pulled up the threat model diagram and started explaining it in detail. Within five minutes, an important missing component had been identified.

A few minutes later, Ashish and Ricardo got into an extended discussion of how the Web server was built. It was not the ideal way for a meeting to proceed, but everyone eventually agreed that discovering the discrepancy early was going to save them time in the future.

In the second meeting, the team walked through the threats, discussed some ways to address them, and signed off on the threat model. They checked the document into source control and continued with development.

Thinking about assets

Some readers who have threat modeled may notice that we haven't talked about assets at all. We've discovered that many software engineers understand their software better than they understand the concept of assets and what assets an attacker may be interested in.

If you're going to threat model a house, you might start by thinking about your family, irreplaceable photos or valuable artwork. Perhaps you might start by thinking about who might break in and the current security system. Or you might start by considering the physical features, like the pool or the front porch. These are analogous to thinking about assets, attackers, or software design. Any of these three approaches work.

The approach to threat modeling we've presented here is substantially simpler than what Microsoft has done in the past. We found that the software design approach works well for many teams. We hope that include yours.

Next Steps

Send your questions, comments and concerns to tmtextsupport@microsoft.com. **Download** the Threat Modeling Tool to get started.

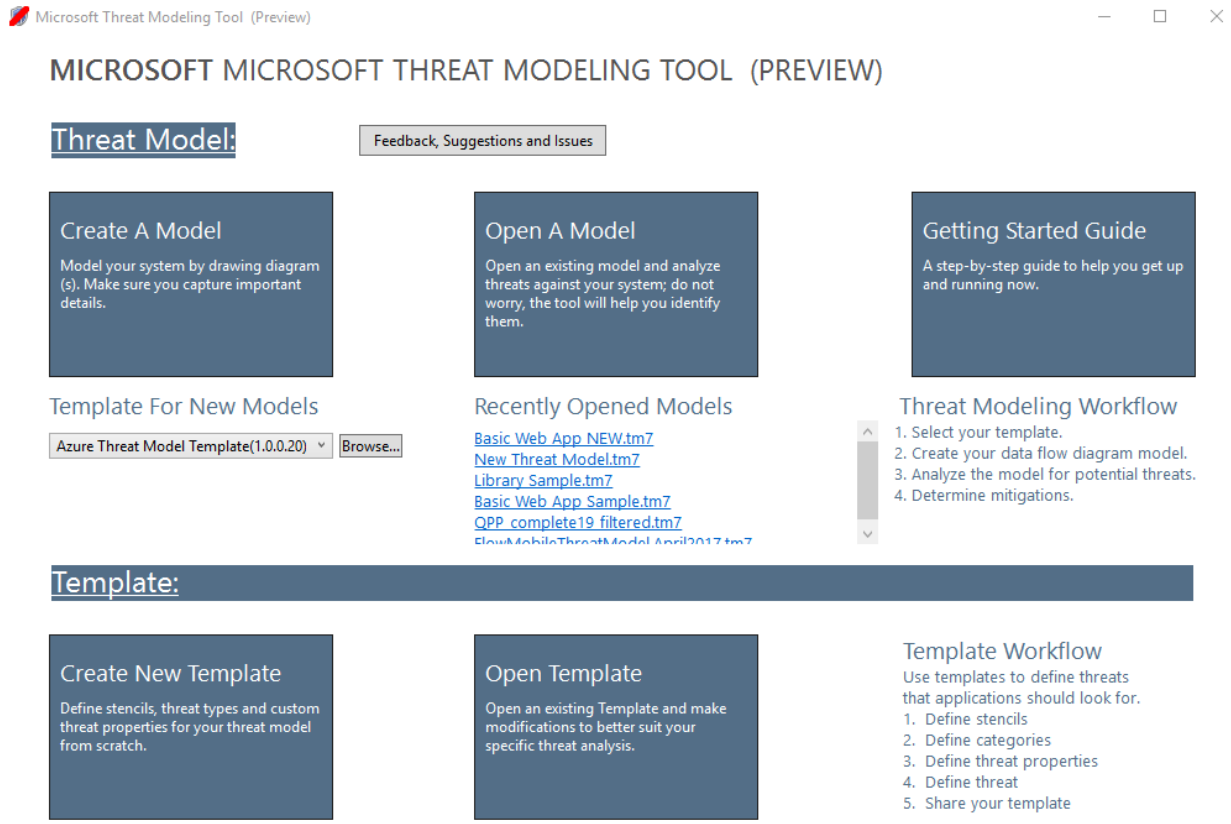
Threat Modeling Tool feature overview

11/2/2020 • 5 minutes to read • Edit Online

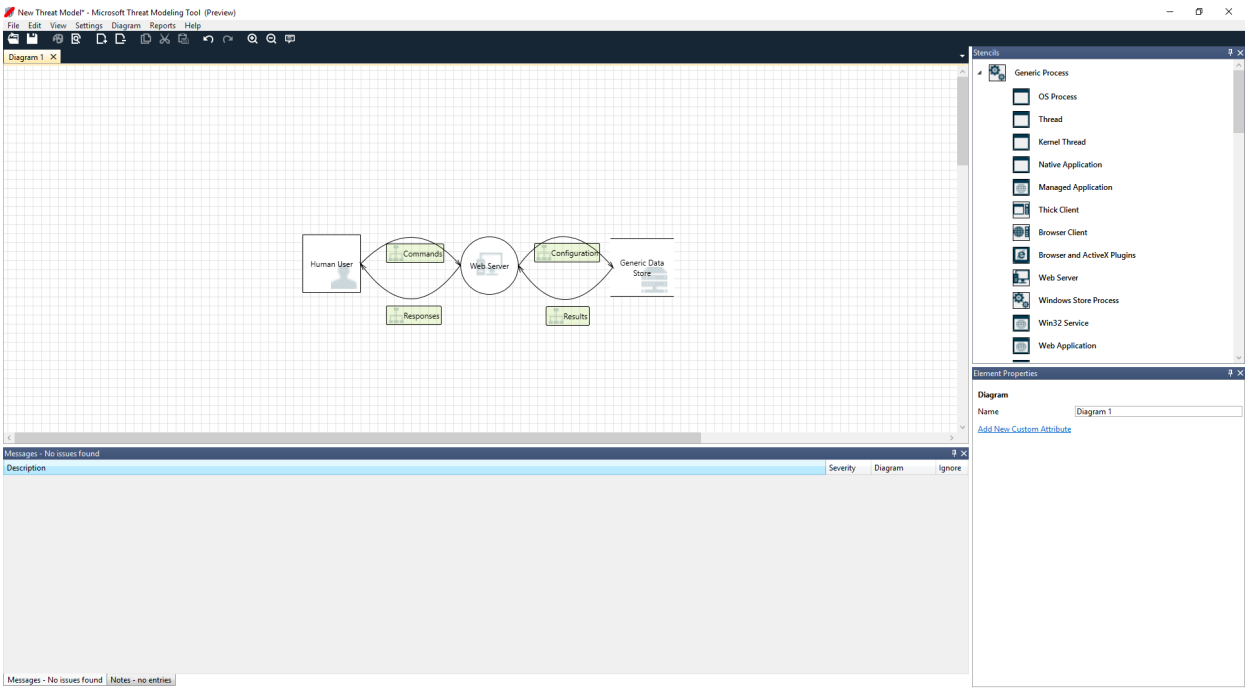
The Threat Modeling Tool can help you with your threat modeling needs. For a basic introduction to the tool, see Get started with the Threat Modeling Tool.

> **NOTE**
> The Threat Modeling Tool is updated frequently, so check this guide often to see our latest features and improvements.

To open a blank page, select **Create A Model**.



To see the features currently available in the tool, use the threat model created by our team in the Get started example.

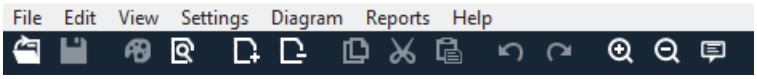


Navigation

Before we discuss the built-in features, let's review the main components found in the tool.

Menu items

The experience is similar to other Microsoft products. Let's review the top-level menu items.



LABEL	DETAILS
File	• Open, save, and close files • Sign in and sign out of OneDrive accounts. • Share links (view and edit). • View file information. • Apply a new template to existing models.
Edit	Undo and redo actions, as well as copy, paste, and delete.
View	• Switch between **Analysis** and **Design** views. • Open closed windows (for example, stencils, element properties, and messages). • Reset layout to default settings.
Diagram	Add and delete diagrams, and move through tabs of diagrams.
Reports	Create HTML reports to share with others.
Help	Find guides to help you use the tool.

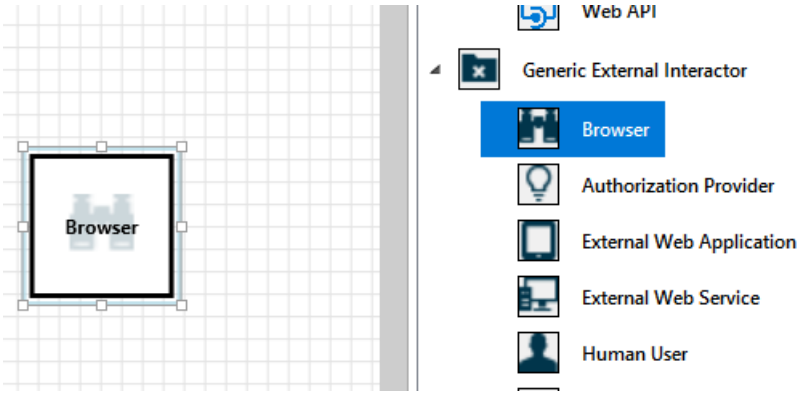
The symbols are shortcuts for the top-level menus:

SYMBOL	DETAILS
Open	Opens a new file.
Save	Saves the current file.
Design	Opens the **Design** view, where you can create models.
Analyze	Shows generated threats and their properties.
Add diagram	Adds a new diagram (similar to new tabs in Excel).
Delete diagram	Deletes the current diagram.
Copy/Cut/Paste	Copies, cuts, and pastes elements.
Undo/Redo	Undoes and redoes actions.
Zoom in/Zoom out	Zooms in and out of the diagram for a better view.
Feedback	Opens the MSDN Forum.

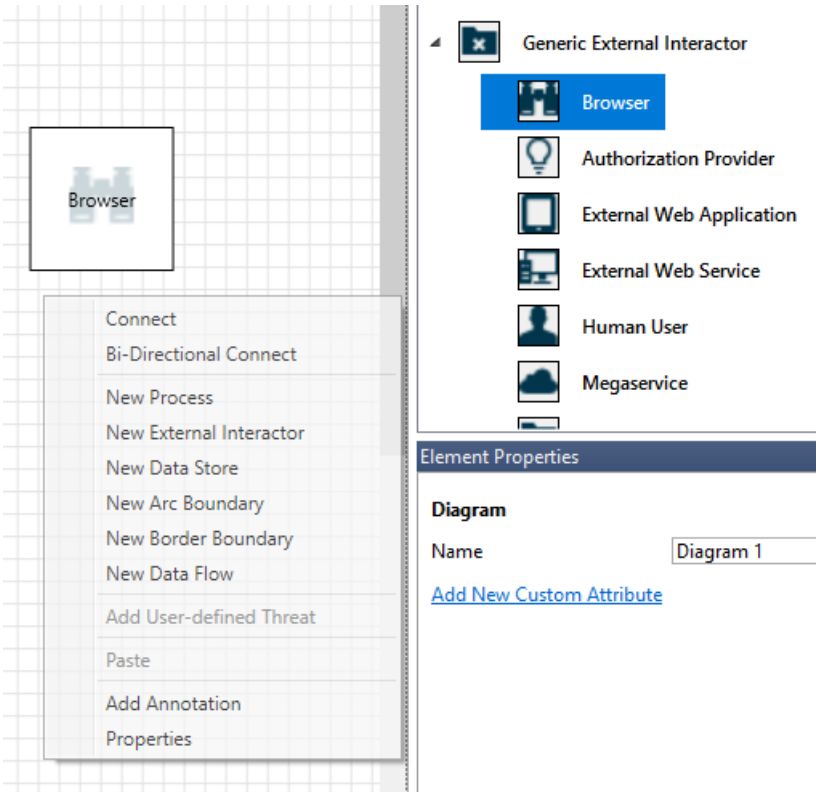
Canvas

The canvas is the space where you drag and drop elements. Drag and drop is the quickest and most efficient way to build models. You can also right-click and select items from the menu to add generic versions of elements, as shown:

Drop the stencil on the canvas



Select the stencil



Stencils

Based on the template you select, you can find all the stencils available to use. If you can't find the right elements, use another template. Or you can modify a template to fit your needs. Generally, you can find a combination of categories like these:

STENCIL NAME	DETAILS
Process	Applications, browser plug-ins, threads, virtual machines
External interactor	Authentication providers, browsers, users, web applications
Data store	Cache, storage, configuration files, databases, registry
Data flow	Binary, ALPC, HTTP, HTTPS/TLS/SSL, IOCTL, IPSec, named pipe, RPC/DCOM, SMB, UDP
Trust line/Border boundary	Corporate networks, internet, machine, sandbox, user/kernel mode

Notes/messages

COMPONENT	DETAILS
Messages	Internal tool logic that alerts users whenever there's an error, such as no data flows between elements.
Notes	Manual notes are added to the file by engineering teams throughout the design and review process.

Element properties

Element properties vary by the elements you select. Apart from trust boundaries, all other elements contain three general selections:

ELEMENT PROPERTY	DETAILS
Name	Useful for naming your processes, stores, interactors, and flows so that they're easily recognized.
Out of scope	If selected, the element is taken out of the threat-generation matrix (not recommended).
Reason for out of scope	Justification field to let users know why out of scope was selected.

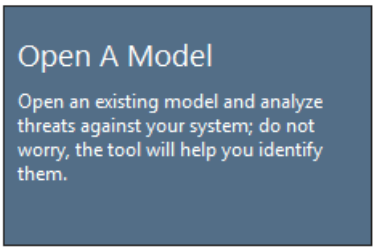
Properties are changed under each element category. Select each element to inspect the available options. Or you can open the template to learn more. Let's review the features.

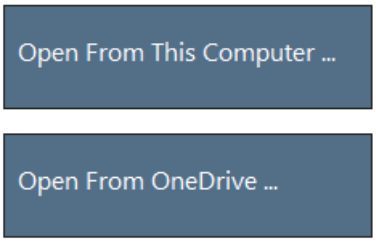
Welcome screen

When you open the app, you see the **Welcome** screen.

Open a model

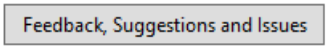
Hover over **Open A Model** to reveal two options: **Open From This Computer** and **Open From OneDrive**. The first option opens the **File Open** screen. The second option takes you through the sign-in process for OneDrive. After successful authentication, you can select folders and files.





Feedback, suggestions, and issues

When you select **Feedback, Suggestions and Issues**, you go to the MSDN Forum for SDL Tools. You can read what other people are saying about the tool, including workarounds and new ideas.



Design view

When you open or create a new model, the **Design** view opens.

Add elements

You can add elements on the grid in two ways:

- **Drag and drop**: Drag the desired element to the grid. Then use the element properties to provide additional information.
- **Right-click**: Right-click anywhere on the grid, and select items from the drop-down menu. A generic

representation of the element you select appears on the screen.

Connect elements

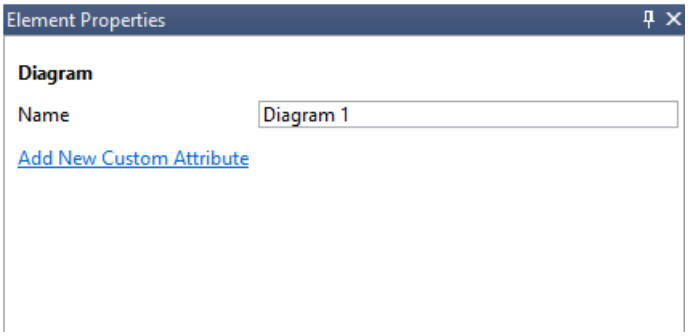
You can connect elements in two ways:

- **Drag and drop**: Drag the desired dataflow to the grid, and connect both ends to the appropriate elements.
- **Click + Shift**: Click the first element (sending data), press and hold the Shift key, and then select the second element (receiving data). Right-click, and select **Connect**. If you use a bi-directional data flow, the order is not as important.

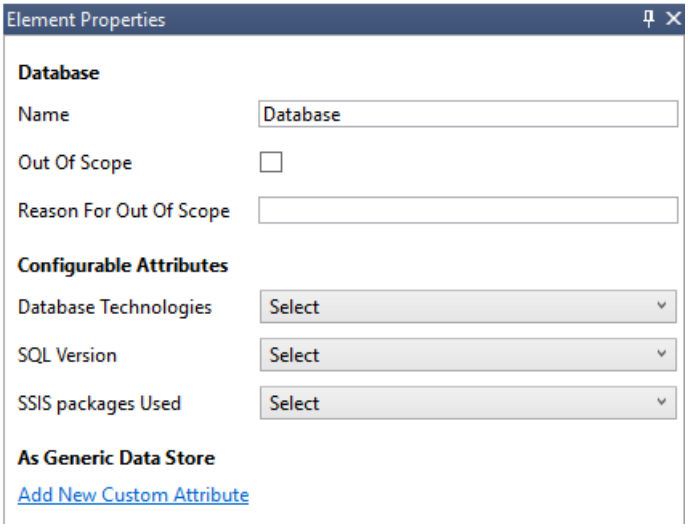
Properties

To see the properties that can be modified on the stencils, select the stencil and the information populates accordingly. The following example shows before and after a **Database** stencil is dragged onto the diagram:

Before

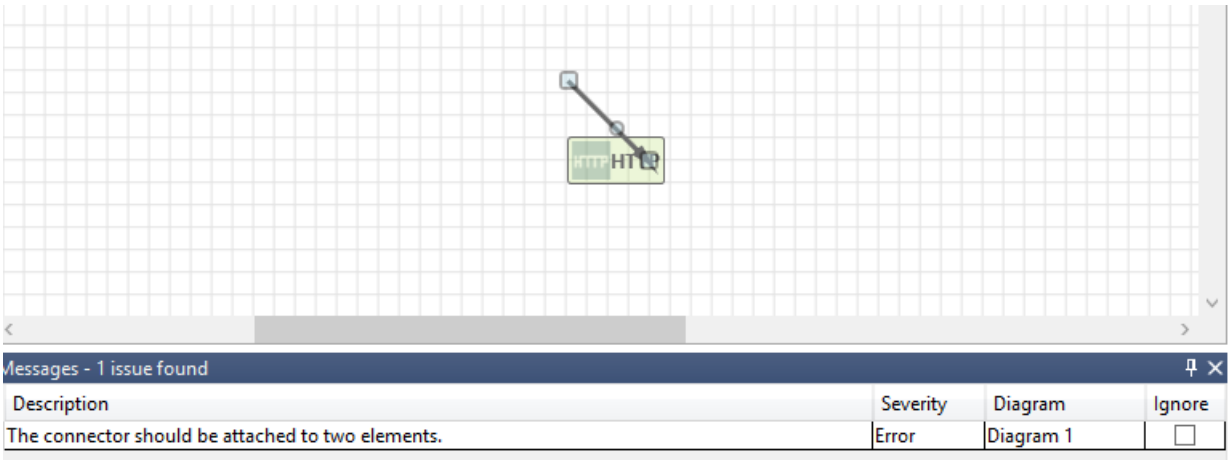


After



Messages

If you create a threat model and forget to connect data flows to elements, you get a notification. You can ignore the message, or you can follow the instructions to fix the issue.

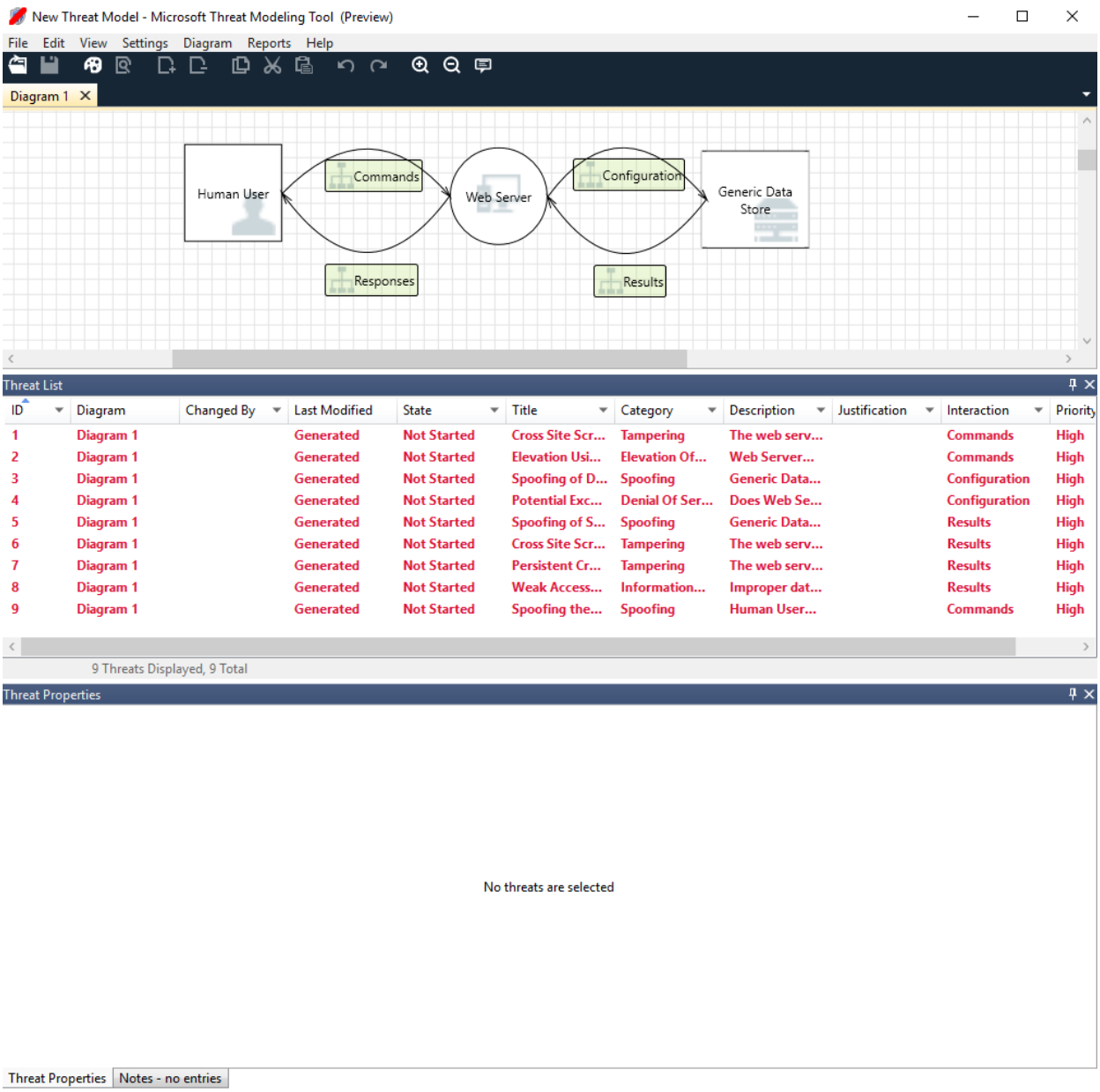


Notes

To add notes to your diagram, switch from the **Messages** tab to the **Notes** tab.

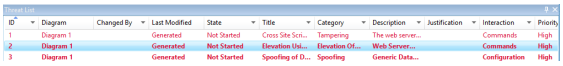
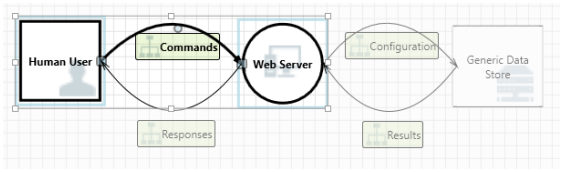
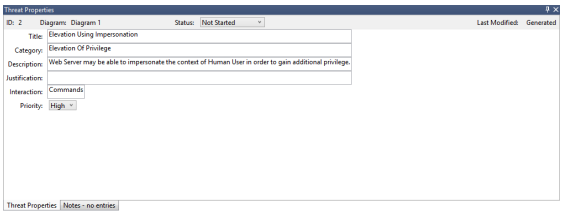
Analysis view

After you build your diagram, select the **Analysis** symbol (the magnifying glass) on the shortcuts toolbar to switch to the **Analysis** view.



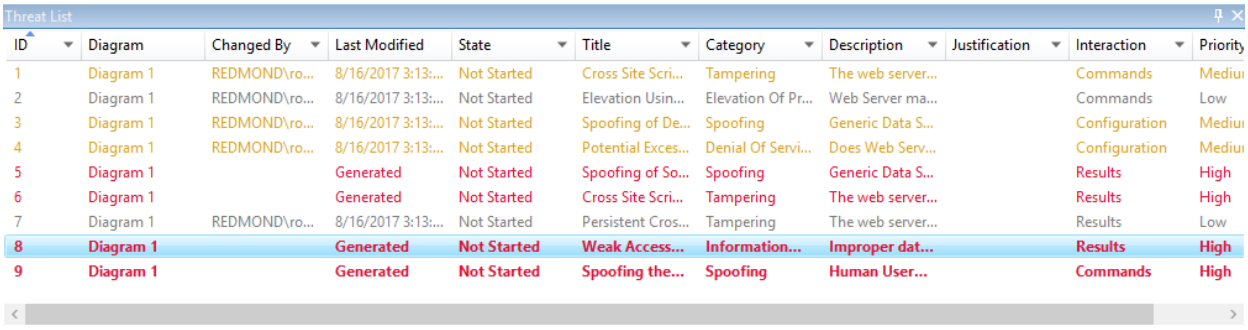
Generated threat selection

When you select a threat, you can use three distinct functions:

FEATURE	INFORMATION
Read indicator	The threat is marked as read, which helps you keep track of the items you reviewed. 
Interaction focus	Interaction in the diagram that belongs to a threat is highlighted. 
Threat properties	Additional information about the threat appears in the **Threat Properties** window. 

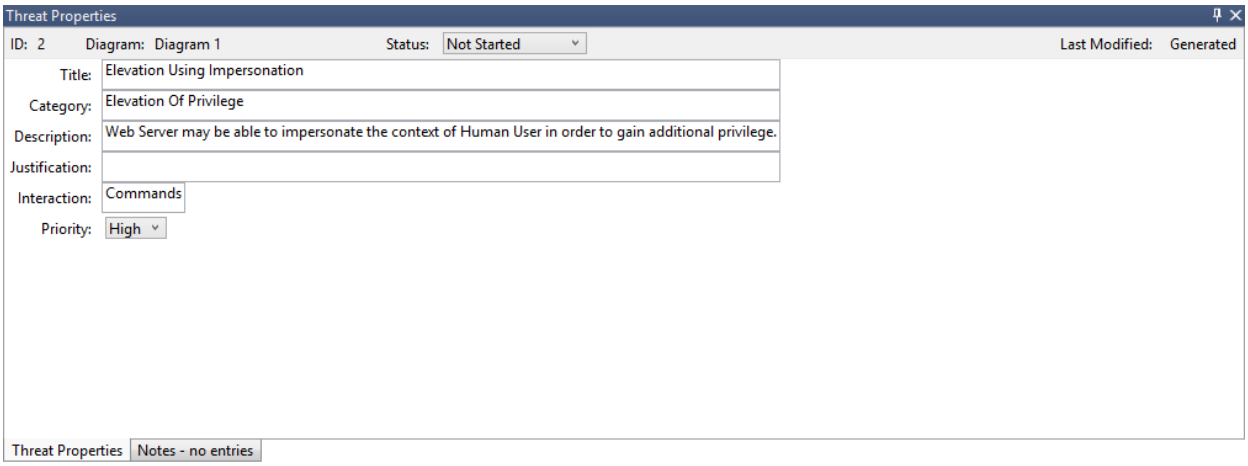
Priority change

You can change the priority level of each generated threat. Different colors make it easy to identify high-, medium-, and low-priority threats.



Threat properties editable fields

As seen in the preceding image, you can change the information generated by the tool. You can also add information to certain fields, such as justification. These fields are generated by the template. If you need more information for each threat, you can make modifications.



Reports

After you finish changing priorities and updating the status of each generated threat, you can save the file and/or print out a report. Go to **Report** > **Create Full Report**. Name the report, and you should see something similar to the following image:

Threat Modeling Report

Created on 7/31/2017 12:35:42 PM
Threat Model Name:
Owner:
Reviewer:
Contributors:
Description:
Assumptions:
External Dependencies:

Threat Model Summary:

Not Started	9
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	0
Total	9
Total Migrated	0

Diagram: Diagram 1

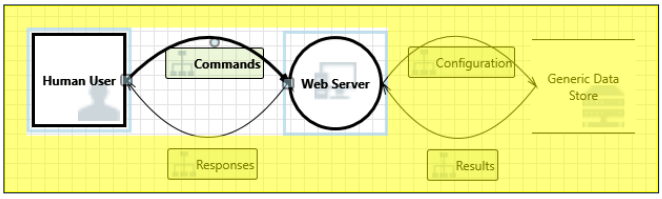
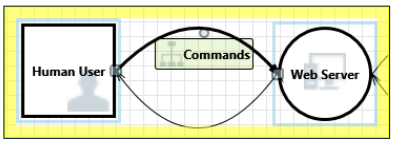


Diagram 1 Diagram Summary:

Not Started	9
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	0
Total	9
Total Migrated	0

Interaction: Commands



1. Spoofing the Human User External Entity [State: Not Started] [Priority: High]

Category:	Spoofing
Description:	Human User may be spoofed by an attacker and this may lead to unauthorized access to Web Server. Consider using a standard authentication mechanism to identify the external entity.
Justification:	<no mitigation provided>
Possible Mitigation(s):	
SDL Phase:	Design

2. Cross Site Scripting [State: Not Started] [Priority: High]

Category:	Tampering
Description:	The web server 'Web Server' could be a subject to a cross-site scripting attack because it does not sanitize untrusted input.
Justification:	<no mitigation provided>
Possible Mitigation(s):	
SDL Phase:	Design

Next steps

- Send your questions, comments and concerns to tmtextsupport@microsoft.com. **Download** the Threat Modeling Tool to get started.
- To contribute a template for the community, go to our GitHub page.

Microsoft Threat Modeling Tool threats

8/1/2019 • 2 minutes to read • Edit Online

The Threat Modeling Tool is a core element of the Microsoft Security Development Lifecycle (SDL). It allows software architects to identify and mitigate potential security issues early, when they are relatively easy and cost-effective to resolve. As a result, it greatly reduces the total cost of development. Also, we designed the tool with non-security experts in mind, making threat modeling easier for all developers by providing clear guidance on creating and analyzing threat models.

> Visit the **Threat Modeling Tool** to get started today!

The Threat Modeling Tool helps you answer certain questions, such as the ones below:

- How can an attacker change the authentication data?
- What is the impact if an attacker can read the user profile data?
- What happens if access is denied to the user profile database?

STRIDE model

To better help you formulate these kinds of pointed questions, Microsoft uses the STRIDE model, which categorizes different types of threats and simplifies the overall security conversations.

CATEGORY	DESCRIPTION
Spoofing	Involves illegally accessing and then using another user's authentication information, such as username and password
Tampering	Involves the malicious modification of data. Examples include unauthorized changes made to persistent data, such as that held in a database, and the alteration of data as it flows between two computers over an open network, such as the Internet
Repudiation	Associated with users who deny performing an action without other parties having any way to prove otherwise—for example, a user performs an illegal operation in a system that lacks the ability to trace the prohibited operations. Non-Repudiation refers to the ability of a system to counter repudiation threats. For example, a user who purchases an item might have to sign for the item upon receipt. The vendor can then use the signed receipt as evidence that the user did receive the package
Information Disclosure	Involves the exposure of information to individuals who are not supposed to have access to it—for example, the ability of users to read a file that they were not granted access to, or the ability of an intruder to read data in transit between two computers

CATEGORY	DESCRIPTION
Denial of Service	Denial of service (DoS) attacks deny service to valid users—for example, by making a Web server temporarily unavailable or unusable. You must protect against certain types of DoS threats simply to improve system availability and reliability
Elevation of Privilege	An unprivileged user gains privileged access and thereby has sufficient access to compromise or destroy the entire system. Elevation of privilege threats include those situations in which an attacker has effectively penetrated all system defenses and become part of the trusted system itself, a dangerous situation indeed

Next steps

Proceed to **Threat Modeling Tool Mitigations** to learn the different ways you can mitigate these threats with Azure.

Threat Modeling Tool Releases

11/2/2020 • 2 minutes to read • Edit Online

The Microsoft Threat Modeling Tool is currently released as a free click-to-download application for Windows. This delivery mechanism allows us to push the latest improvements and bug fixes to customers each time they open the tool.

System Requirements

- Supported Operating Systems
  - Microsoft Windows 10 Anniversary Update or later
- .NET Version Required
  - .NET 4.7.1 or later
- Additional Requirements
  - An Internet connection is required to receive updates to the tool and templates.

Release Notes

- Microsoft Threat Modeling Tool GA Release Version 7.3.00729.1 - July 29 2020
- Microsoft Threat Modeling Tool GA Release Version 7.3.00714.2 - July 14 2020
- Microsoft Threat Modeling Tool GA Release Version 7.3.00316.1 - March 22 2020
- Microsoft Threat Modeling Tool GA Release Version 7.3.00206.1 - February 11 2020
- Microsoft Threat Modeling Tool GA Release Version 7.1.61015.1 - October 16 2019
- Microsoft Threat Modeling Tool GA Release Version 7.1.60702.1 - July 2 2019
- Microsoft Threat Modeling Tool GA Release Version 7.1.60408.1 - April 9 2019
- Microsoft Threat Modeling Tool GA Release Version 7.1.60126.1 - January 29 2019
- Microsoft Threat Modeling Tool GA Release Version 7.1.51023.1 - November 1 2018
- Microsoft Threat Modeling Tool GA Release Version 7.1.50911.2 - September 12 2018

Next steps

Download the latest version of the Microsoft Threat Modeling Tool.

Threat Modeling Tool GA release 7.1.50911.2 - 9/12/2018

11/2/2020 • 3 minutes to read • Edit Online

We are excited to announce the Microsoft Threat Modeling Tool is now available to download as a supported generally available (GA) release. This release contains important privacy and security updates as well as bug fixes, feature updates, and stability improvements. Existing users of the 2017 Preview version will be prompted to update to the latest release through the ClickOnce technology upon opening the client. For new users of the tool, you can download the client.

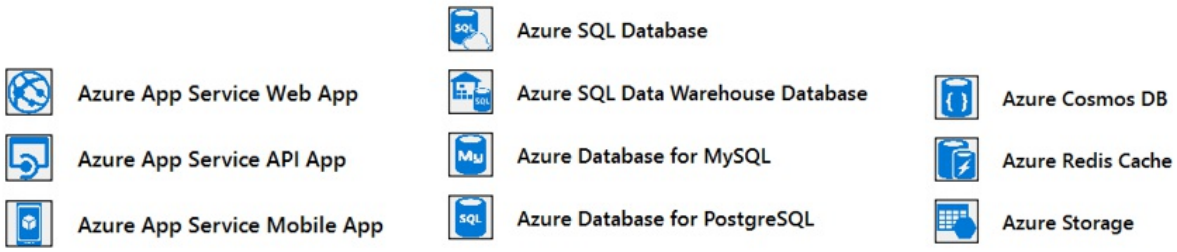
With this release, we are ending support for the 2017 Preview and recommend all users of the Preview update to the GA release. On or after October 15 2018, we will set the minimum required ClickOnce version for the Threat Modeling Tool, and all Preview clients will be required to upgrade.

The Microsoft Threat Modeling Tool 2016, which is available from the Microsoft Download Center, remains supported until October 1 2019 for critical security fixes only.

Feature changes

Azure stencil updates

Additional Azure stencils and their associated threats and mitigations have been added to the stencil set shipping with this release. Significant changes were made in the focus areas of "Azure App Services", "Azure Database Offerings", and "Azure Storage."



OneDrive integration feature removed

The "Save To OneDrive", "Open From OneDrive", and "Share a Link" features of the Preview have been removed. Users of OneDrive are encouraged to use Microsoft's OneDrive for Windows client to access their files stored on OneDrive through the standard file save and open dialogs.

Notable fixed bugs reported by customers

In TMT Preview, the tool crashes when using the standard template

- When a Generic stencil (for example "Generic Data Flow") is added to the drawing surface and generates threats, the tool may crash. This issue has been fixed.

In TMT Preview, when I save a report or copy the threats, the risk levels are incorrect

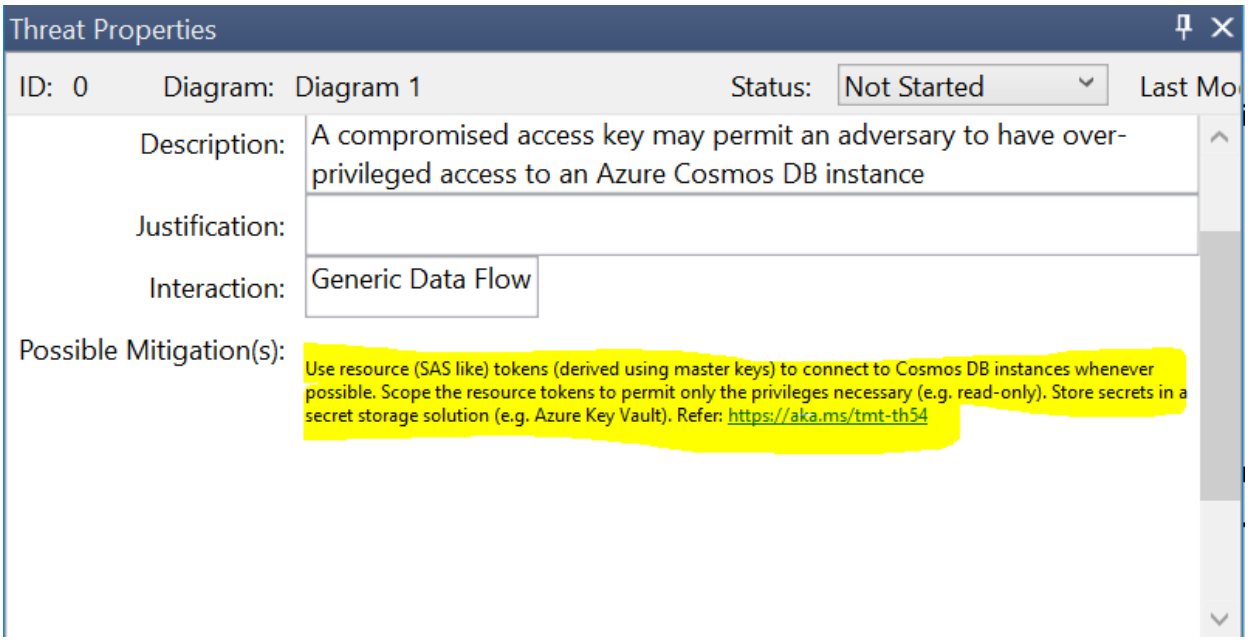
- If a user modifies the Risk level of specific threats and then saves a report or copies the risks, the risk level may revert to "High". This issue has been fixed.

Known issues and FAQ

Users of high-resolution screens may experience small text in the threat properties

Issue

In the Analysis View of the tool, if the user has a high-resolution screen that is set by default to magnify for readability in Windows, the "Possible Mitigation(s)" section of a threat may appear with small text.



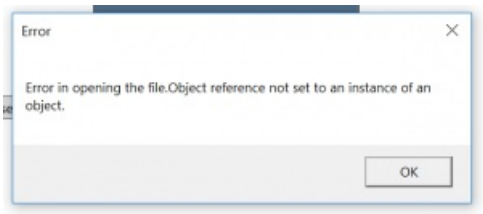
Workaround

The user can click on the mitigation text and use the standard Windows zoom control (Crtl-Mouse Wheel Up) to increase the magnification of that section.

Files in the "Recently Opened Models" section of the main window may fail to open

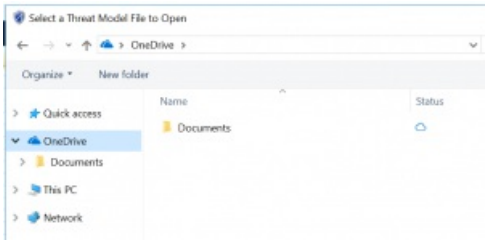
Issue

The "Open From OneDrive" feature of the Preview release has been removed. Users with "Recently Opened Models" that were saved to OneDrive will receive the following error.



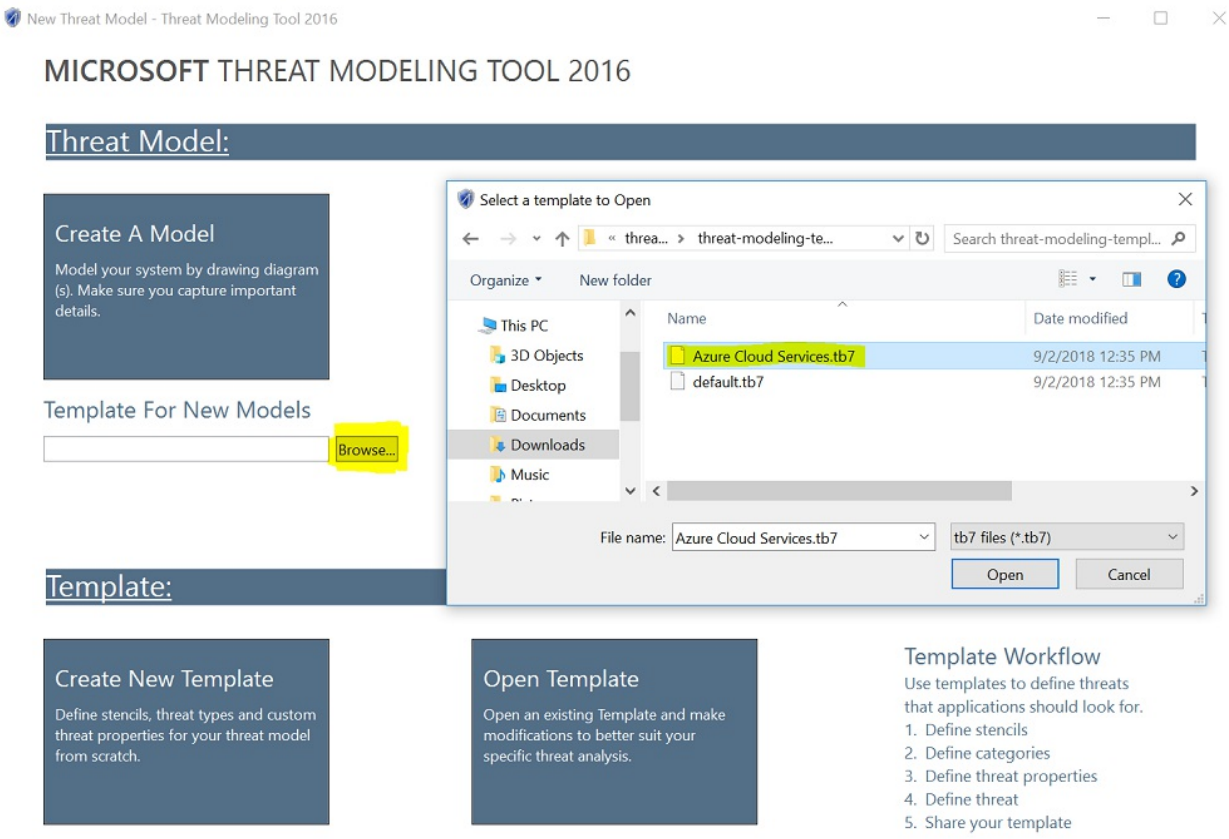
Workaround

Users of OneDrive are encouraged to use Microsoft's OneDrive for Windows client to access their files stored on OneDrive through the standard and "Open a model" dialog.



My organization uses the 2016 version of the tool, can I use the Azure stencil set?

Yes, you can! The Azure stencil set is available on GitHub, and can be loaded in the 2016 version of the tool. To create a new model with the Azure stencil set, use the "Template For New Models" dialog on the main menu screen. TMT 2016 cannot render the links found in the "Possible Mitigations" fields of the Azure stencil set, therefore you may see links displayed as HTML tags.



System requirements

- Supported Operating Systems
  - Microsoft Windows 10
- .NET Version Required
  - .NET 3.5.2
- Additional Requirements
  - An Internet connection is required to receive updates to the tool as well as templates.

Documentation and feedback

- Documentation for the Threat Modeling Tool is located on docs.microsoft.com, and includes information about using the tool.

Next steps

Download the latest version of the Microsoft Threat Modeling Tool.

Threat Modeling Tool update release 7.1.51023.1 - 11/1/2018

11/2/2020 • 2 minutes to read • Edit Online

As originally noted in the GA release notes, we have released an update (7.1.51023.1) to the Microsoft Threat Modeling Tool that will require users of the Preview version (preview clients with version < 7.1.50911.2) to upgrade to the supported GA release. This release does not contain any new functionality or fixes.

- Users of the Preview version will automatically download the upgrade when the client is opened. If you choose not to install the new update, the Preview version of the tool will close.
- Users of the GA version of the tool will be prompted to choose whether or not they want to upgrade.
- Users of the 2016 version of the tool will be unaffected.

Feature changes

- None

System requirements

- Supported Operating Systems
  - Microsoft Windows 10
- .NET Version Required
  - .NET 3.5.2
- Additional Requirements
  - An Internet connection is required to receive updates to the tool as well as templates.

Documentation and feedback

- Documentation for the Threat Modeling Tool is located on docs.microsoft.com, and includes information about using the tool.

Next steps

Download the latest version of the Microsoft Threat Modeling Tool.

Threat Modeling Tool update release 7.1.60126.1 - 1/29/2019

11/2/2020 • 2 minutes to read • Edit Online

Version 7.1.60126.1 of the Microsoft Threat Modeling Tool was released on January 29 2019 and contains the following changes:

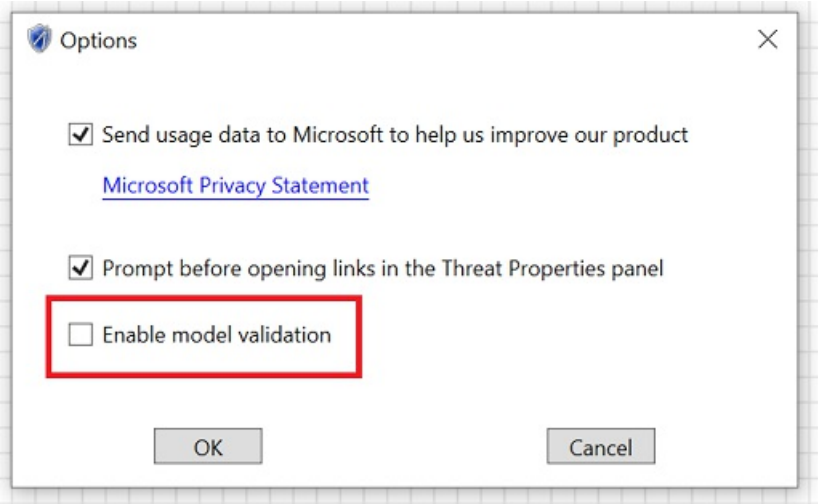
- The minimum required version of .NET has been increased to .NET 4.7.1.
- The minimum required version of Windows has been increased to Windows 10 Anniversary Update due to the .NET dependency.
- A model validation toggle feature has been added to the tool's Options menu.
- Several links in the Threat Properties were updated.
- Minor UX changes to the tool's home screen.
- The Threat Modeling Tool now inherits the TLS settings of the host operating system and is supported in environments that require TLS 1.2 or greater.

Feature changes

Model validation option

Based on customer feedback, an option has been added to the tool to enable or disable the model validation. Previously, if your template used a single unidirectional data flow between two objects, you may have received an error message in the Messages frame stating: ObjectsName requires at least one 'Any'. Disabling model validation will prevent these warnings from showing in the view.

The option to toggle model validation on and off can be found in the File->Settings->Options menu. The default value for this setting is Disabled.



System requirements

- Supported Operating Systems
  - Microsoft Windows 10 Anniversary Update or later
- .NET Version Required
  - .NET 4.7.1 or later
- Additional Requirements

- An Internet connection is required to receive updates to the tool as well as templates.

Known issues

Unsupported systems

Issue

Users of Windows 10 systems that are unable to install .NET 4.7.1 or later, such as Windows 10 Enterprise LTSB (version 1507), will be unable to open the tool after upgrading. These older versions of Windows are no longer supported platforms for the Threat Modeling Tool, and should not install the latest update.

Workaround

Users of Windows 10 Enterprise LTSB (version 1507) that have installed the latest update can revert to the previous version of the Threat Modeling Tool through the uninstall dialog in Apps & Features.

Documentation and feedback

- Documentation for the Threat Modeling Tool is located on docs.microsoft.com, and includes information about using the tool.

Next steps

Download the latest version of the Microsoft Threat Modeling Tool.

Threat Modeling Tool update release 7.1.60408.1 - 4/9/2019

11/2/2020 • 2 minutes to read • Edit Online

Version 7.1.60408.1 of the Microsoft Threat Modeling Tool (TMT) was released on April 9 2019 and contains the following changes:

- New Stencils for Azure Key Vault and Azure Traffic Manager
- TMT version number is now shown on the home screen
- Support links have been updated
- Bug fixes

Feature changes

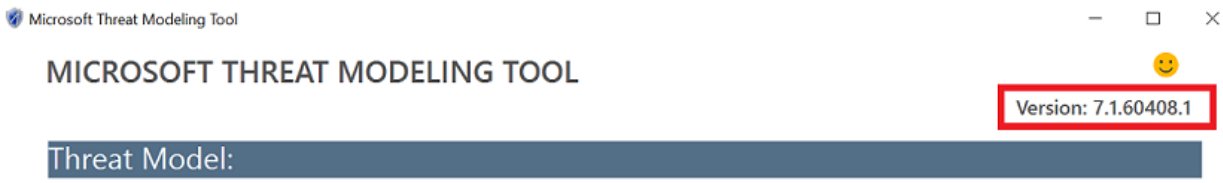
New Stencils for Azure Key Vault and Azure Traffic Manager



New stencils and threats for Azure Key Vault and Azure Traffic Manager have been added to the Azure stencil set. When opening models based on the Azure stencil set, users will be prompted to update the template associated with the model. Updating a model based on the Azure stencil set can also be manually initiated by using the "Apply Template" command in the "File" menu and reapplying the latest Azure Cloud Services.tb7 file.

TMT version number is now shown on the home screen

The client version of the Threat Modeling Tool is now shown on the home screen of the application of for ease of access.



Support links have been updated

All support links within the tool have been updated to direct users to tmtextsupport@microsoft.com rather than an MSDN forum.

System requirements

- Supported Operating Systems
  - Microsoft Windows 10 Anniversary Update or later
- .NET Version Required
  - .Net 4.7.1 or later
- Additional Requirements
  - An Internet connection is required to receive updates to the tool as well as templates.

Documentation and feedback

- Documentation for the Threat Modeling Tool is located on docs.microsoft.com, and includes information

about using the tool.

Next steps

Download the latest version of the Microsoft Threat Modeling Tool.

Threat Modeling Tool update release 7.1.60702.1 - 7/2/2019

11/2/2020 • 2 minutes to read • Edit Online

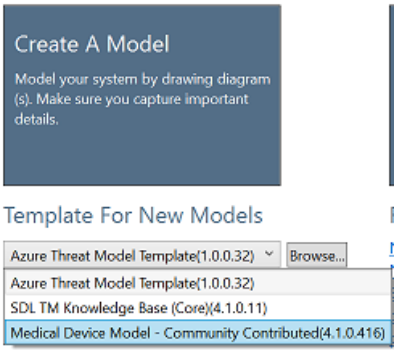
Version 7.1.60702.1 of the Microsoft Threat Modeling Tool (TMT) was released on July 2 2019 and contains the following changes:

- Accessibility improvements
- Bug fixes
- Inclusion of an open-source community provided stencil set

Feature changes

A new medical devices stencil set provided by the open-source community is available

A stencil set for modeling medical devices has been contributed by the open-source community. After updating, the new stencil set will appear in the template selection drop down menu. For information about contributing stencils or content to templates, review the information on the project's GitHub page.



System requirements

- Supported Operating Systems
  - Microsoft Windows 10 Anniversary Update or later
- .NET Version Required
  - .Net 4.7.1 or later
- Additional Requirements
  - An Internet connection is required to receive updates to the tool as well as templates.

Documentation and feedback

- Documentation for the Threat Modeling Tool is located on docs.microsoft.com, and includes information about using the tool.

Next steps

Download the latest version of the Microsoft Threat Modeling Tool.

Threat Modeling Tool update release 7.1.61015.1 - 10/16/2019

3/5/2021 • 2 minutes to read • Edit Online

Version 7.1.61015.1 of the Microsoft Threat Modeling Tool (TMT) was released on October 16 2019 and contains the following changes:

- Accessibility improvements
- Bug fixes
- New stencils for Azure Logic Apps and Azure Data Explorer

Notable bug fixes

Improved backward compatibility with files created in "Threat Modeling Tool 2016"

Several bugs related to the opening or display of threat model files created in "Threat Modeling Tool 2016" have been fixed.

Feature enhancements

New stencils for Azure Logic Apps and Azure Data Explorer

New stencils for Azure Logic Apps and Azure Data Explorer were added to the Azure Stencil along with their associated threats and mitigations.



Known issues

Errors related to priority values outside of the expected ranges

Some customers have reported receiving the following error message when opening files created in the "Threat Modeling Tool 2016" or custom templates:

```
System.InvalidOperationException: Invalid Priority value. Accepted values are [0..4] and 'High', 'Medium',
'Low' at ThreatModeling.Model.Threat.get_Priority()

System.ArgumentOutOfRangeException: Accepted values are 'High', 'Medium', and 'Low' Parameter name: value
Actual value was 5.6. at ThreatModeling.Model.Threat.set_Priority(String value)
```

This issue is under investigation

System requirements

- Supported Operating Systems
  - Microsoft Windows 10 Anniversary Update or later
- .NET Version Required
  - .Net 4.7.1 or later
- Additional Requirements
  - An Internet connection is required to receive updates to the tool as well as templates.

Documentation and feedback

- Documentation for the Threat Modeling Tool is located on docs.microsoft.com, and includes information about using the tool.

Next steps

Download the latest version of the Microsoft Threat Modeling Tool.

Threat Modeling Tool update release 7.3.00206.1 - 02/11/2020

3/5/2021 • 2 minutes to read • Edit Online

Version 7.3.00206.1 of the Microsoft Threat Modeling Tool (TMT) was released on February 11 2020 and contains the following changes:

- Bug fixes

Notable bug fixes

Errors related to priority values outside of the expected ranges

Some customers had reported receiving the following error message when opening files created in the "Threat Modeling Tool 2016" or custom templates:

```
System.InvalidOperationException: Invalid Priority value. Accepted values are [0..4] and 'High', 'Medium',
'Low' at ThreatModeling.Model.Threat.get_Priority()

System.ArgumentOutOfRangeException: Accepted values are 'High', 'Medium', and 'Low' Parameter name: value
Actual value was 5.6. at ThreatModeling.Model.Threat.set_Priority(String value)
```

This issue has been resolved in this release.

System requirements

- Supported Operating Systems
  - Microsoft Windows 10 Anniversary Update or later
- .NET Version Required
  - .Net 4.7.1 or later
- Additional Requirements
  - An Internet connection is required to receive updates to the tool as well as templates.

Documentation and feedback

- Documentation for the Threat Modeling Tool is located on docs.microsoft.com, and includes information about using the tool.

Next steps

Download the latest version of the Microsoft Threat Modeling Tool.

Threat Modeling Tool update release 7.3.00316.1 - 03/22/2020

3/5/2021 • 2 minutes to read • Edit Online

Version 7.3.00316.1 of the Microsoft Threat Modeling Tool (TMT) was released on March 22 2020 and contains the following changes:

- Accessibility improvements
- Bug fixes
- New DiagramReader feature

Notable bug fixes

Exporting the threat list to CSV

The export to CSV function was inconsistently selecting which fields from the threat list would be exported. Now all fields from the threat list will be exported into the CSV file.

UX bugs

- The help menus in the primary workflow (create/open/analyze) and the template editor experience now have consistent menu options.
- The search bar in the stencils pane now has a standard cursor and appropriate labels have been added.

New features

DiagramReader feature has been added

A new DiagramReader feature has been added in the main menu while a model is open. This feature will convert the graphical representation of the model into a text-based narrative.

System requirements

- Supported operating systems:
  - Microsoft Windows 10 Anniversary Update or later
- .NET version required:
  - .NET 4.7.1 or later
- Additional requirements:
  - An internet connection to receive updates to the tool as well as templates

Documentation and feedback

- Documentation for the Threat Modeling Tool is located on docs.microsoft.com, and includes information about using the tool.

Next steps

Download the latest version of the Microsoft Threat Modeling Tool.

Threat Modeling Tool update release 7.3.00714.2 - 07/14/2020

3/5/2021 • 2 minutes to read • Edit Online

Version 7.3.00714.2 of the Microsoft Threat Modeling Tool (TMT) was released on July 14 2020 and contains the following changes:

- Accessibility improvements

System requirements

- Supported Operating Systems
  - Microsoft Windows 10 Anniversary Update or later
- .NET Version Required
  - .NET 4.7.1 or later
- Additional Requirements
  - An Internet connection is required to receive updates to the tool as well as templates.

Documentation and feedback

- Documentation for the Threat Modeling Tool is located on docs.microsoft.com, and includes information about using the tool.

Next steps

Download the latest version of the Microsoft Threat Modeling Tool.

Threat Modeling Tool update release 7.3.00729.1 - 07/29/2020

3/5/2021 • 2 minutes to read • Edit Online

Version 7.3.00729.1 of the Microsoft Threat Modeling Tool (TMT) was released on July 29 2020 and contains the following changes:

- Bug fixes

Known issues

Errors related to TMT7.application file deserialization

Issue

Some customers have reported receiving the following error message when downloading the Threat Modeling Tool:

```
The threat model file '$PATH\TMT7.application' could not be deserialized. File is not an actual threat model
or the threat model may be corrupted.
```

This error occurs because some browsers do not natively support ClickOnce installation. In those cases the ClickOnce application file is downloaded to the user's hard drive.

Workaround

This error will continue to appear if the Threat Modeling Tool is launched by double-clicking on the TMT7.application file. However, after bypassing the error the tool will function normally. Rather than launching the Threat Modeling Tool by double-clicking the TMT7.application file, users should utilize shortcuts created in the Windows Menu during the installation to start the Threat Modeling Tool.

System requirements

- Supported Operating Systems
  - Microsoft Windows 10 Anniversary Update or later
- .NET Version Required
  - .NET 4.7.1 or later
- Additional Requirements
  - An Internet connection is required to receive updates to the tool as well as templates.

Documentation and feedback

- Documentation for the Threat Modeling Tool is located on docs.microsoft.com, and includes information about using the tool.

Next steps

Download the latest version of the Microsoft Threat Modeling Tool.

Microsoft Threat Modeling Tool mitigations

8/1/2019 • 2 minutes to read • Edit Online

The Threat Modeling Tool is a core element of the Microsoft Security Development Lifecycle (SDL). It allows software architects to identify and mitigate potential security issues early, when they are relatively easy and cost-effective to resolve. As a result, it greatly reduces the total cost of development. Also, we designed the tool with non-security experts in mind, making threat modeling easier for all developers by providing clear guidance on creating and analyzing threat models.

Visit the **Threat Modeling Tool** to get started today!

Mitigation categories

The Threat Modeling Tool mitigations are categorized according to the Web Application Security Frame, which consists of the following:

CATEGORY	DESCRIPTION
Auditing and Logging	Who did what and when? Auditing and logging refer to how your application records security-related events
Authentication	Who are you? Authentication is the process where an entity proves the identity of another entity, typically through credentials, such as a user name and password
Authorization	What can you do? Authorization is how your application provides access controls for resources and operations
Communication Security	Who are you talking to? Communication Security ensures all communication done is as secure as possible
Configuration Management	Who does your application run as? Which databases does it connect to? How is your application administered? How are these settings secured? Configuration management refers to how your application handles these operational issues
Cryptography	How are you keeping secrets (confidentiality)? How are you tamper-proofing your data or libraries (integrity)? How are you providing seeds for random values that must be cryptographically strong? Cryptography refers to how your application enforces confidentiality and integrity
Exception Management	When a method call in your application fails, what does your application do? How much do you reveal? Do you return friendly error information to end users? Do you pass valuable exception information back to the caller? Does your application fail gracefully?

CATEGORY	DESCRIPTION
Input Validation	How do you know that the input your application receives is valid and safe? Input validation refers to how your application filters, scrubs, or rejects input before additional processing. Consider constraining input through entry points and encoding output through exit points. Do you trust data from sources such as databases and file shares?
Sensitive Data	How does your application handle sensitive data? Sensitive data refers to how your application handles any data that must be protected either in memory, over the network, or in persistent stores
Session Management	How does your application handle and protect user sessions? A session refers to a series of related interactions between a user and your Web application

This helps you identify:

- Where are the most common mistakes made
- Where are the most actionable improvements

As a result, you use these categories to focus and prioritize your security work, so that if you know the most prevalent security issues occur in the input validation, authentication and authorization categories, you can start there. For more information visit **this patent link**

Next steps

Visit **Threat Modeling Tool Threats** to learn more about the threat categories the tool uses to generate possible design threats.

Security Frame: Auditing and Logging | Mitigations

3/5/2021 • 8 minutes to read • Edit Online

PRODUCT/SERVICE	ARTICLE
Dynamics CRM	• Identify sensitive entities in your solution and implement change auditing
Web Application	• Ensure that auditing and logging is enforced on the application • Ensure that log rotation and separation are in place • Ensure that the application does not log sensitive user data • Ensure that Audit and Log Files have Restricted Access • Ensure that User Management Events are Logged • Ensure that the system has inbuilt defenses against misuse • Enable diagnostics logging for web apps in Azure App Service
Database	• Ensure that login auditing is enabled on SQL Server • Enable Threat detection on Azure SQL
Azure Storage	• Use Azure Storage Analytics to audit access of Azure Storage
WCF	• Implement sufficient Logging • Implement sufficient Audit Failure Handling
Web API	• Ensure that auditing and logging is enforced on Web API
IoT Field Gateway	• Ensure that appropriate auditing and logging is enforced on Field Gateway
IoT Cloud Gateway	• Ensure that appropriate auditing and logging is enforced on Cloud Gateway

Identify sensitive entities in your solution and implement change auditing

TITLE	DETAILS
Component	Dynamics CRM

TITLE	DETAILS
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Identify entities in your solution containing sensitive data and implement change auditing on those entities and fields

Ensure that auditing and logging is enforced on the application

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Enable auditing and logging on all components. Audit logs should capture user context. Identify all important events and log those events. Implement centralized logging

Ensure that log rotation and separation are in place

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	Log rotation is an automated process used in system administration in which dated log files are archived. Servers which run large applications often log every request: in the face of bulky logs, log rotation is a way to limit the total size of the logs while still allowing analysis of recent events. Log separation basically means that you have to store your log files on a different partition as where your OS/application is running on in order to avert a Denial of service attack or the downgrading of your application its performance

Ensure that the application does not log sensitive user data

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Check that you do not log any sensitive data that a user submits to your site. Check for intentional logging as well as side effects caused by design issues. Examples of sensitive data include: • User Credentials • Social Security number or other identifying information • Credit card numbers or other financial information • Health information • Private keys or other data that could be used to decrypt encrypted information • System or application information that can be used to more effectively attack the application

Ensure that Audit and Log Files have Restricted Access

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic

TITLE	DETAILS
Attributes	N/A
References	N/A
Steps	Check to ensure access rights to log files are appropriately set. Application accounts should have write-only access and operators and support personnel should have read-only access as needed. Administrators accounts are the only accounts which should have full access. Check Windows ACL on log files to ensure they are properly restricted: • Application accounts should have write-only access • Operators and support personnel should have read-only access as needed • Administrators are the only accounts that should have full access

Ensure that User Management Events are Logged

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that the application monitors user management events such as successful and failed user logins, password resets, password changes, account lockout, user registration. Doing this helps to detect and react to potentially suspicious behavior. It also enables to gather operations data; for example, to track who is accessing the application

Ensure that the system has inbuilt defenses against misuse

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic

TITLE	DETAILS
Attributes	N/A
References	N/A
Steps	Controls should be in place which throw security exception in case of application misuse. E.g., If input validation is in place and an attacker attempts to inject malicious code that does not match the regex, a security exception can be thrown which can be an indicative of system misuse For example, it is recommended to have security exceptions logged and actions taken for the following issues: • Input validation • CSRF violations • Brute force (upper limit for number of requests per user per resource) • File upload violations

Enable diagnostics logging for web apps in Azure App Service

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	EnvironmentType - Azure
References	N/A
Steps	Azure provides built-in diagnostics to assist with debugging an App Service web app. It also applies to API apps and mobile apps. App Service web apps provide diagnostic functionality for logging information from both the web server and the web application. These are logically separated into web server diagnostics and application diagnostics

Ensure that login auditing is enabled on SQL Server

TITLE	DETAILS
Component	Database
SDL Phase	Build

TITLE	DETAILS
Applicable Technologies	Generic
Attributes	N/A
References	Configure Login Auditing
Steps	Database Server login auditing must be enabled to detect/confirm password guessing attacks. It is important to capture failed login attempts. Capturing both successful and failed login attempts provides additional benefit during forensic investigations

Enable Threat detection on Azure SQL

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	SQL Azure
Attributes	SQL Version - V12
References	Get Started with SQL Database Threat Detection
Steps	Threat Detection detects anomalous database activities indicating potential security threats to the database. It provides a new layer of security, which enables customers to detect and respond to potential threats as they occur by providing security alerts on anomalous activities. Users can explore the suspicious events using Azure SQL Database Auditing to determine if they result from an attempt to access, breach or exploit data in the database. Threat Detection makes it simple to address potential threats to the database without the need to be a security expert or manage advanced security monitoring systems

Use Azure Storage Analytics to audit access of Azure Storage

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Deployment
Applicable Technologies	Generic

TITLE	DETAILS
Attributes	N/A
References	Using Storage Analytics to monitor authorization type
Steps	For each storage account, one can enable Azure Storage Analytics to perform logging and store metrics data. The storage analytics logs provide important information such as authentication method used by someone when they access storage. This can be really helpful if you are tightly guarding access to storage. For example, in Blob Storage you can set all of the containers to private and implement the use of an SAS service throughout your applications. Then you can check the logs regularly to see if your blobs are accessed using the storage account keys, which may indicate a breach of security, or if the blobs are public but they shouldn't be.

Implement sufficient Logging

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	.NET Framework
Attributes	N/A
References	MSDN, Fortify Kingdom
Steps	The lack of a proper audit trail after a security incident can hamper forensic efforts. Windows Communication Foundation (WCF) offers the ability to log successful and/or failed authentication attempts. Logging failed authentication attempts can warn administrators of potential brute-force attacks. Similarly, logging successful authentication events can provide a useful audit trail when a legitimate account is compromised. Enable WCF's service security audit feature

Example

The following is an example configuration with auditing enabled

```
<system.serviceModel>
    <behaviors>
        <serviceBehaviors>
            <behavior name=""NewBehavior"">
                <serviceSecurityAudit auditLogLocation=""Default""
                suppressAuditFailure=""false""
                serviceAuthorizationAuditLevel=""SuccessAndFailure""
                messageAuthenticationAuditLevel=""SuccessAndFailure"" />
                ...
            </behavior>
        </servicebehaviors>
    </behaviors>
</system.serviceModel>
```

Implement sufficient Audit Failure Handling

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	.NET Framework
Attributes	N/A
References	MSDN, Fortify Kingdom
Steps	Developed solution is configured not to generate an exception when it fails to write to an audit log. If WCF is configured not to throw an exception when it is unable to write to an audit log, the program will not be notified of the failure and auditing of critical security events may not occur.

Example

The `<behavior/>` element of the WCF configuration file below instructs WCF to not notify the application when WCF fails to write to an audit log.

```
<behaviors>
    <serviceBehaviors>
        <behavior name="NewBehavior">
            <serviceSecurityAudit auditLogLocation="Application"
            suppressAuditFailure="true"
            serviceAuthorizationAuditLevel="Success"
            messageAuthenticationAuditLevel="Success" />
        </behavior>
    </serviceBehaviors>
</behaviors>
```

Configure WCF to notify the program whenever it is unable to write to an audit log. The program should have an alternative notification scheme in place to alert the organization that audit trails are not being maintained.

Ensure that auditing and logging is enforced on Web API

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Enable auditing and logging on Web APIs. Audit logs should capture user context. Identify all important events and log those events. Implement centralized logging

Ensure that appropriate auditing and logging is enforced on Field Gateway

TITLE	DETAILS
Component	IoT Field Gateway
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	When multiple devices connect to a Field Gateway, ensure that connection attempts and authentication status (success or failure) for individual devices are logged and maintained on the Field Gateway. Also, in cases where Field Gateway is maintaining the IoT Hub credentials for individual devices, ensure that auditing is performed when these credentials are retrieved.Develop a process to periodically upload the logs to Azure IoT Hub/storage for long term retention.

Ensure that appropriate auditing and logging is enforced on Cloud Gateway

TITLE	DETAILS
Component	IoT Cloud Gateway
SDL Phase	Build
Applicable Technologies	Generic

TITLE	DETAILS
Attributes	N/A
References	Introduction to IoT Hub operations monitoring
Steps	Design for collecting and storing audit data gathered through IoT Hub Operations Monitoring. Enable the following monitoring categories: • Device identity operations • Device-to-cloud communications • Cloud-to-device communications • Connections • File uploads

Security Frame: Authentication | Mitigations

7/15/2021 • 25 minutes to read • Edit Online

PRODUCT/SERVICE	ARTICLE
Web Application	• Consider using a standard authentication mechanism to authenticate to Web Application • Applications must handle failed authentication scenarios securely • Enable step up or adaptive authentication • Ensure that administrative interfaces are appropriately locked down • Implement forgot password functionalities securely • Ensure that password and account policy are implemented • Implement controls to prevent username enumeration
Database	• When possible, use Windows Authentication for connecting to SQL Server • When possible use Azure Active Directory Authentication for Connecting to SQL Database • When SQL authentication mode is used, ensure that account and password policy are enforced on SQL server • Do not use SQL Authentication in contained databases
Azure Event Hub	• Use per device authentication credentials using SaS tokens
Azure Trust Boundary	• Enable Azure AD Multi-Factor Authentication for Azure Administrators
Service Fabric Trust Boundary	• Restrict anonymous access to Service Fabric Cluster • Ensure that Service Fabric client-to-node certificate is different from node-to-node certificate • Use AAD to authenticate clients to service fabric clusters • Ensure that service fabric certificates are obtained from an approved Certificate Authority (CA)
Identity Server	• Use standard authentication scenarios supported by Identity Server • Override the default Identity Server token cache with a scalable alternative
Machine Trust Boundary	• Ensure that deployed application's binaries are digitally signed

PRODUCT/SERVICE	ARTICLE
WCF	• Enable authentication when connecting to MSMQ queues in WCF • WCF-Do not set Message clientCredentialType to none • WCF-Do not set Transport clientCredentialType to none
Web API	• Ensure that standard authentication techniques are used to secure Web APIs
Azure AD	• Use standard authentication scenarios supported by Azure Active Directory • Override the default ADAL token cache with a scalable alternative • Ensure that TokenReplayCache is used to prevent the replay of ADAL authentication tokens • Use ADAL libraries to manage token requests from OAuth2 clients to AAD (or on-premises AD)
IoT Field Gateway	• Authenticate devices connecting to the Field Gateway
IoT Cloud Gateway	• Ensure that devices connecting to Cloud gateway are authenticated • Use per-device authentication credentials
Azure Storage	• Ensure that only the required containers and blobs are given anonymous read access • Grant limited access to objects in Azure storage using SAS or SAP

Consider using a standard authentication mechanism to authenticate to Web Application

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Details	Authentication is the process where an entity proves its identity, typically through credentials, such as a user name and password. There are multiple authentication protocols available which may be considered. Some of them are listed below: • Client certificates • Windows based • Forms based • Federation - ADFS • Federation - Azure AD • Federation - Identity Server Consider using a standard authentication mechanism to identify the source process

Applications must handle failed authentication scenarios securely

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Details	Applications that explicitly authenticate users must handle failed authentication scenarios securely.The authentication mechanism must: • Deny access to privileged resources when authentication fails • Display a generic error message after failed authentication and access denied occurs Test for: • Protection of privileged resources after failed logins • A generic error message is displayed on failed authentication and access denied event(s) • Accounts are disabled after an excessive number of failed attempts

Enable step up or adaptive authentication

TITLE	DETAILS
Component	Web Application

TITLE	DETAILS
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Details	Verify the application has additional authorization (such as step up or adaptive authentication, via multi-factor authentication such as sending OTP in SMS, email etc. or prompting for re-authentication) so the user is challenged before being granted access to sensitive information. This rule also applies for making critical changes to an account or action This also means that the adaptation of authentication has to be implemented in such a manner that the application correctly enforces context-sensitive authorization so as to not allow unauthorized manipulation by means of in example, parameter tampering

Ensure that administrative interfaces are appropriately locked down

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Details	The first solution is to grant access only from a certain source IP range to the administrative interface. If that solution would not be possible than it is always recommended to enforce a step-up or adaptive authentication for logging in into the administrative interface

Implement forgot password functionalities securely

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic

TITLE	DETAILS
Attributes	N/A
References	N/A
Details	The first thing is to verify that forgot password and other recovery paths send a link including a time-limited activation token rather than the password itself. Additional authentication based on soft-tokens (e.g. SMS token, native mobile applications, etc.) can be required as well before the link is sent over. Second, you should not lock out the users account whilst the process of getting a new password is in progress. This could lead to a Denial of service attack whenever an attacker decides to intentionally lock out the users with an automated attack. Third, whenever the new password request was set in progress, the message you display should be generalized in order to prevent username enumeration. Fourth, always disallow the use of old passwords and implement a strong password policy.

Ensure that password and account policy are implemented

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Details	Password and account policy in compliance with organizational policy and best practices should be implemented. To defend against brute-force and dictionary based guessing: Strong password policy must be implemented to ensure that users create complex password (e.g., 12 characters minimum length, alphanumeric and special characters). Account lockout policies may be implemented in the following manner: • **Soft lock-out:** This can be a good option for protecting your users against brute force attacks. For example, whenever the user enters a wrong password three times the application could lock down the account for a minute in order to slow down the process of brute forcing their password making it less profitable for the attacker to proceed. If you were to implement hard lock-out countermeasures for this example you would achieve a "DoS" by permanently locking out accounts. Alternatively, application may generate an OTP (One Time Password) and send it out-of-band (through email, sms etc.) to the user. Another approach may be to implement CAPTCHA after a threshold number of failed attempts is reached. • **Hard lock-out:** This type of lockout should be applied whenever you detect a user attacking your application and counter them by means of permanently locking out their account until a response team had time to do their forensics. After this process you can decide to give the user back their account or take further legal actions against them. This type of approach prevents the attacker from further penetrating your application and infrastructure. To defend against attacks on default and predictable accounts, verify that all keys and passwords are replaceable, and are generated or replaced after installation time. If the application has to auto-generate passwords, ensure that the generated passwords are random and have high entropy.

Implement controls to prevent username enumeration

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic

TITLE	DETAILS
Attributes	N/A
References	N/A
Steps	All error messages should be generalized in order to prevent username enumeration. Also sometimes you cannot avoid information leaking in functionalities such as a registration page. Here you need to use rate-limiting methods like CAPTCHA to prevent an automated attack by an attacker.

When possible, use Windows Authentication for connecting to SQL Server

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	OnPrem
Attributes	SQL Version - All
References	SQL Server - Choose an Authentication Mode
Steps	Windows Authentication uses Kerberos security protocol, provides password policy enforcement with regard to complexity validation for strong passwords, provides support for account lockout, and supports password expiration.

When possible use Azure Active Directory Authentication for Connecting to SQL Database

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	SQL Azure
Attributes	SQL Version - V12
References	Connecting to SQL Database By Using Azure Active Directory Authentication
Steps	**Minimum version:** Azure SQL Database V12 required to allow Azure SQL Database to use AAD Authentication against the Microsoft Directory

When SQL authentication mode is used, ensure that account and password policy are enforced on SQL server

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	SQL Server password policy
Steps	When using SQL Server Authentication, logins are created in SQL Server that are not based on Windows user accounts. Both the user name and the password are created by using SQL Server and stored in SQL Server. SQL Server can use Windows password policy mechanisms. It can apply the same complexity and expiration policies used in Windows to passwords used inside SQL Server.

Do not use SQL Authentication in contained databases

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	OnPrem, SQL Azure
Attributes	SQL Version - MSSQL2012, SQL Version - V12
References	Security Best Practices with Contained Databases
Steps	The absence of an enforced password policy may increase the likelihood of a weak credential being established in a contained database. Leverage Windows Authentication.

Use per device authentication credentials using SaS tokens

TITLE	DETAILS
Component	Azure Event Hub
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A

TITLE	DETAILS
References	Event Hubs authentication and security model overview
Steps	The Event Hubs security model is based on a combination of Shared Access Signature (SAS) tokens and event publishers. The publisher name represents the DeviceID that receives the token. This would help associate the tokens generated with the respective devices. All messages are tagged with originator on service side allowing detection of in-payload origin spoofing attempts. When authenticating devices, generate a per device SaS token scoped to a unique publisher.

Enable Azure AD Multi-Factor Authentication for Azure Administrators

TITLE	DETAILS
Component	Azure Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	What is Azure AD Multi-Factor Authentication?
Steps	Multi-factor authentication (MFA) is a method of authentication that requires more than one verification method and adds a critical second layer of security to user sign-ins and transactions. It works by requiring any two or more of the following verification methods: • Something you know (typically a password) • Something you have (a trusted device that is not easily duplicated, like a phone) • Something you are (biometrics)

Restrict anonymous access to Service Fabric Cluster

TITLE	DETAILS
Component	Service Fabric Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	Environment - Azure

TITLE	DETAILS
References	Service Fabric cluster security scenarios
Steps	Clusters should always be secured to prevent unauthorized users from connecting to your cluster, especially when it has production workloads running on it. While creating a service fabric cluster, ensure that the security mode is set to "secure" and configure the required X.509 server certificate. Creating an "insecure" cluster will allow any anonymous user to connect to it if it exposes management endpoints to the public Internet.

Ensure that Service Fabric client-to-node certificate is different from node-to-node certificate

TITLE	DETAILS
Component	Service Fabric Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	Environment - Azure, Environment - Stand alone
References	Service Fabric Client-to-node certificate security, Connect to a secure cluster using client certificate
Steps	Client-to-node certificate security is configured while creating the cluster either through the Azure portal, Resource Manager templates or a standalone JSON template by specifying an admin client certificate and/or a user client certificate. The admin client and user client certificates you specify should be different than the primary and secondary certificates you specify for Node-to-node security.

Use AAD to authenticate clients to service fabric clusters

TITLE	DETAILS
Component	Service Fabric Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	Environment - Azure

TITLE	DETAILS
References	Cluster security scenarios - Security Recommendations
Steps	Clusters running on Azure can also secure access to the management endpoints using Azure Active Directory (AAD), apart from client certificates. For Azure clusters, it is recommended that you use AAD security to authenticate clients and certificates for node-to-node security.

Ensure that service fabric certificates are obtained from an approved Certificate Authority (CA)

TITLE	DETAILS
Component	Service Fabric Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	Environment - Azure
References	X.509 certificates and Service Fabric
Steps	Service Fabric uses X.509 server certificates for authenticating nodes and clients. Some important things to consider while using certificates in service fabrics: • Certificates used in clusters running production workloads should be created by using a correctly configured Windows Server certificate service or obtained from an approved Certificate Authority (CA). The CA can be an approved external CA or a properly managed internal Public Key Infrastructure (PKI) • Never use any temporary or test certificates in production that are created with tools such as MakeCert.exe • You can use a self-signed certificate, but should only do so for test clusters and not in production

Use standard authentication scenarios supported by Identity Server

TITLE	DETAILS
Component	Identity Server
SDL Phase	Build
Applicable Technologies	Generic

TITLE	DETAILS
Attributes	N/A
References	IdentityServer3 - The Big Picture
Steps	Below are the typical interactions supported by Identity Server: • Browsers communicate with web applications • Web applications communicate with web APIs (sometimes on their own, sometimes on behalf of a user) • Browser-based applications communicate with web APIs • Native applications communicate with web APIs • Server-based applications communicate with web APIs • Web APIs communicate with web APIs (sometimes on their own, sometimes on behalf of a user)

Override the default Identity Server token cache with a scalable alternative

TITLE	DETAILS
Component	Identity Server
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	Identity Server Deployment - Caching

TITLE	DETAILS
Steps	IdentityServer has a simple built-in in-memory cache. While this is good for small scale native apps, it does not scale for mid tier and backend applications for the following reasons: • These applications are accessed by many users at once. Saving all access tokens in the same store creates isolation issues and presents challenges when operating at scale: many users, each with as many tokens as the resources the app accesses on their behalf, can mean huge numbers and very expensive lookup operations • These applications are typically deployed on distributed topologies, where multiple nodes must have access to the same cache • Cached tokens must survive process recycles and deactivations • For all the above reasons, while implementing web apps, it is recommended to override the default Identity Server's token cache with a scalable alternative such as Azure Cache for Redis

Ensure that deployed application's binaries are digitally signed

TITLE	DETAILS
Component	Machine Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that deployed application's binaries are digitally signed so that the integrity of the binaries can be verified

Enable authentication when connecting to MSMQ queues in WCF

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	Generic, NET Framework 3
Attributes	N/A
References	MSDN

TITLE	DETAILS
Steps	Program fails to enable authentication when connecting to MSMQ queues, an attacker can anonymously submit messages to the queue for processing. If authentication is not used to connect to an MSMQ queue used to deliver a message to another program, an attacker could submit an anonymous message that is malicious.

Example

The `<netMsmqBinding/>` element of the WCF configuration file below instructs WCF to disable authentication when connecting to an MSMQ queue for message delivery.

```
<bindings>
    <netMsmqBinding>
        <binding>
            <security>
                <transport msmqAuthenticationMode=""None"" />
            </security>
        </binding>
    </netMsmqBinding>
</bindings>
```

Configure MSMQ to require Windows Domain or Certificate authentication at all times for any incoming or outgoing messages.

Example

The `<netMsmqBinding/>` element of the WCF configuration file below instructs WCF to enable certificate authentication when connecting to an MSMQ queue. The client is authenticated using X.509 certificates. The client certificate must be present in the certificate store of the server.

```
<bindings>
    <netMsmqBinding>
        <binding>
            <security>
                <transport msmqAuthenticationMode=""Certificate"" />
            </security>
        </binding>
    </netMsmqBinding>
</bindings>
```

WCF-Do not set Message clientCredentialType to none

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	.NET Framework 3
Attributes	Client Credential Type - None
References	MSDN, Fortify

TITLE	DETAILS
Steps	The absence of authentication means everyone is able to access this service. A service that does not authenticate its clients allows access to all users. Configure the application to authenticate against client credentials. This can be done by setting the message clientCredentialType to Windows or Certificate.

Example

```
<message clientCredentialType=""Certificate""/>
```

WCF-Do not set Transport clientCredentialType to none

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	Generic, .NET Framework 3
Attributes	Client Credential Type - None
References	MSDN, Fortify
Steps	The absence of authentication means everyone is able to access this service. A service that does not authenticate its clients allows all users to access its functionality. Configure the application to authenticate against client credentials. This can be done by setting the transport clientCredentialType to Windows or Certificate.

Example

```
<transport clientCredentialType=""Certificate""/>
```

Ensure that standard authentication techniques are used to secure Web APIs

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A

TITLE	DETAILS
References	Authentication and Authorization in ASP.NET Web API, External Authentication Services with ASP.NET Web API (C#)
Steps	Authentication is the process where an entity proves its identity, typically through credentials, such as a user name and password. There are multiple authentication protocols available which may be considered. Some of them are listed below: • Client certificates • Windows based • Forms based • Federation - ADFS • Federation - Azure AD • Federation - Identity Server Links in the references section provide low-level details on how each of the authentication schemes can be implemented to secure a Web API.

Use standard authentication scenarios supported by Azure Active Directory

TITLE	DETAILS
Component	Azure AD
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Authentication Scenarios for Azure AD, Azure Active Directory Code Samples, Azure Active Directory developer's guide

TITLE	DETAILS
Steps	Azure Active Directory (Azure AD) simplifies authentication for developers by providing identity as a service, with support for industry-standard protocols such as OAuth 2.0 and OpenID Connect. Below are the five primary application scenarios supported by Azure AD: • Web Browser to Web Application: A user needs to sign in to a web application that is secured by Azure AD • Single Page Application (SPA): A user needs to sign in to a single page application that is secured by Azure AD • Native Application to Web API: A native application that runs on a phone, tablet, or PC needs to authenticate a user to get resources from a web API that is secured by Azure AD • Web Application to Web API: A web application needs to get resources from a web API secured by Azure AD • Daemon or Server Application to Web API: A daemon application or a server application with no web user interface needs to get resources from a web API secured by Azure AD Please refer to the links in the references section for low-level implementation details

Override the default ADAL token cache with a scalable alternative

TITLE	DETAILS
Component	Azure AD
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Modern Authentication with Azure Active Directory for Web Applications, Using Redis as ADAL token cache

TITLE	DETAILS
Steps	The default cache that ADAL (Active Directory Authentication Library) uses is an in-memory cache that relies on a static store, available process-wide. While this works for native applications, it does not scale for mid tier and backend applications for the following reasons: • These applications are accessed by many users at once. Saving all access tokens in the same store creates isolation issues and presents challenges when operating at scale: many users, each with as many tokens as the resources the app accesses on their behalf, can mean huge numbers and very expensive lookup operations • These applications are typically deployed on distributed topologies, where multiple nodes must have access to the same cache • Cached tokens must survive process recycles and deactivations For all the above reasons, while implementing web apps, it is recommended to override the default ADAL token cache with a scalable alternative such as Azure Cache for Redis.

Ensure that TokenReplayCache is used to prevent the replay of ADAL authentication tokens

TITLE	DETAILS
Component	Azure AD
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Modern Authentication with Azure Active Directory for Web Applications

TITLE	DETAILS
Steps	The TokenReplayCache property allows developers to define a token replay cache, a store that can be used for saving tokens for the purpose of verifying that no token can be used more than once. This is a measure against a common attack, the aptly called token replay attack: an attacker intercepting the token sent at sign-in might try to send it to the app again (“replay” it) for establishing a new session. E.g., In OIDC code-grant flow, after successful user authentication, a request to "/signin-oidc" endpoint of the relying party is made with "id_token", "code" and "state" parameters. The relying party validates this request and establishes a new session. If an adversary captures this request and replays it, he/she can establish a successful session and spoof the user. The presence of the nonce in OpenID Connect can limit but not fully eliminate the circumstances in which the attack can be successfully enacted. To protect their applications, developers can provide an implementation of ITokenReplayCache and assign an instance to TokenReplayCache.

Example

```
// ITokenReplayCache defined in ADAL
public interface ITokenReplayCache
{
bool TryAdd(string securityToken, DateTime expiresOn);
bool TryFind(string securityToken);
}
```

Example

Here is a sample implementation of the ITokenReplayCache interface. (Please customize and implement your project-specific caching framework)

```
public class TokenReplayCache : ITokenReplayCache
{
    private readonly ICacheProvider cache; // Your project-specific cache provider
    public TokenReplayCache(ICacheProvider cache)
    {
        this.cache = cache;
    }
    public bool TryAdd(string securityToken, DateTime expiresOn)
    {
        if (this.cache.Get<string>(securityToken) == null)
        {
            this.cache.Set(securityToken, securityToken);
            return true;
        }
        return false;
    }
    public bool TryFind(string securityToken)
    {
        return this.cache.Get<string>(securityToken) != null;
    }
}
```

The implemented cache has to be referenced in OIDC options via the "TokenValidationParameters" property as follows.

```
OpenIdConnectOptions openIdConnectOptions = new OpenIdConnectOptions
{
    AutomaticAuthenticate = true,
    ... // other configuration properties follow..
    TokenValidationParameters = new TokenValidationParameters
    {
        TokenReplayCache = new TokenReplayCache(/*Inject your cache provider*/);
    }
}
```

Please note that to test the effectiveness of this configuration, login into your local OIDC-protected application and capture the request to `"/signin-oidc"` endpoint in fiddler. When the protection is not in place, replaying this request in fiddler will set a new session cookie. When the request is replayed after the TokenReplayCache protection is added, the application will throw an exception as follows:

```
SecurityTokenReplayDetectedException: IDX10228: The securityToken has previously been validated,
securityToken:
'eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsIng1dCI6Ik1uQ19WWmNBVGZNNXBPWWlKSE1iYTlnb0VLWSIsImtpZCI6Ik1uQ1.....
```

Use ADAL libraries to manage token requests from OAuth2 clients to AAD (or on-premises AD)

TITLE	DETAILS
Component	Azure AD
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	ADAL
Steps	The Azure AD authentication Library (ADAL) enables client application developers to easily authenticate users to cloud or on-premises Active Directory (AD), and then obtain access tokens for securing API calls. ADAL has many features that make authentication easier for developers, such as asynchronous support, a configurable token cache that stores access tokens and refresh tokens, automatic token refresh when an access token expires and a refresh token is available, and more. By handling most of the complexity, ADAL can help a developer focus on business logic in their application and easily secure resources without being an expert on security. Separate libraries are available for .NET, JavaScript (client and Node.js), Python, iOS, Android and Java.

Authenticate devices connecting to the Field Gateway

TITLE	DETAILS
Component	IoT Field Gateway
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that each device is authenticated by the Field Gateway before accepting data from them and before facilitating upstream communications with the Cloud Gateway. Also, ensure that devices connect with a per device credential so that individual devices can be uniquely identified.

Ensure that devices connecting to Cloud gateway are authenticated

TITLE	DETAILS
Component	IoT Cloud Gateway
SDL Phase	Build
Applicable Technologies	Generic, C#, Node.JS,
Attributes	N/A, Gateway choice - Azure IoT Hub
References	N/A, Azure IoT hub with .NET, Getting Started with IoT hub and Node JS, Securing IoT with SAS and certificates, Git repository
Steps	• **Generic:** Authenticate the device using Transport Layer Security (TLS) or IPSec. Infrastructure should support using pre-shared key (PSK) on those devices that cannot handle full asymmetric cryptography. Leverage Azure AD, Oauth. • **C#:** When creating a DeviceClient instance, by default, the Create method creates a DeviceClient instance that uses the AMQP protocol to communicate with IoT Hub. To use the HTTPS protocol, use the override of the Create method that enables you to specify the protocol. If you use the HTTPS protocol, you should also add the `Microsoft.AspNet.WebApi.Client` NuGet package to your project to include the `System.Net.Http.Formatting` namespace.

Example

```
static DeviceClient deviceClient;

static string deviceKey = "{device key}";
static string iotHubUri = "{iot hub hostname}";

var messageString = "{message in string format}";
var message = new Message(Encoding.ASCII.GetBytes(messageString));

deviceClient = DeviceClient.Create(iotHubUri, new
DeviceAuthenticationWithRegistrySymmetricKey("myFirstDevice", deviceKey));

await deviceClient.SendEventAsync(message);
```

Example

Node.JS: Authentication

Symmetric key

- Create an IoT hub on Azure
- Create an entry in the device identity registry

```
var device = new iothub.Device(null);
device.deviceId = <DeviceId >
registry.create(device, function(err, deviceInfo, res) {})
```

- Create a simulated device

```
var clientFromConnectionString = require('azure-iot-device-amqp').clientFromConnectionString;
var Message = require('azure-iot-device').Message;
var connectionString = 'HostName=<HostName>DeviceId=<DeviceId>SharedAccessKey=<SharedAccessKey>';
var client = clientFromConnectionString(connectionString);
```

SAS Token

- Gets internally generated when using symmetric key but we can generate and use it explicitly as well
- Define a protocol : `var Http = require('azure-iot-device-http').Http;`
- Create a sas token :

```
resourceUri = encodeURIComponent(resourceUri.toLowerCase()).toLowerCase();
var deviceName = "<deviceName >";
var expires = (Date.now() / 1000) + expiresInMins * 60;
var toSign = resourceUri + '\n' + expires;
// using crypto
var decodedPassword = new Buffer(signingKey, 'base64').toString('binary');
const hmac = crypto.createHmac('sha256', decodedPassword);
hmac.update(toSign);
var base64signature = hmac.digest('base64');
var base64UriEncoded = encodeURIComponent(base64signature);
// construct authorization string
var token = "SharedAccessSignature sr=" + resourceUri + "%2fdevices%2f"+deviceName+"&sig="
+ base64UriEncoded + "&se=" + expires;
if (policyName) token += "&skn="+policyName;
return token;
```

- Connect using sas token:

```
Client.fromSharedAccessSignature(sas, Http);
```

Certificates

- Generate a self signed X509 certificate using any tool such as OpenSSL to generate a .cert and .key files to store the certificate and the key respectively
- Provision a device that accepts secured connection using certificates.

```
var connectionString = '<connectionString>';
var registry = iothub.Registry.fromConnectionString(connectionString);
var deviceJSON = {deviceId:"<deviceId>",
authentication: {
    x509Thumbprint: {
    primaryThumbprint: "<PrimaryThumbprint>",
    secondaryThumbprint: "<SecondaryThumbprint>"
    }
}}
var device = deviceJSON;
registry.create(device, function (err) {});
```

- Connect a device using a certificate

```
var Protocol = require('azure-iot-device-http').Http;
var Client = require('azure-iot-device').Client;
var connectionString = 'HostName=<HostName>DeviceId=<DeviceId>x509=true';
var client = Client.fromConnectionString(connectionString, Protocol);
var options = {
    key: fs.readFileSync('./key.pem', 'utf8'),
    cert: fs.readFileSync('./server.crt', 'utf8')
};
// Calling setOptions with the x509 certificate and key (and optionally, passphrase) will configure
the client //transport to use x509 when connecting to IoT Hub
client.setOptions(options);
//call fn to execute after the connection is set up
client.open(fn);
```

Use per-device authentication credentials

TITLE	DETAILS
Component	IoT Cloud Gateway
SDL Phase	Build
Applicable Technologies	Generic
Attributes	Gateway choice - Azure IoT Hub
References	Azure IoT Hub Security Tokens
Steps	Use per device authentication credentials using SaS tokens based on Device key or Client Certificate, instead of IoT Hub-level shared access policies. This prevents the reuse of authentication tokens of one device or field gateway by another

Ensure that only the required containers and blobs are given anonymous read access

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Build
Applicable Technologies	Generic
Attributes	StorageType - Blob
References	Manage anonymous read access to containers and blobs, Shared Access Signatures, Part 1: Understanding the SAS model
Steps	By default, a container and any blobs within it may be accessed only by the owner of the storage account. To give anonymous users read permissions to a container and its blobs, one can set the container permissions to allow public access. Anonymous users can read blobs within a publicly accessible container without authenticating the request. Containers provide the following options for managing container access: • Full public read access: Container and blob data can be read via anonymous request. Clients can enumerate blobs within the container via anonymous request, but cannot enumerate containers within the storage account. • Public read access for blobs only: Blob data within this container can be read via anonymous request, but container data is not available. Clients cannot enumerate blobs within the container via anonymous request • No public read access: Container and blob data can be read by the account owner only Anonymous access is best for scenarios where certain blobs should always be available for anonymous read access. For finer-grained control, one can create a shared access signature, which enables to delegate restricted access using different permissions and over a specified time interval. Ensure that containers and blobs, which may potentially contain sensitive data, are not given anonymous access accidentally

Grant limited access to objects in Azure storage using SAS or SAP

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A

TITLE	DETAILS
References	Shared Access Signatures, Part 1: Understanding the SAS model, Shared Access Signatures, Part 2: Create and use a SAS with Blob storage, How to delegate access to objects in your account using Shared Access Signatures and Stored Access Policies
Steps	Using a shared access signature (SAS) is a powerful way to grant limited access to objects in a storage account to other clients, without having to expose account access key. The SAS is a URI that encompasses in its query parameters all of the information necessary for authenticated access to a storage resource. To access storage resources with the SAS, the client only needs to pass in the SAS to the appropriate constructor or method. You can use a SAS when you want to provide access to resources in your storage account to a client that can't be trusted with the account key. Your storage account keys include both a primary and secondary key, both of which grant administrative access to your account and all of the resources in it. Exposing either of your account keys opens your account to the possibility of malicious or negligent use. Shared access signatures provide a safe alternative that allows other clients to read, write, and delete data in your storage account according to the permissions you've granted, and without need for the account key. If you have a logical set of parameters that are similar each time, using a Stored Access Policy (SAP) is a better idea. Because using a SAS derived from a Stored Access Policy gives you the ability to revoke that SAS immediately, it is the recommended best practice to always use Stored Access Policies when possible.

Security Frame: Authorization | Mitigations

3/5/2021 • 17 minutes to read • Edit Online

PRODUCT/SERVICE	ARTICLE
Machine Trust Boundary	• Ensure that proper ACLs are configured to restrict unauthorized access to data on the device • Ensure that sensitive user-specific application content is stored in user-profile directory • Ensure that the deployed applications are run with least privileges
Web Application	• Enforce sequential step order when processing business logic flows • Implement rate limiting mechanism to prevent enumeration • Ensure that proper authorization is in place and principle of least privileges is followed • Business logic and resource access authorization decisions should not be based on incoming request parameters • Ensure that content and resources are not enumerable or accessible via forceful browsing
Database	• Ensure that least-privileged accounts are used to connect to Database server • Implement Row Level Security RLS to prevent tenants from accessing each other's data • Sysadmin role should only have valid necessary users
IoT Cloud Gateway	• Connect to Cloud Gateway using least-privileged tokens
Azure Event Hub	• Use a send-only permissions SAS Key for generating device tokens • Do not use access tokens that provide direct access to the Event Hub • Connect to Event Hub using SAS keys that have the minimum permissions required
Azure Document DB	• Use resource tokens to connect to Azure Cosmos DB whenever possible
Azure Trust Boundary	• Enable fine-grained access management to Azure Subscription using Azure RBAC
Service Fabric Trust Boundary	• Restrict client's access to cluster operations using Azure RBAC

PRODUCT/SERVICE	ARTICLE
Dynamics CRM	• Perform security modeling and use Field Level Security where required
Dynamics CRM Portal	• Perform security modeling of portal accounts keeping in mind that the security model for the portal differs from the rest of CRM
Azure Storage	• Grant fine-grained permission on a range of entities in Azure Table Storage • Enable Azure role-based access control (Azure RBAC) to Azure storage account using Azure Resource Manager
Mobile Client	• Implement implicit jailbreak or rooting detection
WCF	• Weak Class Reference in WCF • WCF-Implement Authorization control
Web API	• Implement proper authorization mechanism in ASP.NET Web API
IoT Device	• Perform authorization checks in the device if it supports various actions that require different permission levels
IoT Field Gateway	• Perform authorization checks in the Field Gateway if it supports various actions that require different permission levels

Ensure that proper ACLs are configured to restrict unauthorized access to data on the device

TITLE	DETAILS
Component	Machine Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that proper ACLs are configured to restrict unauthorized access to data on the device

Ensure that sensitive user-specific application content is stored in user-profile directory

TITLE	DETAILS
Component	Machine Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that sensitive user-specific application content is stored in user-profile directory. This is to prevent multiple users of the machine from accessing each other's data.

Ensure that the deployed applications are run with least privileges

TITLE	DETAILS
Component	Machine Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that the deployed application is run with least privileges.

Enforce sequential step order when processing business logic flows

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	In order to verify that this stage was run through by a genuine user you want to enforce the application to only process business logic flows in sequential step order, with all steps being processed in realistic human time, and not process out of order, skipped steps, processed steps from another user, or too quickly submitted transactions.

Implement rate limiting mechanism to prevent enumeration

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that sensitive identifiers are random. Implement CAPTCHA control on anonymous pages. Ensure that error and exception should not reveal specific data

Ensure that proper authorization is in place and principle of least privileges is followed

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	The principle means giving a user account only those privileges which are essential to that users work. For example, a backup user does not need to install software: hence, the backup user has rights only to run backup and backup-related applications. Any other privileges, such as installing new software, are blocked. The principle applies also to a personal computer user who usually does work in a normal user account, and opens a privileged, password protected account (that is, a superuser) only when the situation absolutely demands it. This principle can also be applied to your web-applications. Instead of solely depending on role-based authentication methods using sessions, we rather want to assign privileges to users by means of a Database-Based Authentication system. We still use sessions in order to identify if the user was logged in correctly, only now instead of assigning that user with a specific role we assign him with privileges to verify which actions he is privileged to perform on the system. Also a big pro of this method is, whenever a user has to be assigned fewer privileges your changes will be applied on the fly since the assigning does not depend on the session which otherwise had to expire first.

Business logic and resource access authorization decisions should not be based on incoming request parameters

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Whenever you are checking whether a user is restricted to review certain data, the access restrictions should be processed server-side. The userID should be stored inside of a session variable on login and should be used to retrieve user data from the database

Example

```
SELECT data
FROM personaldata
WHERE userID=:id < - session var
```

Now an possible attacker can not tamper and change the application operation since the identifier for retrieving

the data is handled server-side.

Ensure that content and resources are not enumerable or accessible via forceful browsing

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Sensitive static and configuration files should not be kept in the web-root. For content not required to be public, either proper access controls should be applied or removal of the content itself. Also, forceful browsing is usually combined with Brute Force techniques to gather information by attempting to access as many URLs as possible to enumerate directories and files on a server. Attackers may check for all variations of commonly existing files. For example, a password file search would encompass files including psswd.txt, password.htm, password.dat, and other variations. To mitigate this, capabilities for detection of brute force attempts should be included.

Ensure that least-privileged accounts are used to connect to Database server

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	SQL permissions hierarchy, SQL securables
Steps	Least-privileged accounts should be used to connect to the database. Application login should be restricted in the database and should only execute selected stored procedures. Application's login should have no direct table access.

Implement Row Level Security RLS to prevent tenants from accessing each other's data

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Sql Azure, OnPrem
Attributes	SQL Version - V12, SQL Version - MsSQL2016
References	SQL Server Row-Level Security (RLS)
Steps	Row-Level Security enables customers to control access to rows in a database table based on the characteristics of the user executing a query (e.g., group membership or execution context). Row-Level Security (RLS) simplifies the design and coding of security in your application. RLS enables you to implement restrictions on data row access. For example ensuring that workers can access only those data rows that are pertinent to their department, or restricting a customer's data access to only the data relevant to their company. The access restriction logic is located in the database tier rather than away from the data in another application tier. The database system applies the access restrictions every time that data access is attempted from any tier. This makes the security system more reliable and robust by reducing the surface area of the security system.

Please note that RLS as an out-of-the-box database feature is applicable only to SQL Server starting 2016, Azure SQL Database, and SQL Managed Instance. If the out-of-the-box RLS feature is not implemented, it should be ensured that data access is restricted Using Views and Procedures

Sysadmin role should only have valid necessary users

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	SQL permissions hierarchy, SQL securables

TITLE	DETAILS
Steps	Members of the SysAdmin fixed server role should be very limited and never contain accounts used by applications. Please review the list of users in the role and remove any unnecessary accounts

Connect to Cloud Gateway using least-privileged tokens

TITLE	DETAILS
Component	IoT Cloud Gateway
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	Gateway choice - Azure IoT Hub
References	Iot Hub Access Control
Steps	Provide least privilege permissions to various components that connect to Cloud Gateway (IoT Hub). Typical example is – Device management/provisioning component uses registryread/write, Event Processor (ASA) uses Service Connect. Individual devices connect using Device credentials

Use a send-only permissions SAS Key for generating device tokens

TITLE	DETAILS
Component	Azure Event Hub
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Event Hubs authentication and security model overview
Steps	A SAS key is used to generate individual device tokens. Use a send-only permissions SAS key while generating the device token for a given publisher

Do not use access tokens that provide direct access to the Event Hub

TITLE	DETAILS
Component	Azure Event Hub
SDL Phase	Build

TITLE	DETAILS
Applicable Technologies	Generic
Attributes	N/A
References	Event Hubs authentication and security model overview
Steps	A token that grants direct access to the event hub should not be given to the device. Using a least privileged token for the device that gives access only to a publisher would help identify and disallow it if found to be a rogue or compromised device.

Connect to Event Hub using SAS keys that have the minimum permissions required

TITLE	DETAILS
Component	Azure Event Hub
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Event Hubs authentication and security model overview
Steps	Provide least privilege permissions to various back-end applications that connect to the Event Hub. Generate separate SAS keys for each back-end application and only provide the required permissions - Send, Receive or Manage to them.

Use resource tokens to connect to Cosmos DB whenever possible

TITLE	DETAILS
Component	Azure Document DB
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	A resource token is associated with an Azure Cosmos DB permission resource and captures the relationship between the user of a database and the permission that user has for a specific Azure Cosmos DB application resource (e.g. collection, document). Always use a resource token to access the Azure Cosmos DB if the client cannot be trusted with handling master or read-only keys - like an end user application like a mobile or desktop client.Use Master key or read-only keys from backend applications which can store these keys securely.

Enable fine-grained access management to Azure Subscription using Azure RBAC

TITLE	DETAILS
Component	Azure Trust Boundary
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Assign Azure roles to manage access to your Azure subscription resources
Steps	Azure role-based access control (Azure RBAC) enables fine-grained access management for Azure. Using Azure RBAC, you can grant only the amount of access that users need to perform their jobs.

Restrict client's access to cluster operations using Service Fabric RBAC

TITLE	DETAILS
Component	Service Fabric Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	Environment - Azure
References	Service Fabric role-based access control for Service Fabric clients

TITLE	DETAILS
Steps	Azure Service Fabric supports two different access control types for clients that are connected to a Service Fabric cluster: administrator and user. Access control allows the cluster administrator to limit access to certain cluster operations for different groups of users, making the cluster more secure. Administrators have full access to management capabilities (including read/write capabilities). Users, by default, have only read access to management capabilities (for example, query capabilities), and the ability to resolve applications and services. You specify the two client roles (administrator and client) at the time of cluster creation by providing separate certificates for each.

Perform security modeling and use Field Level Security where required

TITLE	DETAILS
Component	Dynamics CRM
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Perform security modeling and use Field Level Security where required

Perform security modeling of portal accounts keeping in mind that the security model for the portal differs from the rest of CRM

TITLE	DETAILS
Component	Dynamics CRM Portal
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	Perform security modeling of portal accounts keeping in mind that the security model for the portal differs from the rest of CRM

Grant fine-grained permission on a range of entities in Azure Table Storage

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Build
Applicable Technologies	Generic
Attributes	StorageType - Table
References	How to delegate access to objects in your Azure storage account using SAS
Steps	In certain business scenarios, Azure Table Storage may be required to store sensitive data that caters to different parties. E.g., sensitive data pertaining to different countries/regions. In such cases, SAS signatures can be constructed by specifying the partition and row key ranges, such that a user can access data specific to a particular country/region.

Enable Azure role-based access control (Azure RBAC) to Azure storage account using Azure Resource Manager

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	How to secure your storage account with Azure role-based access control (Azure RBAC)

TITLE	DETAILS
Steps	When you create a new storage account, you select a deployment model of Classic or Azure Resource Manager. The Classic model of creating resources in Azure only allows all-or-nothing access to the subscription, and in turn, the storage account. With the Azure Resource Manager model, you put the storage account in a resource group and control access to the management plane of that specific storage account using Azure Active Directory. For example, you can give specific users the ability to access the storage account keys, while other users can view information about the storage account, but cannot access the storage account keys.

Implement implicit jailbreak or rooting detection

TITLE	DETAILS
Component	Mobile Client
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Application should safeguard its own configuration and user data in case if phone is rooted or jail broken. Rooting/jail breaking implies unauthorized access, which normal users won't do on their own phones. Therefore application should have implicit detection logic on application startup, to detect if the phone has been rooted. The detection logic can be simply accessing files which normally only root user can access, for example: • /system/app/Superuser.apk • /sbin/su • /system/bin/su • /system/xbin/su • /data/local/xbin/su • /data/local/bin/su • /system/sd/xbin/su • /system/bin/failsafe/su • /data/local/su If the application can access any of these files, it denotes that the application is running as root user.

Weak Class Reference in WCF

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	Generic, NET Framework 3
Attributes	N/A
References	MSDN, Fortify Kingdom
Steps	The system uses a weak class reference, which might allow an attacker to execute unauthorized code. The program references a user-defined class that is not uniquely identified. When .NET loads this weakly identified class, the CLR type loader searches for the class in the following locations in the specified order: 1. If the assembly of the type is known, the loader searches the configuration file's redirect locations, GAC, the current assembly using configuration information, and the application base directory 2. If the assembly is unknown, the loader searches the current assembly, mscorlib, and the location returned by the TypeResolve event handler 3. This CLR search order can be modified with hooks such as the Type Forwarding mechanism and the AppDomain.TypeResolve event If an attacker exploits the CLR search order by creating an alternative class with the same name and placing it in an alternative location that the CLR will load first, the CLR will unintentionally execute the attacker-supplied code

Example

The `<behaviorExtensions/>` element of the WCF configuration file below instructs WCF to add a custom behavior class to a particular WCF extension.

```
<system.serviceModel>
    <extensions>
        <behaviorExtensions>
            <add name=""myBehavior"" type=""MyBehavior"" />
        </behaviorExtensions>
    </extensions>
</system.serviceModel>
```

Using fully qualified (strong) names uniquely identifies a type and further increases security of your system. Use fully qualified assembly names when registering types in the machine.config and app.config files.

Example

The `<behaviorExtensions/>` element of the WCF configuration file below instructs WCF to add strongly-referenced custom behavior class to a particular WCF extension.

```
<system.serviceModel>
    <extensions>
        <behaviorExtensions>
            <add name=""myBehavior"" type=""Microsoft.ServiceModel.Samples.MyBehaviorSection, MyBehavior,
            Version=1.0.0.0, Culture=neutral, PublicKeyToken=null"" />
        </behaviorExtensions>
    </extensions>
</system.serviceModel>
```

WCF-Implement Authorization control

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	Generic, NET Framework 3
Attributes	N/A
References	MSDN, Fortify Kingdom
Steps	This service does not use an authorization control. When a client calls a particular WCF service, WCF provides various authorization schemes that verify that the caller has permission to execute the service method on the server. If authorization controls are not enabled for WCF services, an authenticated user can achieve privilege escalation.

Example

The following configuration instructs WCF to not check the authorization level of the client when executing the service:

```
<behaviors>
    <serviceBehaviors>
        <behavior>
            ...
            <serviceAuthorization principalPermissionMode=""None"" />
        </behavior>
    </serviceBehaviors>
</behaviors>
```

Use a service authorization scheme to verify that the caller of the service method is authorized to do so. WCF provides two modes and allows the definition of a custom authorization scheme. The UseWindowsGroups mode uses Windows roles and users and the UseAspNetRoles mode uses an ASP.NET role provider, such as SQL Server, to authenticate.

Example

The following configuration instructs WCF to make sure that the client is part of the Administrators group before executing the Add service:

```
<behaviors>
    <serviceBehaviors>
        <behavior>
            ...
            <serviceAuthorization principalPermissionMode=""UseWindowsGroups"" />
        </behavior>
    </serviceBehaviors>
</behaviors>
```

The service is then declared as the following:

```
[PrincipalPermission(SecurityAction.Demand,
Role = ""Builtin\\Administrators"")]
public double Add(double n1, double n2)
{
double result = n1 + n2;
return result;
}
```

Implement proper authorization mechanism in ASP.NET Web API

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	Generic, MVC5
Attributes	N/A, Identity Provider - ADFS, Identity Provider - Azure AD
References	Authentication and Authorization in ASP.NET Web API
Steps	Role information for the application users can be derived from Azure AD or ADFS claims if the application relies on them as Identity provider or the application itself might provided it. In any of these cases, the custom authorization implementation should validate the user role information. Role information for the application users can be derived from Azure AD or ADFS claims if the application relies on them as Identity provider or the application itself might provided it. In any of these cases, the custom authorization implementation should validate the user role information.

Example

```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method, Inherited = true, AllowMultiple = true)]
public class ApiAuthorizeAttribute : System.Web.Http.AuthorizeAttribute
{
        public async override Task OnAuthorizationAsync(HttpActionContext actionContext, CancellationToken
cancellationToken)
        {
            if (actionContext == null)
            {
                throw new Exception();
            }

            if (!string.IsNullOrEmpty(base.Roles))
            {
                bool isAuthorized = ValidateRoles(actionContext);
                if (!isAuthorized)
                {
                    HandleUnauthorizedRequest(actionContext);
                }
            }

            base.OnAuthorization(actionContext);
        }

public bool ValidateRoles(actionContext)
{
   //Authorization logic here; returns true or false
}

}
```

All the controllers and action methods which needs to protected should be decorated with above attribute.

```
[ApiAuthorize]
public class CustomController : ApiController
{
     //Application code goes here
}
```

Perform authorization checks in the device if it supports various actions that require different permission levels

TITLE	DETAILS
Component	IoT Device
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	The Device should authorize the caller to check if the caller has the required permissions to perform the action requested. For e.g. Lets say the device is a Smart Door Lock that can be monitored from the cloud, plus it provides functionalities like Remotely locking the door. The Smart Door Lock provides unlocking functionality only when someone physically comes near the door with a Card. In this case, the implementation of the remote command and control should be done in such a way that it does not provide any functionality to unlock the door as the cloud gateway is not authorized to send a command to unlock the door.

Perform authorization checks in the Field Gateway if it supports various actions that require different permission levels

TITLE	DETAILS
Component	IoT Field Gateway
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	The Field Gateway should authorize the caller to check if the caller has the required permissions to perform the action requested. For e.g. there should be different permissions for an admin user interface/API used to configure a field gateway v/s devices that connect to it.

Security Frame: Communication Security | Mitigations

8/31/2021 • 14 minutes to read • Edit Online

PRODUCT/SERVICE	ARTICLE
Azure Event Hub	• Secure communication to Event Hub using SSL/TLS
Dynamics CRM	• Check service account privileges and check that the custom Services or ASP.NET Pages respect CRM's security
Azure Data Factory	• Use Data management gateway while connecting On-premises SQL Server to Azure Data Factory
Identity Server	• Ensure that all traffic to Identity Server is over HTTPS connection
Web Application	• Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections • Configure TLS/SSL certificate for custom domain in Azure App Service • Force all traffic to Azure App Service over HTTPS connection • Enable HTTP Strict Transport Security (HSTS)
Database	• Ensure SQL server connection encryption and certificate validation • Force Encrypted communication to SQL server
Azure Storage	• Ensure that communication to Azure Storage is over HTTPS • Validate MD5 hash after downloading blob if HTTPS cannot be enabled • Use SMB 3.0 compatible client to ensure in-transit data encryption to Azure File Shares
Mobile Client	• Implement Certificate Pinning
WCF	• Enable HTTPS - Secure Transport channel • WCF: Set Message security Protection level to EncryptAndSign • WCF: Use a least-privileged account to run your WCF service

PRODUCT/SERVICE	ARTICLE
Web API	• Force all traffic to Web APIs over HTTPS connection
Azure Cache for Redis	• Ensure that communication to Azure Cache for Redis is over TLS
IoT Field Gateway	• Secure Device to Field Gateway communication
IoT Cloud Gateway	• Secure Device to Cloud Gateway communication using SSL/TLS

Secure communication to Event Hub using SSL/TLS

TITLE	DETAILS
Component	Azure Event Hub
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Event Hubs authentication and security model overview
Steps	Secure AMQP or HTTP connections to Event Hub using SSL/TLS

Check service account privileges and check that the custom Services or ASP.NET Pages respect CRM's security

TITLE	DETAILS
Component	Dynamics CRM
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Check service account privileges and check that the custom Services or ASP.NET Pages respect CRM's security

Use Data management gateway while connecting On-premises SQL

Server to Azure Data Factory

TITLE	DETAILS
Component	Azure Data Factory
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	Linked Service Types - Azure and On-premises
References	Moving data between On-premises and Azure Data Factory, Data management gateway
Steps	The Data Management Gateway (DMG) tool is required to connect to data sources which are protected behind corpnet or a firewall. 1. Locking down the machine isolates the DMG tool and prevents malfunctioning programs from damaging or snooping on the data source machine. (E.g. latest updates must be installed, enable minimum required ports, controlled accounts provisioning, auditing enabled, disk encryption enabled etc.) 2. Data Gateway key must be rotated at frequent intervals or whenever the DMG service account password renews 3. Data transits through Link Service must be encrypted

Ensure that all traffic to Identity Server is over HTTPS connection

TITLE	DETAILS
Component	Identity Server
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	IdentityServer3 - Keys, Signatures and Cryptography, IdentityServer3 - Deployment
Steps	By default, IdentityServer requires all incoming connections to come over HTTPS. It is absolutely mandatory that communication with IdentityServer is done over secured transports only. There are certain deployment scenarios like TLS offloading where this requirement can be relaxed. See the Identity Server deployment page in the references for more information.

Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Applications that use SSL, TLS, or DTLS must fully verify the X.509 certificates of the entities they connect to. This includes verification of the certificates for: • Domain name • Validity dates (both beginning and expiration dates) • Revocation status • Usage (for example, Server Authentication for servers, Client Authentication for clients) • Trust chain. Certificates must chain to a root certification authority (CA) that is trusted by the platform or explicitly configured by the administrator • Key length of certificate's public key must be >2048 bits • Hashing algorithm must be SHA256 and above

Configure TLS/SSL certificate for custom domain in Azure App Service

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	EnvironmentType - Azure
References	Enable HTTPS for an app in Azure App Service
Steps	By default, Azure already enables HTTPS for every app with a wildcard certificate for the *.azurewebsites.net domain. However, like all wildcard domains, it is not as secure as using a custom domain with own certificate Refer. It is recommended to enable TLS for the custom domain which the deployed app will be accessed through

Force all traffic to Azure App Service over HTTPS connection

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	EnvironmentType - Azure
References	Enforce HTTPS on Azure App Service
Steps	Though Azure already enables HTTPS for Azure app services with a wildcard certificate for the domain *.azurewebsites.net, it do not enforce HTTPS. Visitors may still access the app using HTTP, which may compromise the app's security and hence HTTPS has to be enforced explicitly. ASP.NET MVC applications should use the RequireHttps filter that forces an unsecured HTTP request to be re-sent over HTTPS. Alternatively, the URL Rewrite module, which is included with Azure App Service can be used to enforce HTTPS. URL Rewrite module enables developers to define rules that are applied to incoming requests before the requests are handed to your application. URL Rewrite rules are defined in a web.config file stored in the root of the application

Example

The following example contains a basic URL Rewrite rule that forces all incoming traffic to use HTTPS

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="Force HTTPS" enabled="true">
          <match url="(.*)" ignoreCase="false" />
          <conditions>
            <add input="{HTTPS}" pattern="off" />
          </conditions>
          <action type="Redirect" url="https://{HTTP_HOST}/{R:1}" appendQueryString="true"
redirectType="Permanent" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

This rule works by returning an HTTP status code of 301 (permanent redirect) when the user requests a page using HTTP. The 301 redirects the request to the same URL as the visitor requested, but replaces the HTTP portion of the request with HTTPS. For example, `HTTP://contoso.com` would be redirected to `HTTPS://contoso.com`.

Enable HTTP Strict Transport Security (HSTS)

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	OWASP HTTP Strict Transport Security Cheat Sheet
Steps	HTTP Strict Transport Security (HSTS) is an opt-in security enhancement that is specified by a web application through the use of a special response header. Once a supported browser receives this header that browser will prevent any communications from being sent over HTTP to the specified domain and will instead send all communications over HTTPS. It also prevents HTTPS click through prompts on browsers. To implement HSTS, the following response header has to be configured for a website globally, either in code or in config. Strict-Transport-Security: max-age=300; includeSubDomains HSTS addresses the following threats: • User bookmarks or manually types `https://example.com` and is subject to a man-in-the-middle attacker: HSTS automatically redirects HTTP requests to HTTPS for the target domain • Web application that is intended to be purely HTTPS inadvertently contains HTTP links or serves content over HTTP: HSTS automatically redirects HTTP requests to HTTPS for the target domain • A man-in-the-middle attacker attempts to intercept traffic from a victim user using an invalid certificate and hopes the user will accept the bad certificate: HSTS does not allow a user to override the invalid certificate message

Ensure SQL server connection encryption and certificate validation

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	SQL Azure
Attributes	SQL Version - V12
References	Best Practices on Writing Secure Connection Strings for SQL Database

TITLE	DETAILS
Steps	All communications between SQL Database and a client application are encrypted using Transport Layer Security (TLS), previously known as Secure Sockets Layer (SSL), at all times. SQL Database doesn't support unencrypted connections. To validate certificates with application code or tools, explicitly request an encrypted connection and do not trust the server certificates. If your application code or tools do not request an encrypted connection, they will still receive encrypted connections However, they may not validate the server certificates and thus will be susceptible to "man in the middle" attacks. To validate certificates with ADO.NET application code, set `Encrypt=True` and `TrustServerCertificate=False` in the database connection string. To validate certificates via SQL Server Management Studio, open the Connect to Server dialog box. Click Encrypt connection on the Connection Properties tab

Force Encrypted communication to SQL server

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	OnPrem
Attributes	SQL Version - MsSQL2016, SQL Version - MsSQL2012, SQL Version - MsSQL2014
References	Enable Encrypted Connections to the Database Engine
Steps	Enabling TLS encryption increases the security of data transmitted across networks between instances of SQL Server and applications.

Ensure that communication to Azure Storage is over HTTPS

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	Azure Storage Transport-Level Encryption – Using HTTPS

TITLE	DETAILS
Steps	To ensure the security of Azure Storage data in-transit, always use the HTTPS protocol when calling the REST APIs or accessing objects in storage. Also, Shared Access Signatures, which can be used to delegate access to Azure Storage objects, include an option to specify that only the HTTPS protocol can be used when using Shared Access Signatures, ensuring that anybody sending out links with SAS tokens will use the proper protocol.

Validate MD5 hash after downloading blob if HTTPS cannot be enabled

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Build
Applicable Technologies	Generic
Attributes	StorageType - Blob
References	Windows Azure Blob MD5 Overview
Steps	Windows Azure Blob service provides mechanisms to ensure data integrity both at the application and transport layers. If for any reason you need to use HTTP instead of HTTPS and you are working with block blobs, you can use MD5 checking to help verify the integrity of the blobs being transferred This will help with protection from network/transport layer errors, but not necessarily with intermediary attacks. If you can use HTTPS, which provides transport level security, then using MD5 checking is redundant and unnecessary.

Use SMB 3.x compatible client to ensure in-transit data encryption to Azure file shares

TITLE	DETAILS
Component	Mobile Client
SDL Phase	Build
Applicable Technologies	Generic
Attributes	StorageType - File
References	Azure Files, Azure Files SMB Support for Windows Clients

TITLE	DETAILS
Steps	Azure Files supports HTTPS when using the REST API, but is more commonly used as an SMB file share attached to a VM. SMB 2.1 does not support encryption, so connections are only allowed within the same region in Azure. However, SMB 3.x supports encryption, and can be used with Windows Server 2012 R2, Windows 8, Windows 8.1, and Windows 10, allowing cross-region access and even access on the desktop.

Implement Certificate Pinning

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Build
Applicable Technologies	Generic, Windows Phone
Attributes	N/A
References	Certificate and Public Key Pinning
Steps	Certificate pinning defends against Man-In-The-Middle (MITM) attacks. Pinning is the process of associating a host with their expected X509 certificate or public key. Once a certificate or public key is known or seen for a host, the certificate or public key is associated or 'pinned' to the host. Thus, when an adversary attempts to do TLS MITM attack, during TLS handshake the key from attacker's server will be different from the pinned certificate's key, and the request will be discarded, thus preventing MITM Certificate pinning can be achieved by implementing ServicePointManager's `ServerCertificateValidationCallback` delegate.

Example

```
using System;
using System.Net;
using System.Net.Security;
using System.Security.Cryptography;

namespace CertificatePinningExample
{
    class CertificatePinningExample
    {
        /* Note: In this example, we're hardcoding a the certificate's public key and algorithm for
           demonstration purposes. In a real-world application, this should be stored in a secure
           configuration area that can be updated as needed. */

        private static readonly string PINNED_ALGORITHM = "RSA";

        private static readonly string PINNED_PUBLIC_KEY = "3082010A0282010100B0E75B7CBE56D31658EF79B3A1" +
            "294D506A88DFCDD603F6EF15E7F5BCBDF32291EC50B2B82BA158E905FE6A83EE044A48258B07FAC3D6356AF09B2" +
            "3EDAB15D00507B70DB08DB9A20C7D1201417B3071A346D663A241061C151B6EC5B5B4ECCCDCDBEA24F051962809" +
            "FEC499BF2D093C06E3BDA7D0BB83CDC1C2C6660B8ECB2EA30A685ADE2DC83C88314010FFC7F4F0F895EDDBE5C02" +
            "ABF78E50B708E0A0EB984A9AA536BCE61A0C31DB95425C6FEE5A564B158EE7C4F0693C439AE010EF83CA8155750" +
            "09B17537C29F86071E5DD8CA50EBD8A409494F479B07574D83EDCE6F68A8F7D40447471D05BC3F5EAD7862FA748" +
            "EA3C92A60A128344B1CEF7A0B0D94E50203010001";


        public static void Main(string[] args)
        {
            HttpWebRequest request = (HttpWebRequest)WebRequest.Create("https://azure.microsoft.com");
            request.ServerCertificateValidationCallback = (sender, certificate, chain, sslPolicyErrors) =>
            {
                if (certificate == null || sslPolicyErrors != SslPolicyErrors.None)
                {
                    // Error getting certificate or the certificate failed basic validation
                    return false;
                }

                var targetKeyAlgorithm = new Oid(certificate.GetKeyAlgorithm()).FriendlyName;
                var targetPublicKey = certificate.GetPublicKeyString();

                if (targetKeyAlgorithm == PINNED_ALGORITHM &&
                    targetPublicKey == PINNED_PUBLIC_KEY)
                {
                    // Success, the certificate matches the pinned value.
                    return true;
                }
                // Reject, either the key or the algorithm does not match the expected value.
                return false;
            };

            try
            {
                var response = (HttpWebResponse)request.GetResponse();
                Console.WriteLine($"Success, HTTP status code: {response.StatusCode}");
            }
            catch(Exception ex)
            {
                Console.WriteLine($"Failure, {ex.Message}");
            }
            Console.WriteLine("Press any key to end.");
            Console.ReadKey();
        }
    }
}
```

Enable HTTPS - Secure Transport channel

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	NET Framework 3
Attributes	N/A
References	MSDN, Fortify Kingdom
Steps	The application configuration should ensure that HTTPS is used for all access to sensitive information. • **EXPLANATION:** If an application handles sensitive information and does not use message-level encryption, then it should only be allowed to communicate over an encrypted transport channel. • **RECOMMENDATIONS:** Ensure that HTTP transport is disabled and enable HTTPS transport instead. For example, replace the `<httpTransport/>` with `<httpsTransport/>` tag. Do not rely on a network configuration (firewall) to guarantee that the application can only be accessed over a secure channel. From a philosophical point of view, the application should not depend on the network for its security. From a practical point of view, the people responsible for securing the network do not always track the security requirements of the application as they evolve.

WCF: Set Message security Protection level to EncryptAndSign

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	.NET Framework 3
Attributes	N/A
References	MSDN

TITLE	DETAILS
Steps	• **EXPLANATION:** When Protection level is set to "none" it will disable message protection. Confidentiality and integrity is achieved with appropriate level of setting. • **RECOMMENDATIONS:** ◦ when `Mode=None` - Disables message protection ◦ when `Mode=Sign` - Signs but does not encrypt the message; should be used when data integrity is important ◦ when `Mode=EncryptAndSign` - Signs and encrypts the message Consider turning off encryption and only signing your message when you just need to validate the integrity of the information without concerns of confidentiality. This may be useful for operations or service contracts in which you need to validate the original sender but no sensitive data is transmitted. When reducing the protection level, be careful that the message does not contain any personal data.

Example

Configuring the service and the operation to only sign the message is shown in the following examples. Service Contract Example of `ProtectionLevel.Sign`: The following is an example of using ProtectionLevel.Sign at the Service Contract level:

```
[ServiceContract(Protection Level=ProtectionLevel.Sign]
public interface IService
  {
  string GetData(int value);
  }
```

Example

Operation Contract Example of `ProtectionLevel.Sign` (for Granular Control): The following is an example of using `ProtectionLevel.Sign` at the OperationContract level:

```
[OperationContract(ProtectionLevel=ProtectionLevel.Sign]
string GetData(int value);
```

WCF: Use a least-privileged account to run your WCF service

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	.NET Framework 3
Attributes	N/A

TITLE	DETAILS
References	MSDN
Steps	• **EXPLANATION:** Do not run WCF services under admin or high privilege account. in case of services compromise it will result in high impact. • **RECOMMENDATIONS:** Use a least-privileged account to host your WCF service because it will reduce your application's attack surface and reduce the potential damage if you are attacked. If the service account requires additional access rights on infrastructure resources such as MSMQ, the event log, performance counters, and the file system, appropriate permissions should be given to these resources so that the WCF service can run successfully. If your service needs to access specific resources on behalf of the original caller, use impersonation and delegation to flow the caller's identity for a downstream authorization check. In a development scenario, use the local network service account, which is a special built-in account that has reduced privileges. In a production scenario, create a least-privileged custom domain service account.

Force all traffic to Web APIs over HTTPS connection

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	MVC5, MVC6
Attributes	N/A
References	Enforcing SSL in a Web API Controller
Steps	If an application has both an HTTPS and an HTTP binding, clients can still use HTTP to access the site. To prevent this, use an action filter to ensure that requests to protected APIs are always over HTTPS.

Example

The following code shows a Web API authentication filter that checks for TLS:

```
public class RequireHttpsAttribute : AuthorizationFilterAttribute
{
    public override void OnAuthorization(HttpActionContext actionContext)
    {
        if (actionContext.Request.RequestUri.Scheme != Uri.UriSchemeHttps)
        {
            actionContext.Response = new HttpResponseMessage(System.Net.HttpStatusCode.Forbidden)
            {
                ReasonPhrase = "HTTPS Required"
            };
        }
        else
        {
            base.OnAuthorization(actionContext);
        }
    }
}
```

Add this filter to any Web API actions that require TLS:

```
public class ValuesController : ApiController
{
    [RequireHttps]
    public HttpResponseMessage Get() { ... }
}
```

Ensure that communication to Azure Cache for Redis is over TLS

TITLE	DETAILS
Component	Azure Cache for Redis
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Azure Redis TLS support
Steps	Redis server does not support TLS out of the box, but Azure Cache for Redis does. If you are connecting to Azure Cache for Redis and your client supports TLS, like StackExchange.Redis, then you should use TLS. By default non-TLS port is disabled for new Azure Cache for Redis instances. Ensure that the secure defaults are not changed unless there is a dependency on TLS support for redis clients.

Please note that Redis is designed to be accessed by trusted clients inside trusted environments. This means that usually it is not a good idea to expose the Redis instance directly to the internet or, in general, to an environment where untrusted clients can directly access the Redis TCP port or UNIX socket.

Secure Device to Field Gateway communication

TITLE	DETAILS
Component	IoT Field Gateway
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	For IP based devices, the communication protocol could typically be encapsulated in a SSL/TLS channel to protect data in transit. For other protocols that do not support SSL/TLS investigate if there are secure versions of the protocol that provide security at transport or message layer.

Secure Device to Cloud Gateway communication using SSL/TLS

TITLE	DETAILS
Component	IoT Cloud Gateway
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Choose your Communication Protocol
Steps	Secure HTTP/AMQP or MQTT protocols using SSL/TLS.

Security Frame: Configuration Management | Mitigations

3/5/2021 • 20 minutes to read • Edit Online

PRODUCT/SERVICE	ARTICLE
Web Application	• Implement Content Security Policy (CSP), and disable inline javascript • Enable browser's XSS filter • ASP.NET applications must disable tracing and debugging prior to deployment • Access third-party javascripts from trusted sources only • Ensure that authenticated ASP.NET pages incorporate UI Redressing or click-jacking defenses • Ensure that only trusted origins are allowed if CORS is enabled on ASP.NET Web Applications • Enable ValidateRequest attribute on ASP.NET Pages • Use locally-hosted latest versions of JavaScript libraries • Disable automatic MIME sniffing • Remove standard server headers on Windows Azure Web Sites to avoid fingerprinting
Database	• Configure a Windows Firewall for Database Engine Access
Web API	• Ensure that only trusted origins are allowed if CORS is enabled on ASP.NET Web API • Encrypt sections of Web API's configuration files that contain sensitive data
IoT Device	• Ensure that all admin interfaces are secured with strong credentials • Ensure that unknown code cannot execute on devices • Encrypt OS and additional partitions of IoT Device with bit-locker • Ensure that only the minimum services/features are enabled on devices
IoT Field Gateway	• Encrypt OS and additional partitions of IoT Field Gateway with bit-locker • Ensure that the default login credentials of the field gateway are changed during installation
IoT Cloud Gateway	• Ensure that the Cloud Gateway implements a process to keep the connected devices firmware up to date

PRODUCT/SERVICE	ARTICLE
Machine Trust Boundary	• Ensure that devices have end-point security controls configured as per organizational policies
Azure Storage	• Ensure secure management of Azure storage access keys • Ensure that only trusted origins are allowed if CORS is enabled on Azure storage
WCF	• Enable WCF's service throttling feature • WCF-Information disclosure through metadata

Implement Content Security Policy (CSP), and disable inline javascript

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	An Introduction to Content Security Policy, Content Security Policy Reference, Security features, Introduction to content security policy, Can I use CSP?

TITLE	DETAILS
Steps	Content Security Policy (CSP) is a defense-in-depth security mechanism, a W3C standard, that enables web application owners to have control on the content embedded in their site. CSP is added as an HTTP response header on the web server and is enforced on the client side by browsers. It is a allowed list-based policy - a website can declare a set of trusted domains from which active content such as JavaScript can be loaded. CSP provides the following security benefits: • **Protection against XSS:** If a page is vulnerable to XSS, an attacker can exploit it in 2 ways: ◦ Inject `<script>malicious code</script>`. This exploit will not work due to CSP's Base Restriction-1 ◦ Inject `<script src="http://attacker.com/maliciousCode.js"/>`. This exploit will not work since the attacker controlled domain will not be in CSP's allowed list of domains • **Control over data exfiltration:** If any malicious content on a webpage attempts to connect to an external website and steal data, the connection will be aborted by CSP. This is because the target domain will not be in CSP's allowed list • **Defense against click-jacking:** click-jacking is an attack technique using which an adversary can frame a genuine website and force users to click on UI elements. Currently defense against click-jacking is achieved by configuring a response header- X-Frame-Options. Not all browsers respect this header and going forward CSP will be a standard way to defend against click-jacking • **Real-time attack reporting:** If there is an injection attack on a CSP-enabled website, browsers will automatically trigger a notification to an endpoint configured on the webserver. This way, CSP serves as a real-time warning system.

Example

Example policy:

```
Content-Security-Policy: default-src 'self'; script-src 'self' www.google-analytics.com
```

This policy allows scripts to load only from the web application's server and google analytics server. Scripts loaded from any other site will be rejected. When CSP is enabled on a website, the following features are automatically disabled to mitigate XSS attacks.

Example

Inline scripts will not execute. Following are examples of inline scripts

```
<script> some Javascript code </script>
Event handling attributes of HTML tags (e.g., <button onclick="function(){}">
javascript:alert(1);
```

Example

Strings will not be evaluated as code.

```
Example: var str="alert(1)"; eval(str);
```

Enable browser's XSS filter

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	XSS Protection Filter
Steps	X-XSS-Protection response header configuration controls the browser's cross site script filter. This response header can have following values: • `0:` This will disable the filter • `1: Filter enabled` If a cross-site scripting attack is detected, in order to stop the attack, the browser will sanitize the page • `1: mode=block : Filter enabled`. Rather than sanitize the page, when a XSS attack is detected, the browser will prevent rendering of the page • `1: report=http://[YOURDOMAIN]/your_report_URI : Filter enabled`. The browser will sanitize the page and report the violation. This is a Chromium function utilizing CSP violation reports to send details to a URI of your choice. The last 2 options are considered safe values.

ASP.NET applications must disable tracing and debugging prior to deployment

TITLE	DETAILS
Component	Web Application
SDL Phase	Build

TITLE	DETAILS
Applicable Technologies	Generic
Attributes	N/A
References	ASP.NET Debugging Overview, ASP.NET Tracing Overview, How to: Enable Tracing for an ASP.NET Application, How to: Enable Debugging for ASP.NET Applications
Steps	When tracing is enabled for the page, every browser requesting it also obtains the trace information that contains data about internal server state and workflow. That information could be security sensitive. When debugging is enabled for the page, errors happening on the server result in a full stack trace data presented to the browser. That data may expose security-sensitive information about the server's workflow.

Access third-party javascripts from trusted sources only

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	third-party JavaScripts should be referenced only from trusted sources. The reference endpoints should always be on TLS.

Ensure that authenticated ASP.NET pages incorporate UI Redressing or click-jacking defenses

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	OWASP click-jacking Defense Cheat Sheet, IE Internals - Combating click-jacking With X-Frame-Options

TITLE	DETAILS
Steps	click-jacking, also known as a "UI redress attack", is when an attacker uses multiple transparent or opaque layers to trick a user into clicking on a button or link on another page when they were intending to click on the top-level page. This layering is achieved by crafting a malicious page with an iframe, which loads the victim's page. Thus, the attacker is "hijacking" clicks meant for their page and routing them to another page, most likely owned by another application, domain, or both. To prevent click-jacking attacks, set the proper X-Frame-Options HTTP response headers that instruct the browser to not allow framing from other domains

Example

The X-FRAME-OPTIONS header can be set via IIS web.config. Web.config code snippet for sites that should never be framed:

```
<system.webServer>
    <httpProtocol>
        <customHeader>
            <add name="X-FRAME-OPTIONS" value="DENY"/>
        </customHeaders>
    </httpProtocol>
</system.webServer>
```

Example

Web.config code for sites that should only be framed by pages in the same domain:

```
<system.webServer>
    <httpProtocol>
        <customHeader>
            <add name="X-FRAME-OPTIONS" value="SAMEORIGIN"/>
        </customHeaders>
    </httpProtocol>
</system.webServer>
```

Ensure that only trusted origins are allowed if CORS is enabled on ASP.NET Web Applications

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Web Forms, MVC5
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	Browser security prevents a web page from making AJAX requests to another domain. This restriction is called the same-origin policy, and prevents a malicious site from reading sensitive data from another site. However, sometimes it might be required to expose APIs securely which other sites can consume. Cross Origin Resource Sharing (CORS) is a W3C standard that allows a server to relax the same-origin policy. Using CORS, a server can explicitly allow some cross-origin requests while rejecting others. CORS is safer and more flexible than earlier techniques such as JSONP. At its core, enabling CORS translates to adding a few HTTP response headers (Access-Control-*) to the web application and this can be done in a couple of ways.

Example

If access to Web.config is available, then CORS can be added through the following code:

```
<system.webServer>
    <httpProtocol>
      <customHeaders>
        <clear />
        <add name="Access-Control-Allow-Origin" value="https://example.com" />
      </customHeaders>
    </httpProtocol>
```

Example

If access to web.config is not available, then CORS can be configured by adding the following CSharp code:

```
HttpContext.Response.AppendHeader("Access-Control-Allow-Origin", "https://example.com")
```

Please note that it is critical to ensure that the list of origins in "Access-Control-Allow-Origin" attribute is set to a finite and trusted set of origins. Failing to configure this inappropriately (e.g., setting the value as '*') will allow malicious sites to trigger cross origin requests to the web application >without any restrictions, thereby making the application vulnerable to CSRF attacks.

Enable ValidateRequest attribute on ASP.NET Pages

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Web Forms, MVC5
Attributes	N/A
References	Request Validation - Preventing Script Attacks

TITLE	DETAILS
Steps	Request validation, a feature of ASP.NET since version 1.1, prevents the server from accepting content containing un-encoded HTML. This feature is designed to help prevent some script-injection attacks whereby client script code or HTML can be unknowingly submitted to a server, stored, and then presented to other users. We still strongly recommend that you validate all input data and HTML encode it when appropriate. Request validation is performed by comparing all input data to a list of potentially dangerous values. If a match occurs, ASP.NET raises an `HttpRequestValidationException`. By default, Request Validation feature is enabled.

Example

However, this feature can be disabled at page level:

```
<%@ Page validateRequest="false" %>
```

or, at application level

```
<configuration>
   <system.web>
      <pages validateRequest="false" />
   </system.web>
</configuration>
```

Please note that Request Validation feature is not supported, and is not part of MVC6 pipeline.

Use locally-hosted latest versions of JavaScript libraries

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	Developers using standard JavaScript libraries like JQuery must use approved versions of common JavaScript libraries that do not contain known security flaws. A good practice is to use the most latest version of the libraries, since they contain security fixes for known vulnerabilities in their older versions. If the most recent release cannot be used due to compatibility reasons, the below minimum versions should be used. Acceptable minimum versions: • **JQuery** ◦ JQuery 1.7.1 ◦ JQueryUI 1.10.0 ◦ JQuery Validate 1.9 ◦ JQuery Mobile 1.0.1 ◦ JQuery Cycle 2.99 ◦ JQuery DataTables 1.9.0 • **Ajax Control Toolkit** ◦ Ajax Control Toolkit 40412 • **ASP.NET Web Forms and Ajax** ◦ ASP.NET Web Forms and Ajax 4 ◦ ASP.NET Ajax 3.5 • **ASP.NET MVC** ◦ ASP.NET MVC 3.0 Never load any JavaScript library from external sites such as public CDNs

Disable automatic MIME sniffing

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	IE8 Security Part V: Comprehensive Protection, MIME type
Steps	The X-Content-Type-Options header is an HTTP header that allows developers to specify that their content should not be MIME-sniffed. This header is designed to mitigate MIME-Sniffing attacks. For each page that could contain user controllable content, you must use the HTTP Header X-Content-Type-Options:nosniff. To enable the required header globally for all pages in the application, you can do one of the following

Example

Add the header in the web.config file if the application is hosted by Internet Information Services (IIS) 7 onwards.

```
<system.webServer>
<httpProtocol>
<customHeaders>
<add name="X-Content-Type-Options" value="nosniff"/>
</customHeaders>
</httpProtocol>
</system.webServer>
```

Example

Add the header through the global Application_BeginRequest

```
void Application_BeginRequest(object sender, EventArgs e)
{
this.Response.Headers["X-Content-Type-Options"] = "nosniff";
}
```

Example

Implement custom HTTP module

```
public class XContentTypeOptionsModule : IHttpModule
{
#region IHttpModule Members
public void Dispose()
{
}
public void Init(HttpApplication context)
{
context.PreSendRequestHeaders += newEventHandler(context_PreSendRequestHeaders);
}
#endregion
void context_PreSendRequestHeaders(object sender, EventArgs e)
{
HttpApplication application = sender as HttpApplication;
if (application == null)
  return;
if (application.Response.Headers["X-Content-Type-Options "] != null)
  return;
application.Response.Headers.Add("X-Content-Type-Options ", "nosniff");
}
}
```

Example

You can enable the required header only for specific pages by adding it to individual responses:

```
this.Response.Headers["X-Content-Type-Options"] = "nosniff";
```

Remove standard server headers on Windows Azure Web Sites to avoid fingerprinting

TITLE	DETAILS
Component	Web Application

TITLE	DETAILS
SDL Phase	Build
Applicable Technologies	Generic
Attributes	EnvironmentType - Azure
References	Removing standard server headers on Windows Azure Web Sites
Steps	Headers such as Server, X-Powered-By, X-AspNet-Version reveal information about the server and the underlying technologies. It is recommended to suppress these headers thereby preventing fingerprinting the application

Configure a Windows Firewall for Database Engine Access

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	SQL Azure, OnPrem
Attributes	N/A, SQL Version - V12
References	How to configure an Azure SQL Database firewall, Configure a Windows Firewall for Database Engine Access
Steps	Firewall systems help prevent unauthorized access to computer resources. To access an instance of the SQL Server Database Engine through a firewall, you must configure the firewall on the computer running SQL Server to allow access

Ensure that only trusted origins are allowed if CORS is enabled on ASP.NET Web API

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	MVC 5
Attributes	N/A
References	Enabling Cross-Origin Requests in ASP.NET Web API 2, ASP.NET Web API - CORS Support in ASP.NET Web API 2

TITLE	DETAILS
Steps	Browser security prevents a web page from making AJAX requests to another domain. This restriction is called the same-origin policy, and prevents a malicious site from reading sensitive data from another site. However, sometimes it might be required to expose APIs securely which other sites can consume. Cross Origin Resource Sharing (CORS) is a W3C standard that allows a server to relax the same-origin policy. Using CORS, a server can explicitly allow some cross-origin requests while rejecting others. CORS is safer and more flexible than earlier techniques such as JSONP.

Example

In the App_Start/WebApiConfig.cs, add the following code to the WebApiConfig.Register method

```
using System.Web.Http;
namespace WebService
{
    public static class WebApiConfig
    {
        public static void Register(HttpConfiguration config)
        {
            // New code
            config.EnableCors();

            config.Routes.MapHttpRoute(
                name: "DefaultApi",
                routeTemplate: "api/{controller}/{id}",
                defaults: new { id = RouteParameter.Optional }
            );
        }
    }
}
```

Example

EnableCors attribute can be applied to action methods in a controller as follows:

```
public class ResourcesController : ApiController
{
  [EnableCors("http://localhost:55912", // Origin
              null,                     // Request headers
              "GET",                    // HTTP methods
              "bar",                    // Response headers
              SupportsCredentials=true  // Allow credentials
  )]
  public HttpResponseMessage Get(int id)
  {
    var resp = Request.CreateResponse(HttpStatusCode.NoContent);
    resp.Headers.Add("bar", "a bar value");
    return resp;
  }
  [EnableCors("http://localhost:55912",       // Origin
              "Accept, Origin, Content-Type", // Request headers
              "PUT",                          // HTTP methods
              PreflightMaxAge=600             // Preflight cache duration
  )]
  public HttpResponseMessage Put(Resource data)
  {
    return Request.CreateResponse(HttpStatusCode.OK, data);
  }
  [EnableCors("http://localhost:55912",       // Origin
              "Accept, Origin, Content-Type", // Request headers
              "POST",                         // HTTP methods
              PreflightMaxAge=600             // Preflight cache duration
  )]
  public HttpResponseMessage Post(Resource data)
  {
    return Request.CreateResponse(HttpStatusCode.OK, data);
  }
}
```

Please note that it is critical to ensure that the list of origins in EnableCors attribute is set to a finite and trusted set of origins. Failing to configure this inappropriately (e.g., setting the value as '*') will allow malicious sites to trigger cross origin requests to the API without any restrictions, >thereby making the API vulnerable to CSRF attacks. EnableCors can be decorated at controller level.

Example

To disable CORS on a particular method in a class, the DisableCors attribute can be used as shown below:

```
[EnableCors("https://example.com", "Accept, Origin, Content-Type", "POST")]
public class ResourcesController : ApiController
{
  public HttpResponseMessage Put(Resource data)
  {
    return Request.CreateResponse(HttpStatusCode.OK, data);
  }
  public HttpResponseMessage Post(Resource data)
  {
    return Request.CreateResponse(HttpStatusCode.OK, data);
  }
  // CORS not allowed because of the [DisableCors] attribute
  [DisableCors]
  public HttpResponseMessage Delete(int id)
  {
    return Request.CreateResponse(HttpStatusCode.NoContent);
  }
}
```

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	MVC 6
Attributes	N/A
References	Enabling Cross-Origin Requests (CORS) in ASP.NET Core 1.0
Steps	In ASP.NET Core 1.0, CORS can be enabled either using middleware or using MVC. When using MVC to enable CORS the same CORS services are used, but the CORS middleware is not.

Approach-1 Enabling CORS with middleware: To enable CORS for the entire application add the CORS middleware to the request pipeline using the UseCors extension method. A cross-origin policy can be specified when adding the CORS middleware using the CorsPolicyBuilder class. There are two ways to do this:

Example

The first is to call UseCors with a lambda. The lambda takes a CorsPolicyBuilder object:

```
public void Configure(IApplicationBuilder app)
{
    app.UseCors(builder =>
        builder.WithOrigins("https://example.com")
        .WithMethods("GET", "POST", "HEAD")
        .WithHeaders("accept", "content-type", "origin", "x-custom-header"));
}
```

Example

The second is to define one or more named CORS policies, and then select the policy by name at run time.

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddCors(options =>
    {
        options.AddPolicy("AllowSpecificOrigin",
            builder => builder.WithOrigins("https://example.com"));
    });
}
public void Configure(IApplicationBuilder app)
{
    app.UseCors("AllowSpecificOrigin");
    app.Run(async (context) =>
    {
        await context.Response.WriteAsync("Hello World!");
    });
}
```

Approach-2 Enabling CORS in MVC: Developers can alternatively use MVC to apply specific CORS per action, per controller, or globally for all controllers.

Example

Per action: To specify a CORS policy for a specific action add the [EnableCors] attribute to the action. Specify the policy name.

```
public class HomeController : Controller
{
    [EnableCors("AllowSpecificOrigin")]
    public IActionResult Index()
    {
        return View();
    }
```

Example

Per controller:

```
[EnableCors("AllowSpecificOrigin")]
public class HomeController : Controller
{
```

Example

Globally:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvc();
    services.Configure<MvcOptions>(options =>
    {
        options.Filters.Add(new CorsAuthorizationFilterFactory("AllowSpecificOrigin"));
    });
}
```

Please note that it is critical to ensure that the list of origins in EnableCors attribute is set to a finite and trusted set of origins. Failing to configure this inappropriately (e.g., setting the value as '*') will allow malicious sites to trigger cross origin requests to the API without any restrictions, >thereby making the API vulnerable to CSRF attacks.

Example

To disable CORS for a controller or action, use the [DisableCors] attribute.

```
[DisableCors]
    public IActionResult About()
    {
        return View();
    }
```

Encrypt sections of Web API's configuration files that contain sensitive data

TITLE	DETAILS
Component	Web API
SDL Phase	Deployment

TITLE	DETAILS
Applicable Technologies	Generic
Attributes	N/A
References	How To: Encrypt Configuration Sections in ASP.NET 2.0 Using DPAPI, Specifying a Protected Configuration Provider, Using Azure Key Vault to protect application secrets
Steps	Configuration files such as the Web.config, appsettings.json are often used to hold sensitive information, including user names, passwords, database connection strings, and encryption keys. If you do not protect this information, your application is vulnerable to attackers or malicious users obtaining sensitive information such as account user names and passwords, database names and server names. Based on the deployment type (azure/on-prem), encrypt the sensitive sections of config files using DPAPI or services like Azure Key Vault.

Ensure that all admin interfaces are secured with strong credentials

TITLE	DETAILS
Component	IoT Device
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Any administrative interfaces that the device or field gateway exposes should be secured using strong credentials. Also, any other exposed interfaces like WiFi, SSH, File shares, FTP should be secured with strong credentials. Default weak passwords should not be used.

Ensure that unknown code cannot execute on devices

TITLE	DETAILS
Component	IoT Device
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A

TITLE	DETAILS
References	Enabling Secure Boot and bit-locker Device Encryption on Windows 10 IoT Core
Steps	UEFI Secure Boot restricts the system to only allow execution of binaries signed by a specified authority. This feature prevents unknown code from being executed on the platform and potentially weakening the security posture of it. Enable UEFI Secure Boot and restrict the list of certificate authorities that are trusted for signing code. Sign all code that is deployed on the device using one of the trusted authorities.

Encrypt OS and additional partitions of IoT Device with bit-locker

TITLE	DETAILS
Component	IoT Device
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Windows 10 IoT Core implements a lightweight version of bit-locker Device Encryption, which has a strong dependency on the presence of a TPM on the platform, including the necessary preOS protocol in UEFI that conducts the necessary measurements. These preOS measurements ensure that the OS later has a definitive record of how the OS was launched.Encrypt OS partitions using bit-locker and any additional partitions also in case they store any sensitive data.

Ensure that only the minimum services/features are enabled on devices

TITLE	DETAILS
Component	IoT Device
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	Do not enable or turn off any features or services in the OS that is not required for the functioning of the solution. For e.g. if the device does not require a UI to be deployed, install Windows IoT Core in headless mode.

Encrypt OS and additional partitions of IoT Field Gateway with bit-locker

TITLE	DETAILS
Component	IoT Field Gateway
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Windows 10 IoT Core implements a lightweight version of bit-locker Device Encryption, which has a strong dependency on the presence of a TPM on the platform, including the necessary preOS protocol in UEFI that conducts the necessary measurements. These preOS measurements ensure that the OS later has a definitive record of how the OS was launched.Encrypt OS partitions using bit-locker and any additional partitions also in case they store any sensitive data.

Ensure that the default login credentials of the field gateway are changed during installation

TITLE	DETAILS
Component	IoT Field Gateway
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that the default login credentials of the field gateway are changed during installation

Ensure that the Cloud Gateway implements a process to keep the connected devices firmware up to date

TITLE	DETAILS
Component	IoT Cloud Gateway
SDL Phase	Build
Applicable Technologies	Generic
Attributes	Gateway choice - Azure IoT Hub
References	IoT Hub Device Management Overview, How to update Device Firmware
Steps	LWM2M is a protocol from the Open Mobile Alliance for IoT Device Management. Azure IoT device management allows to interact with physical devices using device jobs. Ensure that the Cloud Gateway implements a process to routinely keep the device and other configuration data up to date using Azure IoT Hub Device Management.

Ensure that devices have end-point security controls configured as per organizational policies

TITLE	DETAILS
Component	Machine Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that devices have end-point security controls such as bit-locker for disk-level encryption, anti-virus with updated signatures, host based firewall, OS upgrades, group policies etc. are configured as per organizational security policies.

Ensure secure management of Azure storage access keys

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A

TITLE	DETAILS
References	Azure Storage security guide - Managing Your Storage Account Keys
Steps	Key Storage: It is recommended to store the Azure Storage access keys in Azure Key Vault as a secret and have the applications retrieve the key from key vault. This is recommended due to the following reasons: • The application will never have the storage key hardcoded in a configuration file, which removes that avenue of somebody getting access to the keys without specific permission • Access to the keys can be controlled using Azure Active Directory. This means an account owner can grant access to the handful of applications that need to retrieve the keys from Azure Key Vault. Other applications will not be able to access the keys without granting them permission specifically • Key Regeneration: It is recommended to have a process in place to regenerate Azure storage access keys for security reasons. Details on why and how to plan for key regeneration are documented in the Azure Storage Security Guide reference article

Ensure that only trusted origins are allowed if CORS is enabled on Azure storage

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	CORS Support for the Azure Storage Services
Steps	Azure Storage allows you to enable CORS – Cross Origin Resource Sharing. For each storage account, you can specify domains that can access the resources in that storage account. By default, CORS is disabled on all services. You can enable CORS by using the REST API or the storage client library to call one of the methods to set the service policies.

Enable WCF's service throttling feature

TITLE	DETAILS
Component	WCF

TITLE	DETAILS
SDL Phase	Build
Applicable Technologies	.NET Framework 3
Attributes	N/A
References	MSDN, Fortify Kingdom
Steps	Not placing a limit on the use of system resources could result in resource exhaustion and ultimately a denial of service. • **EXPLANATION:** Windows Communication Foundation (WCF) offers the ability to throttle service requests. Allowing too many client requests can flood a system and exhaust its resources. On the other hand, allowing only a small number of requests to a service can prevent legitimate users from using the service. Each service should be individually tuned to and configured to allow the appropriate amount of resources. • **RECOMMENDATIONS** Enable WCF's service throttling feature and set limits appropriate for your application.

Example

The following is an example configuration with throttling enabled:

```
<system.serviceModel>
  <behaviors>
    <serviceBehaviors>
    <behavior name="Throttled">
    <serviceThrottling maxConcurrentCalls="[YOUR SERVICE VALUE]" maxConcurrentSessions="[YOUR SERVICE
VALUE]" maxConcurrentInstances="[YOUR SERVICE VALUE]" />
  ...
</system.serviceModel>
```

WCF-Information disclosure through metadata

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	.NET Framework 3
Attributes	N/A
References	MSDN, Fortify Kingdom

TITLE	DETAILS
Steps	Metadata can help attackers learn about the system and plan a form of attack. WCF services can be configured to expose metadata. Metadata gives detailed service description information and should not be broadcast in production environments. The `HttpGetEnabled` / `HttpsGetEnabled` properties of the ServiceMetaData class defines whether a service will expose the metadata

Example

The code below instructs WCF to broadcast a service's metadata

```
ServiceMetadataBehavior smb = new ServiceMetadataBehavior();
smb.HttpGetEnabled = true;
smb.HttpGetUrl = new Uri(EndPointAddress);
Host.Description.Behaviors.Add(smb);
```

Do not broadcast service metadata in a production environment. Set the HttpGetEnabled / HttpsGetEnabled properties of the ServiceMetaData class to false.

Example

The code below instructs WCF to not broadcast a service's metadata.

```
ServiceMetadataBehavior smb = new ServiceMetadataBehavior();
smb.HttpGetEnabled = false;
smb.HttpGetUrl = new Uri(EndPointAddress);
Host.Description.Behaviors.Add(smb);
```

Security Frame: Cryptography | Mitigation

9/30/2021 • 14 minutes to read • Edit Online

PRODUCT/SERVICE	ARTICLE
Web Application	• Use only approved symmetric block ciphers and key lengths • Use approved block cipher modes and initialization vectors for symmetric ciphers • Use approved asymmetric algorithms, key lengths, and padding • Use approved random number generators • Do not use symmetric stream ciphers • Use approved MAC/HMAC/keyed hash algorithms • Use only approved cryptographic hash functions
Database	• Use strong encryption algorithms to encrypt data in the database • SSIS packages should be encrypted and digitally signed • Add digital signature to critical database securables • Use SQL server EKM to protect encryption keys • Use AlwaysEncrypted feature if encryption keys should not be revealed to Database engine
IoT Device	• Store Cryptographic Keys securely on IoT Device
IoT Cloud Gateway	• Generate a random symmetric key of sufficient length for authentication to IoT Hub
Dynamics CRM Mobile Client	• Ensure a device management policy is in place that requires a use PIN and allows remote wiping
Dynamics CRM Outlook Client	• Ensure a device management policy is in place that requires a PIN/password/auto lock and encrypts all data (e.g. BitLocker)
Identity Server	• Ensure that signing keys are rolled over when using Identity Server • Ensure that cryptographically strong client ID, client secret are used in Identity Server

Use only approved symmetric block ciphers and key lengths

TITLE	DETAILS
Component	Web Application

TITLE	DETAILS
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Products must use only those symmetric block ciphers and associated key lengths which have been explicitly approved by the Crypto Advisor in your organization. Approved symmetric algorithms at Microsoft include the following block ciphers: • For new code AES-128, AES-192, and AES-256 are acceptable • For backward compatibility with existing code, three-key 3DES is acceptable • For products using symmetric block ciphers: ◦ Advanced Encryption Standard (AES) is required for new code ◦ Three-key triple Data Encryption Standard (3DES) is permissible in existing code for backward compatibility ◦ All other block ciphers, including RC2, DES, 2 Key 3DES, DESX, and Skipjack, may only be used for decrypting old data, and must be replaced if used for encryption • For symmetric block encryption algorithms, a minimum key length of 128 bits is required. The only block encryption algorithm recommended for new code is AES (AES-128, AES-192 and AES-256 are all acceptable) • Three-key 3DES is currently acceptable if already in use in existing code; transition to AES is recommended. DES, DESX, RC2, and Skipjack are no longer considered secure. These algorithms may only be used for decrypting existing data for the sake of backward-compatibility, and data should be re-encrypted using a recommended block cipher Please note that all symmetric block ciphers must be used with an approved cipher mode, which requires use of an appropriate initialization vector (IV). An appropriate IV, is typically a random number and never a constant value The use of legacy or otherwise unapproved crypto algorithms and smaller key lengths for reading existing data (as opposed to writing new data) may be permitted after your organization's Crypto Board review. However, you must file for an exception against this requirement. Additionally, in enterprise deployments, products should consider warning administrators when weak crypto is used to read data. Such warnings should be explanatory and actionable. In some cases, it may be appropriate to have Group Policy control the use of weak crypto Allowed .NET algorithms for managed crypto agility (in order of preference)

TITLE	DETAILS
	• AesCng (FIPS compliant) • AuthenticatedAesCng (FIPS compliant) • AESCryptoServiceProvider (FIPS compliant) • AESManaged (non-FIPS-compliant) Please note that none of these algorithms can be specified via the `SymmetricAlgorithm.Create` or `CryptoConfig.CreateFromName` methods without making changes to the machine.config file. Also, note that AES in versions of .NET prior to .NET 3.5 is named `RijndaelManaged`, and `AesCng` and `AuthenticatedAesCng` are >available through CodePlex and require CNG in the underlying OS

Use approved block cipher modes and initialization vectors for symmetric ciphers

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	All symmetric block ciphers must be used with an approved symmetric cipher mode. The only approved modes are CBC and CTS. In particular, the electronic code book (ECB) mode of operation should be avoided; use of ECB requires your organization's Crypto Board review. All usage of OFB, CFB, CTR, CCM, and GCM or any other encryption mode must be reviewed by your organization's Crypto Board. Reusing the same initialization vector (IV) with block ciphers in "streaming ciphers modes," such as CTR, may cause encrypted data to be revealed. All symmetric block ciphers must also be used with an appropriate initialization vector (IV). An appropriate IV is a cryptographically strong, random number and never a constant value.

Use approved asymmetric algorithms, key lengths, and padding

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A

TITLE	DETAILS
References	N/A
Steps	The use of banned cryptographic algorithms introduces significant risk to product security and must be avoided. Products must use only those cryptographic algorithms and associated key lengths and padding that have been explicitly approved by your organization's Crypto Board. • **RSA**- may be used for encryption, key exchange and signature. RSA encryption must use only the OAEP or RSA-KEM padding modes. Existing code may use PKCS #1 v1.5 padding mode for compatibility only. Use of null padding is explicitly banned. Keys >= 2048 bits is required for new code. Existing code may support keys < 2048 bits only for backwards compatibility after a review by your organization's Crypto Board. Keys < 1024 bits may only be used for decrypting/verifying old data, and must be replaced if used for encryption or signing operations • **ECDSA**- may be used for signature only. ECDSA with >=256-bit keys is required for new code. ECDSA-based signatures must use one of the three NIST approved curves (P-256, P-384, or P521). Curves that have been thoroughly analyzed may be used only after a review with your organization's Crypto Board. • **ECDH**- may be used for key exchange only. ECDH with >=256-bit keys is required for new code. ECDH-based key exchange must use one of the three NIST approved curves (P-256, P-384, or P521). Curves that have been thoroughly analyzed may be used only after a review with your organization's Crypto Board. • **DSA**- may be acceptable after review and approval from your organization's Crypto Board. Contact your security advisor to schedule your organization's Crypto Board review. If your use of DSA is approved, note that you will need to prohibit use of keys less than 2048 bits in length. CNG supports 2048-bit and greater key lengths as of Windows 8. • **Diffie-Hellman**- may be used for session key management only. Key length >= 2048 bits is required for new code. Existing code may support key lengths < 2048 bits only for backwards compatibility after a review by your organization's Crypto Board. Keys < 1024 bits may not be used.

Use approved random number generators

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic

TITLE	DETAILS
Attributes	N/A
References	N/A
Steps	Products must use approved random number generators. Pseudorandom functions such as the C runtime function rand, the .NET Framework class System.Random, or system functions such as GetTickCount must, therefore, never be used in such code. Use of the dual elliptic curve random number generator (DUAL_EC_DRBG) algorithm is prohibited • **CNG**- BCryptGenRandom(use of the BCRYPT_USE_SYSTEM_PREFERRED_RNG flag recommended unless the caller might run at any IRQL greater than 0 [that is, PASSIVE_LEVEL]) • **CAPI**- cryptGenRandom • **Win32/64**- RtlGenRandom (new implementations should use BCryptGenRandom or CryptGenRandom) * rand_s * SystemPrng (for kernel mode) • **.NET**- RNGCryptoServiceProvider or RNGCng • **Windows Store Apps**- Windows.Security.Cryptography.CryptographicBuffer. GenerateRandom or .GenerateRandomNumber • **Apple OS X (10.7+)/iOS(2.0+)**- int SecRandomCopyBytes (SecRandomRef random, size_t count, uint8_t *bytes) • **Apple OS X (<10.7)**- Use /dev/random to retrieve random numbers • **Java(including Google Android Java code)**- java.security.SecureRandom class. Note that for Android 4.3 (Jelly Bean), developers must follow the Android recommended workaround and update their applications to explicitly initialize the PRNG with entropy from /dev/urandom or /dev/random

Do not use symmetric stream ciphers

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Symmetric stream ciphers, such as RC4, must not be used. Instead of symmetric stream ciphers, products should use a block cipher, specifically AES with a key length of at least 128 bits.

Use approved MAC/HMAC/keyed hash algorithms

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Products must use only approved message authentication code (MAC) or hash-based message authentication code (HMAC) algorithms. A message authentication code (MAC) is a piece of information attached to a message that allows its recipient to verify both the authenticity of the sender and the integrity of the message using a secret key. The use of either a hash-based MAC (HMAC) or block-cipher-based MAC is permissible as long as all underlying hash or symmetric encryption algorithms are also approved for use; currently this includes the HMAC-SHA2 functions (HMAC-SHA256, HMAC-SHA384 and HMAC-SHA512) and the CMAC/OMAC1 and OMAC2 block cipher-based MACs (these are based on AES). Use of HMAC-SHA1 may be permissible for platform compatibility, but you will be required to file an exception to this procedure and undergo your organization's Crypto review. Truncation of HMACs to less than 128 bits is not permitted. Using customer methods to hash a key and data is not approved, and must undergo your organization's Crypto Board review prior to use.

Use only approved cryptographic hash functions

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	Products must use the SHA-2 family of hash algorithms (SHA256, SHA384, and SHA512). If a shorter hash is needed, such as a 128-bit output length in order to fit a data structure designed with the shorter MD5 hash in mind, product teams may truncate one of the SHA2 hashes (typically SHA256). Note that SHA384 is a truncated version of SHA512. Truncation of cryptographic hashes for security purposes to less than 128 bits is not permitted. New code must not use the MD2, MD4, MD5, SHA-0, SHA-1, or RIPEMD hash algorithms. Hash collisions are computationally feasible for these algorithms, which effectively breaks them. Allowed .NET hash algorithms for managed crypto agility (in order of preference): • SHA512Cng (FIPS compliant) • SHA384Cng (FIPS compliant) • SHA256Cng (FIPS compliant) • SHA512Managed (non-FIPS-compliant) (use SHA512 as algorithm name in calls to HashAlgorithm.Create or CryptoConfig.CreateFromName) • SHA384Managed (non-FIPS-compliant) (use SHA384 as algorithm name in calls to HashAlgorithm.Create or CryptoConfig.CreateFromName) • SHA256Managed (non-FIPS-compliant) (use SHA256 as algorithm name in calls to HashAlgorithm.Create or CryptoConfig.CreateFromName) • SHA512CryptoServiceProvider (FIPS compliant) • SHA256CryptoServiceProvider (FIPS compliant) • SHA384CryptoServiceProvider (FIPS compliant)

Use strong encryption algorithms to encrypt data in the database

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Choosing an encryption algorithm
Steps	Encryption algorithms define data transformations that cannot be easily reversed by unauthorized users. SQL Server allows administrators and developers to choose from among several algorithms, including DES, Triple DES, TRIPLE_DES_3KEY, RC2, RC4, 128-bit RC4, DESX, 128-bit AES, 192-bit AES, and 256-bit AES

SSIS packages should be encrypted and digitally signed

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Identify the Source of Packages with Digital Signatures, Threat and Vulnerability Mitigation (Integration Services)
Steps	The source of a package is the individual or organization that created the package. Running a package from an unknown or untrusted source might be risky. To prevent unauthorized tampering of SSIS packages, digital signatures should be used. Also, to ensure the confidentiality of the packages during storage/transit, SSIS packages have to be encrypted

Add digital signature to critical database securables

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	ADD SIGNATURE (Transact-SQL)
Steps	In cases where the integrity of a critical database securable has to be verified, digital signatures should be used. Database securables such as a stored procedure, function, assembly, or trigger can be digitally signed. Below is an example of when this can be useful: Let us say an ISV (Independent Software Vendor) has provided support to a software delivered to one of their customers. Before providing support, the ISV would want to ensure that a database securable in the software was not tampered either by mistake or by a malicious attempt. If the securable is digitally signed, the ISV can verify its digital signature and validate its integrity.

Use SQL server EKM to protect encryption keys

TITLE	DETAILS
Component	Database
SDL Phase	Build

TITLE	DETAILS
Applicable Technologies	Generic
Attributes	N/A
References	SQL Server Extensible Key Management (EKM), Extensible Key Management Using Azure Key Vault (SQL Server)
Steps	SQL Server Extensible Key Management enables the encryption keys that protect the database files to be stored in an off-box device such as a smartcard, USB device, or EKM/HSM module. This also enables data protection from database administrators (except members of the sysadmin group). Data can be encrypted by using encryption keys that only the database user has access to on the external EKM/HSM module.

Use AlwaysEncrypted feature if encryption keys should not be revealed to Database engine

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	SQL Azure, OnPrem
Attributes	SQL Version - V12, MsSQL2016
References	Always Encrypted (Database Engine)
Steps	Always Encrypted is a feature designed to protect sensitive data, such as credit card numbers or national identification numbers (e.g. U.S. social security numbers), stored in Azure SQL Database or SQL Server databases. Always Encrypted allows clients to encrypt sensitive data inside client applications and never reveal the encryption keys to the Database Engine (SQL Database or SQL Server). As a result, Always Encrypted provides a separation between those who own the data (and can view it) and those who manage the data (but should have no access)

Store Cryptographic Keys securely on IoT Device

TITLE	DETAILS
Component	IoT Device
SDL Phase	Build
Applicable Technologies	Generic

TITLE	DETAILS
Attributes	Device OS - Windows IoT Core, Device Connectivity - Azure IoT device SDKs
References	TPM on Windows IoT Core, Set up TPM on Windows IoT Core, Azure IoT Device SDK TPM
Steps	Symmetric or Certificate Private keys securely in a hardware protected storage like TPM or Smart Card chips. Windows 10 IoT Core supports the user of a TPM and there are several compatible TPMs that can be used: Discrete TPM (dTPM). It is recommended to use a Firmware or Discrete TPM. A Software TPM should only be used for development and testing purposes. Once a TPM is available and the keys are provisioned in it, the code that generates the token should be written without hard coding any sensitive information in it.

Example

```
TpmDevice myDevice = new TpmDevice(0);
// Use logical device 0 on the TPM
string hubUri = myDevice.GetHostName();
string deviceId = myDevice.GetDeviceId();
string sasToken = myDevice.GetSASToken();

var deviceClient = DeviceClient.Create( hubUri, AuthenticationMethodFactory.
CreateAuthenticationWithToken(deviceId, sasToken), TransportType.Amqp);
```

As can be seen, the device primary key is not present in the code. Instead, it is stored in the TPM at slot 0. TPM device generates a short-lived SAS token that is then used to connect to the IoT Hub.

Generate a random symmetric key of sufficient length for authentication to IoT Hub

TITLE	DETAILS
Component	IoT Cloud Gateway
SDL Phase	Build
Applicable Technologies	Generic
Attributes	Gateway choice - Azure IoT Hub
References	N/A
Steps	IoT Hub contains a device Identity Registry and while provisioning a device, automatically generates a random Symmetric key. It is recommended to use this feature of the Azure IoT Hub Identity Registry to generate the key used for authentication. IoT Hub also allows for a key to be specified while creating the device. If a key is generated outside of IoT Hub during device provisioning, it is recommended to create a random symmetric key or at least 256 bits.

Ensure a device management policy is in place that requires a use PIN and allows remote wiping

TITLE	DETAILS
Component	Dynamics CRM Mobile Client
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure a device management policy is in place that requires a use PIN and allows remote wiping

Ensure a device management policy is in place that requires a PIN/password/auto lock and encrypts all data (e.g. BitLocker)

TITLE	DETAILS
Component	Dynamics CRM Outlook Client
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure a device management policy is in place that requires a PIN/password/auto lock and encrypts all data (e.g. BitLocker)

Ensure that signing keys are rolled over when using Identity Server

TITLE	DETAILS
Component	Identity Server
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	Identity Server - Keys, Signatures and Cryptography

TITLE	DETAILS
Steps	Ensure that signing keys are rolled over when using Identity Server. The link in the references section explains how this should be planned without causing outages to applications relying on Identity Server.

Ensure that cryptographically strong client ID, client secret are used in Identity Server

TITLE	DETAILS
Component	Identity Server
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that cryptographically strong client ID, client secret are used in Identity Server. The following guidelines should be used while generating a client ID and secret: • Generate a random GUID as the client ID • Generate a cryptographically random 256-bit key as the secret

Security Frame: Exception Management | Mitigations

3/5/2021 • 7 minutes to read • Edit Online

PRODUCT/SERVICE	ARTICLE
WCF	• WCF- Do not include serviceDebug node in configuration file • WCF- Do not include serviceMetadata node in configuration file
Web API	• Ensure that proper exception handling is done in ASP.NET Web API
Web Application	• Do not expose security details in error messages • Implement Default error handling page • Set Deployment Method to Retail in IIS • Exceptions should fail safely

WCF- Do not include serviceDebug node in configuration file

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	Generic, NET Framework 3
Attributes	N/A
References	MSDN, Fortify Kingdom
Steps	Windows Communication Framework (WCF) services can be configured to expose debugging information. Debug information should not be used in production environments. The `<serviceDebug>` tag defines whether the debug information feature is enabled for a WCF service. If the attribute includeExceptionDetailInFaults is set to true, exception information from the application will be returned to clients. Attackers can leverage the additional information they gain from debugging output to mount attacks targeted on the framework, database, or other resources used by the application.

Example

The following configuration file includes the `<serviceDebug>` tag:

```
<configuration>
<system.serviceModel>
<behaviors>
<serviceBehaviors>
<behavior name=""MyServiceBehavior"">
<serviceDebug includeExceptionDetailInFaults=""True"" httpHelpPageEnabled=""True""/>
...
```

Disable debugging information in the service. This can be accomplished by removing the `<serviceDebug>` tag from your application's configuration file.

WCF- Do not include serviceMetadata node in configuration file

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	Generic
Attributes	Generic, NET Framework 3
References	MSDN, Fortify Kingdom
Steps	Publicly exposing information about a service can provide attackers with valuable insight into how they might exploit the service. The `<serviceMetadata>` tag enables the metadata publishing feature. Service metadata could contain sensitive information that should not be publicly accessible. At a minimum, only allow trusted users to access the metadata and ensure that unnecessary information is not exposed. Better yet, entirely disable the ability to publish metadata. A safe WCF configuration will not contain the `<serviceMetadata>` tag.

Ensure that proper exception handling is done in ASP.NET Web API

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	MVC 5, MVC 6
Attributes	N/A
References	Exception Handling in ASP.NET Web API, Model Validation in ASP.NET Web API
Steps	By default, most uncaught exceptions in ASP.NET Web API are translated into an HTTP response with status code `500, Internal Server Error`

Example

To control the status code returned by the API, `HttpResponseException` can be used as shown below:

```
public Product GetProduct(int id)
{
    Product item = repository.Get(id);
    if (item == null)
    {
        throw new HttpResponseException(HttpStatusCode.NotFound);
    }
    return item;
}
```

Example

For further control on the exception response, the `HttpResponseMessage` class can be used as shown below:

```
public Product GetProduct(int id)
{
    Product item = repository.Get(id);
    if (item == null)
    {
        var resp = new HttpResponseMessage(HttpStatusCode.NotFound)
        {
            Content = new StringContent(string.Format("No product with ID = {0}", id)),
            ReasonPhrase = "Product ID Not Found"
        }
        throw new HttpResponseException(resp);
    }
    return item;
}
```

To catch unhandled exceptions that are not of the type `HttpResponseException`, Exception Filters can be used. Exception filters implement the `System.Web.Http.Filters.IExceptionFilter` interface. The simplest way to write an exception filter is to derive from the `System.Web.Http.Filters.ExceptionFilterAttribute` class and override the OnException method.

Example

Here is a filter that converts `NotImplementedException` exceptions into HTTP status code `501, Not Implemented`:

```
namespace ProductStore.Filters
{
    using System;
    using System.Net;
    using System.Net.Http;
    using System.Web.Http.Filters;

    public class NotImplExceptionFilterAttribute : ExceptionFilterAttribute
    {
        public override void OnException(HttpActionExecutedContext context)
        {
            if (context.Exception is NotImplementedException)
            {
                context.Response = new HttpResponseMessage(HttpStatusCode.NotImplemented);
            }
        }
    }
}
```

There are several ways to register a Web API exception filter:

- By action
- By controller
- Globally

Example

To apply the filter to a specific action, add the filter as an attribute to the action:

```
public class ProductsController : ApiController
{
    [NotImplExceptionFilter]
    public Contact GetContact(int id)
    {
        throw new NotImplementedException("This method is not implemented");
    }
}
```

Example

To apply the filter to all of the actions on a `controller`, add the filter as an attribute to the `controller` class:

```
[NotImplExceptionFilter]
public class ProductsController : ApiController
{
    // ...
}
```

Example

To apply the filter globally to all Web API controllers, add an instance of the filter to the `GlobalConfiguration.Configuration.Filters` collection. Exception filters in this collection apply to any Web API controller action.

```
GlobalConfiguration.Configuration.Filters.Add(
    new ProductStore.NotImplExceptionFilterAttribute());
```

Example

For model validation, the model state can be passed to CreateErrorResponse method as shown below:

```
public HttpResponseMessage PostProduct(Product item)
{
    if (!ModelState.IsValid)
    {
        return Request.CreateErrorResponse(HttpStatusCode.BadRequest, ModelState);
    }
    // Implementation not shown...
}
```

Check the links in the references section for additional details about exceptional handling and model validation in ASP.NET Web API

Do not expose security details in error messages

TITLE	DETAILS
Component	Web Application

TITLE	DETAILS
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Generic error messages are provided directly to the user without including sensitive application data. Examples of sensitive data include: • Server names • Connection strings • Usernames • Passwords • SQL procedures • Details of dynamic SQL failures • Stack trace and lines of code • Variables stored in memory • Drive and folder locations • Application install points • Host configuration settings • Other internal application details Trapping all errors within an application and providing generic error messages, as well as enabling custom errors within IIS will help prevent information disclosure. SQL Server database and .NET Exception handling, among other error handling architectures, are especially verbose and extremely useful to a malicious user profiling your application. Do not directly display the contents of a class derived from the .NET Exception class, and ensure that you have proper exception handling so that an unexpected exception isn't inadvertently raised directly to the user. • Provide generic error messages directly to the user that abstract away specific details found directly in the exception/error message • Do not display the contents of a .NET exception class directly to the user • Trap all error messages and if appropriate inform the user via a generic error message sent to the application client • Do not expose the contents of the Exception class directly to the user, especially the return value from `.ToString()`, or the values of the Message or StackTrace properties. Securely log this information and display a more innocuous message to the user

Implement Default error handling page

TITLE	DETAILS
Component	Web Application

TITLE	DETAILS
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Edit ASP.NET Error Pages Settings Dialog Box
Steps	When an ASP.NET application fails and causes an HTTP/1.x 500 Internal Server Error, or a feature configuration (such as Request Filtering) prevents a page from being displayed, an error message will be generated. Administrators can choose whether or not the application should display a friendly message to the client, detailed error message to the client, or detailed error message to localhost only. The `<customErrors>` tag in the web.config has three modes: • **On:** Specifies that custom errors are enabled. If no defaultRedirect attribute is specified, users see a generic error. The custom errors are shown to the remote clients and to the local host • **Off:** Specifies that custom errors are disabled. The detailed ASP.NET errors are shown to the remote clients and to the local host • **RemoteOnly:** Specifies that custom errors are shown only to the remote clients, and that ASP.NET errors are shown to the local host. This is the default value Open the `web.config` file for the application/site and ensure that the tag has either `<customErrors mode="RemoteOnly" />` or `<customErrors mode="On" />` defined.

Set Deployment Method to Retail in IIS

TITLE	DETAILS
Component	Web Application
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	deployment Element (ASP.NET Settings Schema)

TITLE	DETAILS
Steps	The `<deployment retail>` switch is intended for use by production IIS servers. This switch is used to help applications run with the best possible performance and least possible security information leakages by disabling the application's ability to generate trace output on a page, disabling the ability to display detailed error messages to end users, and disabling the debug switch. Often times, switches and options that are developer-focused, such as failed request tracing and debugging, are enabled during active development. It is recommended that the deployment method on any production server be set to retail. open the machine.config file and ensure that `<deployment retail="true" />` remains set to true.

Exceptions should fail safely

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Fail securely
Steps	Application should fail safely. Any method that returns a Boolean value, based on which certain decision is made, should have exception block carefully created. There are lot of logical errors due to which security issues creep in, when the exception block is written carelessly.

Example

```
        public static bool ValidateDomain(string pathToValidate, Uri currentUrl)
        {
            try
            {
                if (!string.IsNullOrWhiteSpace(pathToValidate))
                {
                    var domain = RetrieveDomain(currentUrl);
                    var replyPath = new Uri(pathToValidate);
                    var replyDomain = RetrieveDomain(replyPath);

                    if (string.Compare(domain, replyDomain, StringComparison.OrdinalIgnoreCase) != 0)
                    {
                        //// Adding additional check to enable CMS urls if they are not hosted on same
domain.
                        if (!string.IsNullOrWhiteSpace(Utilities.CmsBase))
                        {
                            var cmsDomain = RetrieveDomain(new Uri(Utilities.Base.Trim()));
                            if (string.Compare(cmDomain, replyDomain, StringComparison.OrdinalIgnoreCase) !=
0)
                            {
                                return false;
                            }
                            else
                            {
                                return true;
                            }
                        }

                        return false;
                    }
                }

                return true;
            }
            catch (UriFormatException ex)
            {
                LogHelper.LogException("Utilities:ValidateDomain", ex);
                return true;
            }
        }
```

The above method will always return True, if some exception happens. If the end user provides a malformed URL, that the browser respects, but the `Uri()` constructor doesn't, this will throw an exception, and the victim will be taken to the valid but malformed URL.

Security Frame: Input Validation | Mitigations

3/5/2021 • 31 minutes to read • Edit Online

PRODUCT/SERVICE	ARTICLE
Web Application	• Disable XSLT scripting for all transforms using untrusted style sheets • Ensure that each page that could contain user controllable content opts out of automatic MIME sniffing • Harden or Disable XML Entity Resolution • Applications utilizing http.sys perform URL canonicalization verification • Ensure appropriate controls are in place when accepting files from users • Ensure that type-safe parameters are used in Web Application for data access • Use separate model binding classes or binding filter lists to prevent MVC mass assignment vulnerability • Encode untrusted web output prior to rendering • Perform input validation and filtering on all string type Model properties • Sanitization should be applied on form fields that accept all characters, e.g, rich text editor • Do not assign DOM elements to sinks that do not have inbuilt encoding • Validate all redirects within the application are closed or done safely • Implement input validation on all string type parameters accepted by Controller methods • Set upper limit timeout for regular expression processing to prevent DoS due to bad regular expressions • Avoid using Html.Raw in Razor views
Database	• Do not use dynamic queries in stored procedures
Web API	• Ensure that model validation is done on Web API methods • Implement input validation on all string type parameters accepted by Web API methods • Ensure that type-safe parameters are used in Web API for data access
Azure Document DB	• Use parameterized SQL queries for Azure Cosmos DB
WCF	• WCF Input validation through Schema binding • WCF- Input validation through Parameter Inspectors

Disable XSLT scripting for all transforms using untrusted style sheets

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	XSLT Security, XsltSettings.EnableScript Property
Steps	XSLT supports scripting inside style sheets using the `<msxml:script>` element. This allows custom functions to be used in an XSLT transformation. The script is executed under the context of the process performing the transform. XSLT script must be disabled when in an untrusted environment to prevent execution of untrusted code. *If using .NET:* XSLT scripting is disabled by default; however, you must ensure that it has not been explicitly enabled through the `XsltSettings.EnableScript` property.

Example

```
XsltSettings settings = new XsltSettings();
settings.EnableScript = true; // WRONG: THIS SHOULD BE SET TO false
```

Example

If you are using MSXML 6.0, XSLT scripting is disabled by default; however, you must ensure that it has not been explicitly enabled through the XML DOM object property AllowXsltScript.

```
doc.setProperty("AllowXsltScript", true); // WRONG: THIS SHOULD BE SET TO false
```

Example

If you are using MSXML 5 or below, XSLT scripting is enabled by default and you must explicitly disable it. Set the XML DOM object property AllowXsltScript to false.

```
doc.setProperty("AllowXsltScript", false); // CORRECT. Setting to false disables XSLT scripting.
```

Ensure that each page that could contain user controllable content opts out of automatic MIME sniffing

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A

TITLE	DETAILS
References	IE8 Security Part V - Comprehensive Protection
Steps	For each page that could contain user controllable content, you must use the HTTP Header `X-Content-Type-Options:nosniff`. To comply with this requirement, you can either set the required header page by page for only those pages that might contain user-controllable content, or you can set it globally for all pages in the application. Each type of file delivered from a web server has an associated MIME type (also called a *content-type*) that describes the nature of the content (that is, image, text, application, etc.) The X-Content-Type-Options header is an HTTP header that allows developers to specify that their content should not be MIME-sniffed. This header is designed to mitigate MIME-Sniffing attacks. Support for this header was added in Internet Explorer 8 (IE8) Only users of Internet Explorer 8 (IE8) will benefit from X-Content-Type-Options. Previous versions of Internet Explorer do not currently respect the X-Content-Type-Options header Internet Explorer 8 (and later) are the only major browsers to implement a MIME-sniffing opt-out feature. If and when other major browsers (Firefox, Safari, Chrome) implement similar features, this recommendation will be updated to include syntax for those browsers as well

Example

To enable the required header globally for all pages in the application, you can do one of the following:

- Add the header in the web.config file if the application is hosted by Internet Information Services (IIS) 7

```
<system.webServer>
  <httpProtocol>
    <customHeaders>
      <add name=""X-Content-Type-Options"" value=""nosniff""/>
    </customHeaders>
  </httpProtocol>
</system.webServer>
```

- Add the header through the global Application_BeginRequest

```
void Application_BeginRequest(object sender, EventArgs e)
{
  this.Response.Headers[""X-Content-Type-Options""] = ""nosniff"";
}
```

- Implement custom HTTP module

```
public class XContentTypeOptionsModule : IHttpModule
  {
    #region IHttpModule Members
    public void Dispose()
    {

    }
    public void Init(HttpApplication context)
    {
      context.PreSendRequestHeaders += newEventHandler(context_PreSendRequestHeaders);
    }
    #endregion
    void context_PreSendRequestHeaders(object sender, EventArgs e)
      {
        HttpApplication application = sender as HttpApplication;
        if (application == null)
          return;
        if (application.Response.Headers[""X-Content-Type-Options ""] != null)
          return;
        application.Response.Headers.Add(""X-Content-Type-Options "", ""nosniff"");
      }
  }
```

- You can enable the required header only for specific pages by adding it to individual responses:

```
this.Response.Headers[""X-Content-Type-Options""] = ""nosniff"";
```

Harden or Disable XML Entity Resolution

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	XML Entity Expansion, XML Denial of Service Attacks and Defenses, MSXML Security Overview, Best Practices for Securing MSXML Code, NSXMLParserDelegate Protocol Reference, Resolving External References

TITLE	DETAILS
Steps	Although it is not widely used, there is a feature of XML that allows the XML parser to expand macro entities with values defined either within the document itself or from external sources. For example, the document might define an entity "companyname" with the value "Microsoft," so that every time the text "&companyname;" appears in the document, it is automatically replaced with the text Microsoft. Or, the document might define an entity "MSFTStock" that references an external web service to fetch the current value of Microsoft stock. Then any time "&MSFTStock;" appears in the document, it is automatically replaced with the current stock price. However, this functionality can be abused to create denial of service (DoS) conditions. An attacker can nest multiple entities to create an exponential expansion XML bomb that consumes all available memory on the system. Alternatively, he can create an external reference that streams back an infinite amount of data or that simply hangs the thread. As a result, all teams must disable internal and/or external XML entity resolution entirely if their application does not use it, or manually limit the amount of memory and time that the application can consume for entity resolution if this functionality is absolutely necessary. If entity resolution is not required by your application, then disable it.

Example

For .NET Framework code, you can use the following approaches:

```
XmlTextReader reader = new XmlTextReader(stream);
reader.ProhibitDtd = true;

XmlReaderSettings settings = new XmlReaderSettings();
settings.ProhibitDtd = true;
XmlReader reader = XmlReader.Create(stream, settings);

// for .NET 4
XmlReaderSettings settings = new XmlReaderSettings();
settings.DtdProcessing = DtdProcessing.Prohibit;
XmlReader reader = XmlReader.Create(stream, settings);
```

Note that the default value of `ProhibitDtd` in `XmlReaderSettings` is true, but in `XmlTextReader` it is false. If you are using XmlReaderSettings, you do not need to set ProhibitDtd to true explicitly, but it is recommended for safety sake that you do. Also note that the XmlDocument class allows entity resolution by default.

Example

To disable entity resolution for XmlDocuments, use the `XmlDocument.Load(XmlReader)` overload of the Load method and set the appropriate properties in the XmlReader argument to disable resolution, as illustrated in the following code:

```
XmlReaderSettings settings = new XmlReaderSettings();
settings.ProhibitDtd = true;
XmlReader reader = XmlReader.Create(stream, settings);
XmlDocument doc = new XmlDocument();
doc.Load(reader);
```

Example

If disabling entity resolution is not possible for your application, set the XmlReaderSettings.MaxCharactersFromEntities property to a reasonable value according to your application's needs. This will limit the impact of potential exponential expansion DoS attacks. The following code provides an example of this approach:

```
XmlReaderSettings settings = new XmlReaderSettings();
settings.ProhibitDtd = false;
settings.MaxCharactersFromEntities = 1000;
XmlReader reader = XmlReader.Create(stream, settings);
```

Example

If you need to resolve inline entities but do not need to resolve external entities, set the XmlReaderSettings.XmlResolver property to null. For example:

```
XmlReaderSettings settings = new XmlReaderSettings();
settings.ProhibitDtd = false;
settings.MaxCharactersFromEntities = 1000;
settings.XmlResolver = null;
XmlReader reader = XmlReader.Create(stream, settings);
```

Note that in MSXML6, ProhibitDTD is set to true (disabling DTD processing) by default. For Apple OSX/iOS code, there are two XML parsers you can use: NSXMLParser and libXML2.

Applications utilizing http.sys perform URL canonicalization verification

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	Any application that uses http.sys should follow these guidelines: • Limit the URL length to no more than 16,384 characters (ASCII or Unicode). This is the absolute maximum URL length based on the default Internet Information Services (IIS) 6 setting. Websites should strive for a length shorter than this if possible • Use the standard .NET Framework file I/O classes (such as FileStream) as these will take advantage of the canonicalization rules in the .NET FX • Explicitly build an allow-list of known filenames • Explicitly reject known filetypes you will not serve UrlScan rejects: exe, bat, cmd, com, htw, ida, idq, htr, idc, shtm[l], stm, printer, ini, pol, dat files • Catch the following exceptions: ◦ System.ArgumentException (for device names) ◦ System.NotSupportedException (for data streams) ◦ System.IO.FileNotFoundException (for invalid escaped filenames) ◦ System.IO.DirectoryNotFoundException (for invalid escaped dirs) • *Do not* call out to Win32 file I/O APIs. On an invalid URL gracefully return a 400 error to the user, and log the real error.

Ensure appropriate controls are in place when accepting files from users

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Unrestricted File Upload, File Signature Table

TITLE	DETAILS
Steps	Uploaded files represent a significant risk to applications. The first step in many attacks is to get some code to the system to be attacked. Then the attack only needs to find a way to get the code executed. Using a file upload helps the attacker accomplish the first step. The consequences of unrestricted file upload can vary, including complete system takeover, an overloaded file system or database, forwarding attacks to back-end systems, and simple defacement. It depends on what the application does with the uploaded file and especially where it is stored. Server side validation of file uploads is missing. Following security controls should be implemented for File Upload functionality: • File Extension check (only a valid set of allowed file type should be accepted) • Maximum file size limit • File should not be uploaded to webroot; the location should be a directory on non-system drive • Naming convention should be followed, such that the uploaded file name have some randomness, so as to prevent file overwrites • Files should be scanned for anti-virus before writing to the disk • Ensure that the file name and any other metadata (e.g., file path) are validated for malicious characters • File format signature should be checked, to prevent a user from uploading a masqueraded file (e.g., uploading an exe file by changing extension to txt)

Example

For the last point regarding file format signature validation, refer to the class below for details:

```
        private static Dictionary<string, List<byte[]>> fileSignature = new Dictionary<string, List<byte[]>>
                    {
                    { ".DOC", new List<byte[]> { new byte[] { 0xD0, 0xCF, 0x11, 0xE0, 0xA1, 0xB1, 0x1A, 0xE1
} } },
                    { ".DOCX", new List<byte[]> { new byte[] { 0x50, 0x4B, 0x03, 0x04 } } },
                    { ".PDF", new List<byte[]> { new byte[] { 0x25, 0x50, 0x44, 0x46 } } },
                    { ".ZIP", new List<byte[]>
                                        {
                                          new byte[] { 0x50, 0x4B, 0x03, 0x04 },
                                          new byte[] { 0x50, 0x4B, 0x4C, 0x49, 0x54, 0x55 },
                                          new byte[] { 0x50, 0x4B, 0x53, 0x70, 0x58 },
                                          new byte[] { 0x50, 0x4B, 0x05, 0x06 },
                                          new byte[] { 0x50, 0x4B, 0x07, 0x08 },
                                          new byte[] { 0x57, 0x69, 0x6E, 0x5A, 0x69, 0x70 }
                                            }
                                        },
                    { ".PNG", new List<byte[]> { new byte[] { 0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A, 0x1A, 0x0A
} } },
                    { ".JPG", new List<byte[]>
                                    {
                                              new byte[] { 0xFF, 0xD8, 0xFF, 0xE0 },
                                              new byte[] { 0xFF, 0xD8, 0xFF, 0xE1 },
                                              new byte[] { 0xFF, 0xD8, 0xFF, 0xE8 }
                                    }
                                    },
                    { ".JPEG", new List<byte[]>
```

```
            { ".JPEG", new List<byte[]>
                                {
                                    new byte[] { 0xFF, 0xD8, 0xFF, 0xE0 },
                                    new byte[] { 0xFF, 0xD8, 0xFF, 0xE2 },
                                    new byte[] { 0xFF, 0xD8, 0xFF, 0xE3 }
                                }
                                },
            { ".XLS", new List<byte[]>
                                {
                                  new byte[] { 0xD0, 0xCF, 0x11, 0xE0, 0xA1, 0xB1, 0x1A, 0xE1 },
                                  new byte[] { 0x09, 0x08, 0x10, 0x00, 0x00, 0x06, 0x05, 0x00 },
                                  new byte[] { 0xFD, 0xFF, 0xFF, 0xFF }
                                }
                                },
            { ".XLSX", new List<byte[]> { new byte[] { 0x50, 0x4B, 0x03, 0x04 } } },
            { ".GIF", new List<byte[]> { new byte[] { 0x47, 0x49, 0x46, 0x38 } } }
        };

    public static bool IsValidFileExtension(string fileName, byte[] fileData, byte[] allowedChars)
    {
        if (string.IsNullOrEmpty(fileName) || fileData == null || fileData.Length == 0)
        {
            return false;
        }

        bool flag = false;
        string ext = Path.GetExtension(fileName);
        if (string.IsNullOrEmpty(ext))
        {
            return false;
        }

        ext = ext.ToUpperInvariant();

        if (ext.Equals(".TXT") || ext.Equals(".CSV") || ext.Equals(".PRN"))
        {
            foreach (byte b in fileData)
            {
                if (b > 0x7F)
                {
                    if (allowedChars != null)
                    {
                        if (!allowedChars.Contains(b))
                        {
                            return false;
                        }
                    }
                    else
                    {
                        return false;
                    }
                }
            }

            return true;
        }

        if (!fileSignature.ContainsKey(ext))
        {
            return true;
        }

        List<byte[]> sig = fileSignature[ext];
        foreach (byte[] b in sig)
        {
            var curFileSig = new byte[b.Length];
            Array.Copy(fileData, curFileSig, b.Length);
            if (curFileSig.SequenceEqual(b))
            {
```

```
                    flag = true;
                    break;
                }
            }

            return flag;
        }
```

Ensure that type-safe parameters are used in Web Application for data access

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	If you use the Parameters collection, SQL treats the input is as a literal value rather then as executable code. The Parameters collection can be used to enforce type and length constraints on input data. Values outside of the range trigger an exception. If type-safe SQL parameters are not used, attackers might be able to execute injection attacks that are embedded in the unfiltered input. Use type safe parameters when constructing SQL queries to avoid possible SQL injection attacks that can occur with unfiltered input. You can use type safe parameters with stored procedures and with dynamic SQL statements. Parameters are treated as literal values by the database and not as executable code. Parameters are also checked for type and length.

Example

The following code shows how to use type safe parameters with the SqlParameterCollection when calling a stored procedure.

```
using System.Data;
using System.Data.SqlClient;

using (SqlConnection connection = new SqlConnection(connectionString))
{
DataSet userDataset = new DataSet();
SqlDataAdapter myCommand = new SqlDataAdapter("LoginStoredProcedure", connection);
myCommand.SelectCommand.CommandType = CommandType.StoredProcedure;
myCommand.SelectCommand.Parameters.Add("@au_id", SqlDbType.VarChar, 11);
myCommand.SelectCommand.Parameters["@au_id"].Value = SSN.Text;
myCommand.Fill(userDataset);
}
```

In the preceding code example, the input value cannot be longer than 11 characters. If the data does not

conform to the type or length defined by the parameter, the SqlParameter class throws an exception.

Use separate model binding classes or binding filter lists to prevent MVC mass assignment vulnerability

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	MVC5, MVC6
Attributes	N/A
References	Metadata Attributes, Public Key Security Vulnerability And Mitigation, Complete Guide to Mass Assignment in ASP.NET MVC, Getting Started with EF using MVC
Steps	• **When should I look for over-posting vulnerabilities?** - Over-posting vulnerabilities can occur any place you bind model classes from user input. Frameworks like MVC can represent user data in custom .NET classes, including Plain Old CLR Objects (POCOs). MVC automatically populates these model classes with data from the request, providing a convenient representation for dealing with user input. When these classes include properties that should not be set by the user, the application can be vulnerable to over-posting attacks, which allow user control of data that the application never intended. Like MVC model binding, database access technologies such as object/relational mappers like Entity Framework often also support using POCO objects to represent database data. These data model classes provide the same convenience in dealing with database data as MVC does in dealing with user input. Because both MVC and the database support similar models, like POCO objects, it seems easy to reuse the same classes for both purposes. This practice fails to preserve separation of concerns, and it is one common area where unintended properties are exposed to model binding, enabling over-posting attacks. • **Why shouldn't I use my unfiltered database model classes as parameters to my MVC actions?** - Because MVC model binding will bind anything in that class. Even if the data does not appear in your view, a malicious user can send an HTTP request with this data included, and MVC will gladly bind it because your action says that database class is the shape of data it should accept for user input. • **Why should I care about the shape used for model binding?** - Using ASP.NET MVC model binding with overly broad models exposes an application to over-posting attacks. Over-posting could enable attackers to change application data beyond what the developer intended, such as overriding the price for an item or the security

TITLE	DETAILS
	privileges for an account. Applications should use action-specific binding models (or specific allowed property filter lists) to provide an explicit contract for what untrusted input to allow via model binding. • **Is having separate binding models just duplicating code?** - No, it is a matter of separation of concerns. If you reuse database models in action methods, you are saying any property (or sub-property) in that class can be set by the user in an HTTP request. If that is not what you want MVC to do, you need a filter list or a separate class shape to show MVC what data can come from user input instead. • **If I have separate binding models for user input, do I have to duplicate all my data annotation attributes?** - Not necessarily. You can use MetadataTypeAttribute on the database model class to link to the metadata on a model binding class. Just note that the type referenced by the MetadataTypeAttribute must be a subset of the referencing type (it can have fewer properties, but not more). • **Moving data back and forth between user input models and database models is tedious. Can I just copy over all properties using reflection?** - Yes. The only properties that appear in the binding models are the ones you have determined to be safe for user input. There is no security reason that prevents using reflection to copy over all properties that exist in common between these two models. • **What about [Bind(Exclude ="â€¦")]. Can I use that instead of having separate binding models?** - This approach is not recommended. Using [Bind(Exclude ="â€¦")] means that any new property is bindable by default. When a new property is added, there is an extra step to remember to keep things secure, rather than having the design be secure by default. Depending on the developer checking this list every time a property is added is risky. • **Is [Bind(Include ="â€¦")] useful for Edit operations?** - No. [Bind(Include ="â€¦")] is only suitable for INSERT-style operations (adding new data). For UPDATE-style operations (revising existing data), use another approach, like having separate binding models or passing an explicit list of allowed properties to UpdateModel or TryUpdateModel. Adding a [Bind(Include ="â€¦")] attribute on an Edit operation means that MVC will create an object instance and set only the listed properties, leaving all others at their default values. When the data is persisted, it will entirely replace the existing entity, resetting the values for any omitted properties to their defaults. For example, if IsAdmin was omitted from a [Bind(Include ="â€¦")] attribute on an Edit operation, any user whose name was edited via this action would be reset to IsAdmin = false (any edited user would lose administrator status). If you want to prevent updates to certain properties, use one of the other approaches above. Note that some versions of MVC tooling generate controller classes with [Bind(Include ="â€¦")] on Edit actions and imply that removing a property from that list will prevent over-

TITLE	DETAILS
	posting attacks. However, as described above, that approach does not work as intended and instead will reset any data in the omitted properties to their default values. • **For Create operations, are there any caveats using [Bind(Include ="â€¦")] rather than separate binding models?** - Yes. First this approach does not work for Edit scenarios, requiring maintaining two separate approaches for mitigating all over-posting vulnerabilities. Second, separate binding models enforce separation of concerns between the shape used for user input and the shape used for persistence, something [Bind(Include ="â€¦")] does not do. Third, note that [Bind(Include ="â€¦")] can only handle top-level properties; you cannot allow only portions of sub-properties (such as "Details.Name") in the attribute. Finally, and perhaps most importantly, using [Bind(Include ="â€¦")] adds an extra step that must be remembered any time the class is used for model binding. If a new action method binds to the data class directly and forgets to include a [Bind(Include ="â€¦")] attribute, it can be vulnerable to over-posting attacks, so the [Bind(Include ="â€¦")] approach is somewhat less secure by default. If you use [Bind(Include ="â€¦")], take care always to remember to specify it every time your data classes appear as action method parameters. • **For Create operations, what about putting the [Bind(Include ="â€¦")] attribute on the model class itself? Does not this approach avoid the need to remember putting the attribute on every action method?** - This approach works in some cases. Using [Bind(Include ="â€¦")] on the model type itself (rather than on action parameters using this class), does avoid the need to remember to include the [Bind(Include ="â€¦")] attribute on every action method. Using the attribute directly on the class effectively creates a separate surface area of this class for model binding purposes. However, this approach only allows for one model binding shape per model class. If one action method needs to allow model binding of a field (for example, an administrator-only action that updates user roles) and other actions need to prevent model binding of this field, this approach will not work. Each class can only have one model binding shape; if different actions need different model binding shapes, they need to represent these separate shapes using either separate model binding classes or separate [Bind(Include ="â€¦")] attributes on the action methods. • **What are binding models? Are they the same thing as view models?** - These are two related concepts. The term binding model refers to a model class used in an action's parameter list (the shape passed from MVC model binding to the action method). The term view model refers to a model class passed from an action method to a view. Using a view-specific model is a common approach for passing data from an action method to a view. Often, this shape is also suitable for model binding, and the term view model can be used to refer the same model used in both places. To be precise, this procedure talks specifically about binding models,

TITLE	DETAILS
	focusing on the shape passed to the action, which is what matters for mass assignment purposes.

Encode untrusted web output prior to rendering

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic, Web Forms, MVC5, MVC6
Attributes	N/A
References	How to prevent Cross-site scripting in ASP.NET, Cross-site Scripting, XSS (Cross Site Scripting) Prevention Cheat Sheet
Steps	Cross-site scripting (commonly abbreviated as XSS) is an attack vector for online services or any application/component that consumes input from the web. XSS vulnerabilities may allow an attacker to execute script on another user's machine through a vulnerable web application. Malicious scripts can be used to steal cookies and otherwise tamper with a victim's machine through JavaScript. XSS is prevented by validating user input, ensuring it is well formed and encoding before it is rendered in a web page. Input validation and output encoding can be done by using Web Protection Library. For Managed code (C#, VB.NET, etc.), use one or more appropriate encoding methods from the Web Protection (Anti-XSS) Library, depending on the context where the user input gets manifested:

Example

```
* Encoder.HtmlEncode
* Encoder.HtmlAttributeEncode
* Encoder.JavaScriptEncode
* Encoder.UrlEncode
* Encoder.VisualBasicScriptEncode
* Encoder.XmlEncode
* Encoder.XmlAttributeEncode
* Encoder.CssEncode
* Encoder.LdapEncode
```

Perform input validation and filtering on all string type Model properties

TITLE	DETAILS
Component	Web Application
SDL Phase	Build

TITLE	DETAILS
Applicable Technologies	Generic, MVC5, MVC6
Attributes	N/A
References	Adding Validation, Validating Model Data in an MVC Application, Guiding Principles For Your ASP.NET MVC Applications
Steps	All the input parameters must be validated before they are used in the application to ensure that the application is safeguarded against malicious user inputs. Validate the input values using regular expression validations on server side with a allowed list validation strategy. Unsanitized user inputs / parameters passed to the methods can cause code injection vulnerabilities. For web applications, entry points can also include form fields, QueryStrings, cookies, HTTP headers, and web service parameters. The following input validation checks must be performed upon model binding: • The model properties should be annotated with RegularExpression annotation, for accepting allowed characters and maximum permissible length • The controller methods should perform ModelState validity

Sanitization should be applied on form fields that accept all characters, e.g, rich text editor

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Encode Unsafe Input, HTML Sanitizer

TITLE	DETAILS
Steps	Identify all static markup tags that you want to use. A common practice is to restrict formatting to safe HTML elements, such as `<b>` (bold) and `<i>` (italic). Before writing the data, HTML-encode it. This makes any malicious script safe by causing it to be handled as text, not as executable code. 1. Disable ASP.NET request validation by the adding the ValidateRequest="false" attribute to the @ Page directive 2. Encode the string input with the HtmlEncode method 3. Use a StringBuilder and call its Replace method to selectively remove the encoding on the HTML elements that you want to permit The page-in the references disables ASP.NET request validation by setting `ValidateRequest="false"`. It HTML-encodes the input and selectively allows the `<b>` and `<i>` Alternatively, a .NET library for HTML sanitization may also be used. HtmlSanitizer is a .NET library for cleaning HTML fragments and documents from constructs that can lead to XSS attacks. It uses AngleSharp to parse, manipulate, and render HTML and CSS. HtmlSanitizer can be installed as a NuGet package, and the user input can be passed through relevant HTML or CSS sanitization methods, as applicable, on the server side. Please note that Sanitization as a security control should be considered only as a last option. Input validation and Output Encoding are considered better security controls.

Do not assign DOM elements to sinks that do not have inbuilt encoding

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Many javascript functions don't do encoding by default. When assigning untrusted input to DOM elements via such functions, may result in cross site script (XSS) executions.

Example

Following are insecure examples:

```
document.getElementByID("div1").innerHtml = value;
$("#userName").html(res.Name);
return $('<div/>').html(value)
$('body').append(resHTML);
```

Don't use `innerHtml`; instead use `innerText`. Similarly, instead of `$("#elm").html()`, use `$("#elm").text()`

Validate all redirects within the application are closed or done safely

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	The OAuth 2.0 Authorization Framework - Open Redirectors
Steps	Application design requiring redirection to a user-supplied location must constrain the possible redirection targets to a predefined "safe" list of sites or domains. All redirects in the application must be closed/safe. To do this: • Identify all redirects • Implement an appropriate mitigation for each redirect. Appropriate mitigations include redirect allowed list or user confirmation. If a web site or service with an open redirect vulnerability uses Facebook/OAuth/OpenID identity providers, an attacker can steal a user's logon token and impersonate that user. This is an inherent risk when using OAuth, which is documented in RFC 6749 "The OAuth 2.0 Authorization Framework", Section 10.15 "Open Redirects" Similarly, users' credentials can be compromised by spear phishing attacks using open redirects

Implement input validation on all string type parameters accepted by Controller methods

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic, MVC5, MVC6
Attributes	N/A

TITLE	DETAILS
References	Validating Model Data in an MVC Application, Guiding Principles For Your ASP.NET MVC Applications
Steps	For methods that just accept primitive data type, and not models as argument,input validation using Regular Expression should be done. Here Regex.IsMatch should be used with a valid regex pattern. If the input doesn't match the specified Regular Expression, control should not proceed further, and an adequate warning regarding validation failure should be displayed.

Set upper limit timeout for regular expression processing to prevent DoS due to bad regular expressions

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic, Web Forms, MVC5, MVC6
Attributes	N/A
References	DefaultRegexMatchTimeout Property
Steps	To ensure denial of service attacks against badly created regular expressions, that cause a lot of backtracking, set the global default timeout. If the processing time takes longer than the defined upper limit, it would throw a Timeout exception. If nothing is configured, the timeout would be infinite.

Example

For example, the following configuration will throw a RegexMatchTimeoutException, if the processing takes more than 5 seconds:

```
<httpRuntime targetFramework="4.5" defaultRegexMatchTimeout="00:00:05" />
```

Avoid using Html.Raw in Razor views

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	MVC5, MVC6
Attributes	N/A

TITLE	DETAILS
References	N/A
Step	ASP.NET WebPages (Razor) perform automatic HTML encoding. All strings printed by embedded code nuggets (@ blocks) are automatically HTML-encoded. However, when `HtmlHelper.Raw` Method is invoked, it returns markup that is not HTML encoded. If `Html.Raw()` helper method is used, it bypasses the automatic encoding protection that Razor provides.

Example

Following is an insecure example:

```
<div class="form-group">
            @Html.Raw(Model.AccountConfirmText)
        </div>
        <div class="form-group">
            @Html.Raw(Model.PaymentConfirmText)
        </div>
</div>
```

Do not use `Html.Raw()` unless you need to display markup. This method does not perform output encoding implicitly. Use other ASP.NET helpers e.g., `@Html.DisplayFor()`

Do not use dynamic queries in stored procedures

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	A SQL injection attack exploits vulnerabilities in input validation to run arbitrary commands in the database. It can occur when your application uses input to construct dynamic SQL statements to access the database. It can also occur if your code uses stored procedures that are passed strings that contain raw user input. Using the SQL injection attack, the attacker can execute arbitrary commands in the database. All SQL statements (including the SQL statements in stored procedures) must be parameterized. Parameterized SQL statements will accept characters that have special meaning to SQL (like single quote) without problems because they are strongly typed.

Example

Following is an example of insecure dynamic Stored Procedure:

```
CREATE PROCEDURE [dbo].[uspGetProductsByCriteria]
(
  @productName nvarchar(200) = NULL,
  @startPrice float = NULL,
  @endPrice float = NULL
)
AS
 BEGIN
  DECLARE @sql nvarchar(max)
  SELECT @sql = ' SELECT ProductID, ProductName, Description, UnitPrice, ImagePath' +
       ' FROM dbo.Products WHERE 1 = 1 '
       PRINT @sql
  IF @productName IS NOT NULL
     SELECT @sql = @sql + ' AND ProductName LIKE ''%' + @productName + '%'''
  IF @startPrice IS NOT NULL
     SELECT @sql = @sql + ' AND UnitPrice > ''' + CONVERT(VARCHAR(10),@startPrice) + ''''
  IF @endPrice IS NOT NULL
     SELECT @sql = @sql + ' AND UnitPrice < ''' + CONVERT(VARCHAR(10),@endPrice) + ''''

  PRINT @sql
  EXEC(@sql)
 END
```

Example

Following is the same stored procedure implemented securely:

```
CREATE PROCEDURE [dbo].[uspGetProductsByCriteriaSecure]
(
             @productName nvarchar(200) = NULL,
             @startPrice float = NULL,
             @endPrice float = NULL
)
AS
       BEGIN
             SELECT ProductID, ProductName, Description, UnitPrice, ImagePath
             FROM dbo.Products where
             (@productName IS NULL or ProductName like '%'+ @productName +'%')
             AND
             (@startPrice IS NULL or UnitPrice > @startPrice)
             AND
             (@endPrice IS NULL or UnitPrice < @endPrice)
       END
```

Ensure that model validation is done on Web API methods

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	MVC5, MVC6
Attributes	N/A
References	Model Validation in ASP.NET Web API

TITLE	DETAILS
Steps	When a client sends data to a web API, it is mandatory to validate the data before doing any processing. For ASP.NET Web APIs which accept models as input, use data annotations on models to set validation rules on the properties of the model.

Example

The following code demonstrates the same:

```
using System.ComponentModel.DataAnnotations;

namespace MyApi.Models
{
    public class Product
    {
        public int Id { get; set; }
        [Required]
        [RegularExpression(@"^[a-zA-Z0-9]*$", ErrorMessage="Only alphanumeric characters are allowed.")]
        public string Name { get; set; }
        public decimal Price { get; set; }
        [Range(0, 999)]
        public double Weight { get; set; }
    }
}
```

Example

In the action method of the API controllers, validity of the model has to be explicitly checked as shown below:

```
namespace MyApi.Controllers
{
    public class ProductsController : ApiController
    {
        public HttpResponseMessage Post(Product product)
        {
            if (ModelState.IsValid)
            {
                // Do something with the product (not shown).

                return new HttpResponseMessage(HttpStatusCode.OK);
            }
            else
            {
                return Request.CreateErrorResponse(HttpStatusCode.BadRequest, ModelState);
            }
        }
    }
}
```

Implement input validation on all string type parameters accepted by Web API methods

TITLE	DETAILS
Component	Web API

TITLE	DETAILS
SDL Phase	Build
Applicable Technologies	Generic, MVC 5, MVC 6
Attributes	N/A
References	Validating Model Data in an MVC Application, Guiding Principles For Your ASP.NET MVC Applications
Steps	For methods that just accept primitive data type, and not models as argument,input validation using Regular Expression should be done. Here Regex.IsMatch should be used with a valid regex pattern. If the input doesn't match the specified Regular Expression, control should not proceed further, and an adequate warning regarding validation failure should be displayed.

Ensure that type-safe parameters are used in Web API for data access

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	If you use the Parameters collection, SQL treats the input is as a literal value rather then as executable code. The Parameters collection can be used to enforce type and length constraints on input data. Values outside of the range trigger an exception. If type-safe SQL parameters are not used, attackers might be able to execute injection attacks that are embedded in the unfiltered input. Use type safe parameters when constructing SQL queries to avoid possible SQL injection attacks that can occur with unfiltered input. You can use type safe parameters with stored procedures and with dynamic SQL statements. Parameters are treated as literal values by the database and not as executable code. Parameters are also checked for type and length.

Example

The following code shows how to use type safe parameters with the SqlParameterCollection when calling a stored procedure.

```
using System.Data;
using System.Data.SqlClient;

using (SqlConnection connection = new SqlConnection(connectionString))
{
DataSet userDataset = new DataSet();
SqlDataAdapter myCommand = new SqlDataAdapter("LoginStoredProcedure", connection);
myCommand.SelectCommand.CommandType = CommandType.StoredProcedure;
myCommand.SelectCommand.Parameters.Add("@au_id", SqlDbType.VarChar, 11);
myCommand.SelectCommand.Parameters["@au_id"].Value = SSN.Text;
myCommand.Fill(userDataset);
}
```

In the preceding code example, the input value cannot be longer than 11 characters. If the data does not conform to the type or length defined by the parameter, the SqlParameter class throws an exception.

Use parameterized SQL queries for Cosmos DB

TITLE	DETAILS
Component	Azure Document DB
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Announcing SQL Parameterization in Azure Cosmos DB
Steps	Although Azure Cosmos DB only supports read-only queries, SQL injection is still possible if queries are constructed by concatenating with user input. It might be possible for a user to gain access to data they shouldn't be accessing within the same collection by crafting malicious SQL queries. Use parameterized SQL queries if queries are constructed based on user input.

WCF Input validation through Schema binding

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	Generic, NET Framework 3
Attributes	N/A
References	MSDN

TITLE	DETAILS
Steps	Lack of validation leads to different type injection attacks. Message validation represents one line of defense in the protection of your WCF application. With this approach, you validate messages using schemas to protect WCF service operations from attack by a malicious client. Validate all messages received by the client to protect the client from attack by a malicious service. Message validation makes it possible to validate messages when operations consume message contracts or data contracts, which cannot be done using parameter validation. Message validation allows you to create validation logic inside schemas, thereby providing more flexibility and reducing development time. Schemas can be reused across different applications inside the organization, creating standards for data representation. Additionally, message validation allows you to protect operations when they consume more complex data types involving contracts representing business logic. To perform message validation, you first build a schema that represents the operations of your service and the data types consumed by those operations. You then create a .NET class that implements a custom client message inspector and custom dispatcher message inspector to validate the messages sent/received to/from the service. Next, you implement a custom endpoint behavior to enable message validation on both the client and the service. Finally, you implement a custom configuration element on the class that allows you to expose the extended custom endpoint behavior in the configuration file of the service or the client"

WCF- Input validation through Parameter Inspectors

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	Generic, NET Framework 3
Attributes	N/A
References	MSDN

TITLE	DETAILS
Steps	Input and data validation represents one important line of defense in the protection of your WCF application. You should validate all parameters exposed in WCF service operations to protect the service from attack by a malicious client. Conversely, you should also validate all return values received by the client to protect the client from attack by a malicious service WCF provides different extensibility points that allow you to customize the WCF runtime behavior by creating custom extensions. Message Inspectors and Parameter Inspectors are two extensibility mechanisms used to gain greater control over the data passing between a client and a service. You should use parameter inspectors for input validation and use message inspectors only when you need to inspect the entire message flowing in and out of a service. To perform input validation, you will build a .NET class and implement a custom parameter inspector in order to validate parameters on operations in your service. You will then implement a custom endpoint behavior to enable validation on both the client and the service. Finally, you will implement a custom configuration element on the class that allows you to expose the extended custom endpoint behavior in the configuration file of the service or the client

Security Frame: Sensitive Data | Mitigations

3/5/2021 • 16 minutes to read • Edit Online

PRODUCT/SERVICE	ARTICLE
Machine Trust Boundary	• Ensure that binaries are obfuscated if they contain sensitive information • Consider using Encrypted File System (EFS) is used to protect confidential user-specific data • Ensure that sensitive data stored by the application on the file system is encrypted
Web Application	• Ensure that sensitive content is not cached on the browser • Encrypt sections of Web App's configuration files that contain sensitive data • Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs • Ensure that sensitive data displayed on the user screen is masked
Database	• Implement dynamic data masking to limit sensitive data exposure non privileged users • Ensure that passwords are stored in salted hash format • Ensure that sensitive data in database columns is encrypted • Ensure that database-level encryption (TDE) is enabled • Ensure that database backups are encrypted
Web API	• Ensure that sensitive data relevant to Web API is not stored in browser's storage
Azure Document DB	• Encrypt sensitive data stored in Azure Cosmos DB
Azure IaaS VM Trust Boundary	• Use Azure Disk Encryption to encrypt disks used by Virtual Machines
Service Fabric Trust Boundary	• Encrypt secrets in Service Fabric applications

PRODUCT/SERVICE	ARTICLE
Dynamics CRM	• Perform security modeling and use Business Units/Teams where required • Minimize access to share feature on critical entities • Train users on the risks associated with the Dynamics CRM Share feature and good security practices • Include a development standards rule proscribing showing config details in exception management
Azure Storage	• Use Azure Storage Service Encryption (SSE) for Data at Rest (Preview) • Use Client-Side Encryption to store sensitive data in Azure Storage
Mobile Client	• Encrypt sensitive or PII data written to phones local storage • Obfuscate generated binaries before distributing to end users
WCF	• Set clientCredentialType to Certificate or Windows • WCF-Security Mode is not enabled

Ensure that binaries are obfuscated if they contain sensitive information

TITLE	DETAILS
Component	Machine Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that binaries are obfuscated if they contain sensitive information such as trade secrets, sensitive business logic that should not reversed. This is to stop reverse engineering of assemblies. Tools like `CryptoObfuscator` may be used for this purpose.

Consider using Encrypted File System (EFS) is used to protect confidential user-specific data

TITLE	DETAILS
Component	Machine Trust Boundary

TITLE	DETAILS
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Consider using Encrypted File System (EFS) is used to protect confidential user-specific data from adversaries with physical access to the computer.

Ensure that sensitive data stored by the application on the file system is encrypted

TITLE	DETAILS
Component	Machine Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Ensure that sensitive data stored by the application on the file system is encrypted (e.g., using DPAPI), if EFS cannot be enforced

Ensure that sensitive content is not cached on the browser

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic, Web Forms, MVC5, MVC6
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	Browsers can store information for purposes of caching and history. These cached files are stored in a folder, like the Temporary Internet Files folder in the case of Internet Explorer. When these pages are referred again, the browser displays them from its cache. If sensitive information is displayed to the user (such as their address, credit card details, Social Security Number, or username), then this information could be stored in browser's cache, and therefore retrievable through examining the browser's cache or by simply pressing the browser's "Back" button. Set cache-control response header value to "no-store" for all pages.

Example

```
<configuration>
  <system.webServer>
   <httpProtocol>
    <customHeaders>
        <add name="Cache-Control" value="no-store" />
        <add name="Pragma" value="no-cache" />
        <add name="Expires" value="-1" />
    </customHeaders>
  </httpProtocol>
 </system.webServer>
</configuration>
```

Example

This may be implemented through a filter. Following example may be used:

```
public override void OnActionExecuting(ActionExecutingContext filterContext)
        {
            if (filterContext == null || (filterContext.HttpContext != null &&
filterContext.HttpContext.Response != null && filterContext.HttpContext.Response.IsRequestBeingRedirected))
            {
                //// Since this is MVC pipeline, this should never be null.
                return;
            }

            var attributes =
filterContext.ActionDescriptor.GetCustomAttributes(typeof(System.Web.Mvc.OutputCacheAttribute), false);
            if (attributes == null || **Attributes**.Count() == 0)
            {
                filterContext.HttpContext.Response.Cache.SetNoStore();
                filterContext.HttpContext.Response.Cache.SetCacheability(HttpCacheability.NoCache);
                filterContext.HttpContext.Response.Cache.SetExpires(DateTime.UtcNow.AddHours(-1));
                if (!filterContext.IsChildAction)
                {
                    filterContext.HttpContext.Response.AppendHeader("Pragma", "no-cache");
                }
            }

            base.OnActionExecuting(filterContext);
        }
```

Encrypt sections of Web App's configuration files that contain sensitive data

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	How To: Encrypt Configuration Sections in ASP.NET 2.0 Using DPAPI, Specifying a Protected Configuration Provider, Using Azure Key Vault to protect application secrets
Steps	Configuration files such as the Web.config, appsettings.json are often used to hold sensitive information, including user names, passwords, database connection strings, and encryption keys. If you do not protect this information, your application is vulnerable to attackers or malicious users obtaining sensitive information such as account user names and passwords, database names and server names. Based on the deployment type (azure/on-prem), encrypt the sensitive sections of config files using DPAPI or services like Azure Key Vault.

Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	MSDN: autocomplete attribute, Using AutoComplete in HTML, HTML Sanitization Vulnerability, Autocomplete.,again?!
Steps	The autocomplete attribute specifies whether a form should have autocomplete on or off. When autocomplete is on, the browser automatically complete values based on values that the user has entered before. For example, when a new name and password is entered in a form and the form is submitted, the browser asks if the password should be saved.Thereafter when the form is displayed, the name and password are filled in automatically or are completed as the name is entered. An attacker with local access could obtain the clear text password from the browser cache. By default autocomplete is enabled, and it must explicitly be disabled.

Example

```
<form action="Login.aspx" method="post " autocomplete="off" >
      Social Security Number: <input type="text" name="ssn" />
      <input type="submit" value="Submit" />
</form>
```

Ensure that sensitive data displayed on the user screen is masked

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Sensitive data such as passwords, credit card numbers, SSN etc. should be masked when displayed on the screen. This is to prevent unauthorized personnel from accessing the data (e.g., shoulder-surfing passwords, support personnel viewing SSN numbers of users) . Ensure that these data elements are not visible in plain text and are appropriately masked. This has to be taken care while accepting them as input (e.g,. input type="password") as well as displaying back on the screen (e.g., display only the last 4 digits of the credit card number).

Implement dynamic data masking to limit sensitive data exposure non privileged users

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Sql Azure, OnPrem
Attributes	SQL Version - V12, SQL Version - MsSQL2016
References	Dynamic Data Masking

TITLE	DETAILS
Steps	The purpose of dynamic data masking is to limit exposure of sensitive data, preventing users who should not have access to the data from viewing it. Dynamic data masking does not aim to prevent database users from connecting directly to the database and running exhaustive queries that expose pieces of the sensitive data. Dynamic data masking is complementary to other SQL Server security features (auditing, encryption, row level security...) and it is highly recommended to use this feature in conjunction with them in addition in order to better protect the sensitive data in the database. Please note that this feature is supported only by SQL Server starting with 2016 and Azure SQL Database.

Ensure that passwords are stored in salted hash format

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Password Hashing using .NET Crypto APIs
Steps	Passwords should not be stored in custom user store databases. Password hashes should be stored with salt values instead. Make sure the salt for the user is always unique and you apply b-crypt, s-crypt or PBKDF2 before storing the password, with a minimum work factor iteration count of 150,000 loops to eliminate the possibility of brute forcing.

Ensure that sensitive data in database columns is encrypted

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Generic
Attributes	SQL Version - All
References	Encrypting sensitive data in SQL server, How to: Encrypt a Column of Data in SQL Server, Encrypt by Certificate

TITLE	DETAILS
Steps	Sensitive data such as credit card numbers has to be encrypted in the database. Data can be encrypted using column-level encryption or by an application function using the encryption functions.

Ensure that database-level encryption (TDE) is enabled

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Understanding SQL Server Transparent Data Encryption (TDE)
Steps	Transparent Data Encryption (TDE) feature in SQL server helps in encrypting sensitive data in a database and protect the keys that are used to encrypt the data with a certificate. This prevents anyone without the keys from using the data. TDE protects data "at rest", meaning the data and log files. It provides the ability to comply with many laws, regulations, and guidelines established in various industries.

Ensure that database backups are encrypted

TITLE	DETAILS
Component	Database
SDL Phase	Build
Applicable Technologies	SQL Azure, OnPrem
Attributes	SQL Version - V12, SQL Version - MsSQL2014
References	SQL database backup encryption
Steps	SQL Server has the ability to encrypt the data while creating a backup. By specifying the encryption algorithm and the encryptor (a Certificate or Asymmetric Key) when creating a backup, one can create an encrypted backup file.

Ensure that sensitive data relevant to Web API is not stored in browser's storage

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	MVC 5, MVC 6
Attributes	Identity Provider - ADFS, Identity Provider - Azure AD
References	N/A
Steps	In certain implementations, sensitive artifacts relevant to Web API's authentication are stored in browser's local storage. E.g., Azure AD authentication artifacts like adal.idtoken, adal.nonce.idtoken, adal.access.token.key, adal.token.keys, adal.state.login, adal.session.state, adal.expiration.key etc. All these artifacts are available even after sign out or browser is closed. If an adversary gets access to these artifacts, he/she can reuse them to access the protected resources (APIs). Ensure that all sensitive artifacts related to Web API is not stored in browser's storage. In cases where client-side storage is unavoidable (e.g., Single Page Applications (SPA) that leverage Implicit OpenIdConnect/OAuth flows need to store access tokens locally), use storage choices with do not have persistence. e.g., prefer SessionStorage to LocalStorage.

Example

The below JavaScript snippet is from a custom authentication library which stores authentication artifacts in local storage. Such implementations should be avoided.

```
ns.AuthHelper.Authenticate = function () {
window.config = {
instance: 'https://login.microsoftonline.com/',
tenant: ns.Configurations.Tenant,
clientId: ns.Configurations.AADApplicationClientID,
postLogoutRedirectUri: window.location.origin,
cacheLocation: 'localStorage', // enable this for IE, as sessionStorage does not work for localhost.
};
```

Encrypt sensitive data stored in Cosmos DB

TITLE	DETAILS
Component	Azure Document DB
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A

TITLE	DETAILS
References	N/A
Steps	Encrypt sensitive data at application level before storing in document DB or store any sensitive data in other storage solutions like Azure Storage or Azure SQL

Use Azure Disk Encryption to encrypt disks used by Virtual Machines

TITLE	DETAILS
Component	Azure IaaS VM Trust Boundary
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	Using Azure Disk Encryption to encrypt disks used by your virtual machines
Steps	Azure Disk Encryption is a new feature that is currently in preview. This feature allows you to encrypt the OS disks and Data disks used by an IaaS Virtual Machine. For Windows, the drives are encrypted using industry-standard BitLocker encryption technology. For Linux, the disks are encrypted using the DM-Crypt technology. This is integrated with Azure Key Vault to allow you to control and manage the disk encryption keys. The Azure Disk Encryption solution supports the following three customer encryption scenarios: • Enable encryption on new IaaS VMs created from customer-encrypted VHD files and customer-provided encryption keys, which are stored in Azure Key Vault. • Enable encryption on new IaaS VMs created from the Azure Marketplace. • Enable encryption on existing IaaS VMs already running in Azure.

Encrypt secrets in Service Fabric applications

TITLE	DETAILS
Component	Service Fabric Trust Boundary
SDL Phase	Build
Applicable Technologies	Generic
Attributes	Environment - Azure

TITLE	DETAILS
References	Managing secrets in Service Fabric applications
Steps	Secrets can be any sensitive information, such as storage connection strings, passwords, or other values that should not be handled in plain text. Use Azure Key Vault to manage keys and secrets in service fabric applications.

Perform security modeling and use Business Units/Teams where required

TITLE	DETAILS
Component	Dynamics CRM
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Perform security modeling and use Business Units/Teams where required

Minimize access to share feature on critical entities

TITLE	DETAILS
Component	Dynamics CRM
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Minimize access to share feature on critical entities

Train users on the risks associated with the Dynamics CRM Share feature and good security practices

TITLE	DETAILS
Component	Dynamics CRM

TITLE	DETAILS
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Train users on the risks associated with the Dynamics CRM Share feature and good security practices

Include a development standards rule proscribing showing config details in exception management

TITLE	DETAILS
Component	Dynamics CRM
SDL Phase	Deployment
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Include a development standards rule proscribing showing config details in exception management outside development. Test for this as part of code reviews or periodic inspection.

Use Azure Storage Service Encryption (SSE) for Data at Rest (Preview)

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Build
Applicable Technologies	Generic
Attributes	StorageType - Blob
References	Azure Storage Service Encryption for Data at Rest (Preview)

TITLE	DETAILS
Steps	Azure Storage Service Encryption (SSE) for Data at Rest helps you protect and safeguard your data to meet your organizational security and compliance commitments. With this feature, Azure Storage automatically encrypts your data prior to persisting to storage and decrypts prior to retrieval. The encryption, decryption and key management is totally transparent to users. SSE applies only to block blobs, page blobs, and append blobs. The other types of data, including tables, queues, and files, will not be encrypted. Encryption and Decryption Workflow: • The customer enables encryption on the storage account • When the customer writes new data (PUT Blob, PUT Block, PUT Page, etc.) to Blob storage; every write is encrypted using 256-bit AES encryption, one of the strongest block ciphers available • When the customer needs to access data (GET Blob, etc.), data is automatically decrypted before returning to the user • If encryption is disabled, new writes are no longer encrypted and existing encrypted data remains encrypted until rewritten by the user. While encryption is enabled, writes to Blob storage will be encrypted. The state of data does not change with the user toggling between enabling/disabling encryption for the storage account • All encryption keys are stored, encrypted, and managed by Microsoft Please note that at this time, the keys used for the encryption are managed by Microsoft. Microsoft generates the keys originally, and manage the secure storage of the keys as well as the regular rotation as defined by internal Microsoft policy. In the future, customers will get the ability to manage their own >encryption keys, and provide a migration path from Microsoft-managed keys to customer-managed keys.

Use Client-Side Encryption to store sensitive data in Azure Storage

TITLE	DETAILS
Component	Azure Storage
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A

TITLE	DETAILS
References	Client-Side Encryption and Azure Key Vault for Microsoft Azure Storage, Tutorial: Encrypt and decrypt blobs in Microsoft Azure Storage using Azure Key Vault, Storing Data Securely in Azure Blob Storage with Azure Encryption Extensions
Steps	The Azure Storage Client Library for .NET Nuget package supports encrypting data within client applications before uploading to Azure Storage, and decrypting data while downloading to the client. The library also supports integration with Azure Key Vault for storage account key management. Here is a brief description of how client side encryption works: • The Azure Storage client SDK generates a content encryption key (CEK), which is a one-time-use symmetric key • Customer data is encrypted using this CEK • The CEK is then wrapped (encrypted) using the key encryption key (KEK). The KEK is identified by a key identifier and can be an asymmetric key pair or a symmetric key and can be managed locally or stored in Azure Key Vault. The Storage client itself never has access to the KEK. It just invokes the key wrapping algorithm that is provided by Key Vault. Customers can choose to use custom providers for key wrapping/unwrapping if they want • The encrypted data is then uploaded to the Azure Storage service. Check the links in the references section for low-level implementation details.

Encrypt sensitive or PII data written to phones local storage

TITLE	DETAILS
Component	Mobile Client
SDL Phase	Build
Applicable Technologies	Generic, Xamarin
Attributes	N/A
References	Manage settings and features on your devices with Microsoft Intune policies, Keychain Valet
Steps	If the application writes sensitive information like user's PII (email, phone number, first name, last name, preferences etc.)- on mobile's file system, then it should be encrypted before writing to the local file system. If the application is an enterprise application, then explore the possibility of publishing application using Windows Intune.

Example

Intune can be configured with following security policies to safeguard sensitive data:

```
Require encryption on mobile device
Require encryption on storage cards
Allow screen capture
```

Example

If the application is not an enterprise application, then use platform provided keystore, keychains to store encryption keys, using which cryptographic operation may be performed on the file system. Following code snippet shows how to access key from keychain using xamarin:

```
        protected static string EncryptionKey
        {
            get
            {
                if (String.IsNullOrEmpty(_Key))
                {
                    var query = new SecRecord(SecKind.GenericPassword);
                    query.Service = NSBundle.MainBundle.BundleIdentifier;
                    query.Account = "UniqueID";

                    NSData uniqueId = SecKeyChain.QueryAsData(query);
                    if (uniqueId == null)
                    {
                        query.ValueData = NSData.FromString(System.Guid.NewGuid().ToString());
                        var err = SecKeyChain.Add(query);
                        _Key = query.ValueData.ToString();
                    }
                    else
                    {
                        _Key = uniqueId.ToString();
                    }
                }

                return _Key;
            }
        }
```

Obfuscate generated binaries before distributing to end users

TITLE	DETAILS
Component	Mobile Client
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Crypto Obfuscation For .Net
Steps	Generated binaries (assemblies within apk) should be obfuscated to stop reverse engineering of assemblies.Tools like `CryptoObfuscator` may be used for this purpose.

Set clientCredentialType to Certificate or Windows

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	.NET Framework 3
Attributes	N/A
References	Fortify
Steps	Using a UsernameToken with a plaintext password over an unencrypted channel exposes the password to attackers who can sniff the SOAP messages. Service Providers that use the UsernameToken might accept passwords sent in plaintext. Sending plaintext passwords over an unencrypted channel can expose the credential to attackers who can sniff the SOAP message.

Example

The following WCF service provider configuration uses the UsernameToken:

```
<security mode="Message">
<message clientCredentialType="UserName" />
```

Set clientCredentialType to Certificate or Windows.

WCF-Security Mode is not enabled

TITLE	DETAILS
Component	WCF
SDL Phase	Build
Applicable Technologies	Generic, .NET Framework 3
Attributes	Security Mode - Transport, Security Mode - Message
References	MSDN, Fortify Kingdom, Fundamentals of WCF Security CoDe Magazine
Steps	No transport or message security has been defined. Applications that transmit messages without transport or message security cannot guarantee the integrity or confidentiality of the messages. When a WCF security binding is set to None, both transport and message security are disabled.

Example

The following configuration sets the security mode to None.

```
<system.serviceModel>
  <bindings>
    <wsHttpBinding>
      <binding name=""MyBinding"">
        <security mode=""None""/>
      </binding>
  </bindings>
</system.serviceModel>
```

Example

Security Mode Across all service bindings there are five possible security modes:

- None. Turns security off.
- Transport. Uses transport security for mutual authentication and message protection.
- Message. Uses message security for mutual authentication and message protection.
- Both. Allows you to supply settings for transport and message-level security (only MSMQ supports this).
- TransportWithMessageCredential. Credentials are passed with the message and message protection and server authentication are provided by the transport layer.
- TransportCredentialOnly. Client credentials are passed with the transport layer and no message protection is applied. Use transport and message security to protect the integrity and confidentiality of messages. The configuration below tells the service to use transport security with message credentials.

  ```
  <system.serviceModel>
  <bindings>
    <wsHttpBinding>
    <binding name=""MyBinding"">
    <security mode=""TransportWithMessageCredential""/>
    <message clientCredentialType=""Windows""/>
    </binding>
  </bindings>
  </system.serviceModel>
  ```

Security Frame: Session Management

3/5/2021 • 14 minutes to read • Edit Online

PRODUCT/SERVICE	ARTICLE
Azure AD	• Implement proper logout using ADAL methods when using Azure AD
IoT Device	• Use finite lifetimes for generated SaS tokens
Azure Document DB	• Use minimum token lifetimes for generated Resource tokens
ADFS	• Implement proper logout using WsFederation methods when using ADFS
Identity Server	• Implement proper logout when using Identity Server
Web Application	• Applications available over HTTPS must use secure cookies • All http based application should specify http only for cookie definition • Mitigate against Cross-Site Request Forgery (CSRF) attacks on ASP.NET web pages • Set up session for inactivity lifetime • Implement proper logout from the application
Web API	• Mitigate against Cross-Site Request Forgery (CSRF) attacks on ASP.NET Web APIs

Implement proper logout using ADAL methods when using Azure AD

TITLE	DETAILS
Component	Azure AD
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	If the application relies on access token issued by Azure AD, the logout event handler should call

Example

```
HttpContext.GetOwinContext().Authentication.SignOut(OpenIdConnectAuthenticationDefaults.AuthenticationType,
CookieAuthenticationDefaults.AuthenticationType)
```

Example

It should also destroy user's session by calling Session.Abandon() method. Following method shows secure implementation of user logout:

```
    [HttpPost]
        [ValidateAntiForgeryToken]
        public void LogOff()
        {
            string userObjectID =
ClaimsPrincipal.Current.FindFirst("http://schemas.microsoft.com/identity/claims/objectidentifier").Value;
            AuthenticationContext authContext = new AuthenticationContext(Authority + TenantId, new
NaiveSessionCache(userObjectID));
            authContext.TokenCache.Clear();
            Session.Clear();
            Session.Abandon();
            Response.SetCookie(new HttpCookie("ASP.NET_SessionId", string.Empty));
            HttpContext.GetOwinContext().Authentication.SignOut(
                OpenIdConnectAuthenticationDefaults.AuthenticationType,
                CookieAuthenticationDefaults.AuthenticationType);
        }
```

Use finite lifetimes for generated SaS tokens

TITLE	DETAILS
Component	IoT Device
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	SaS tokens generated for authenticating to Azure IoT Hub should have a finite expiry period. Keep the SaS token lifetimes to a minimum to limit the amount of time they can be replayed in case the tokens are compromised.

Use minimum token lifetimes for generated Resource tokens

TITLE	DETAILS
Component	Azure Document DB
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Reduce the timespan of resource token to a minimum value required. Resource tokens have a default valid timespan of 1 hour.

Implement proper logout using WsFederation methods when using ADFS

TITLE	DETAILS
Component	ADFS
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	If the application relies on STS token issued by ADFS, the logout event handler should call WSFederationAuthenticationModule.FederatedSignOut() method to log out the user. Also the current session should be destroyed, and the session token value should be reset and nullified.

Example

```
        [HttpPost, ValidateAntiForgeryToken]
        [Authorization]
        public ActionResult SignOut(string redirectUrl)
        {
            if (!this.User.Identity.IsAuthenticated)
            {
                return this.View("LogOff", null);
            }

            // Removes the user profile.
            this.Session.Clear();
            this.Session.Abandon();
            HttpContext.Current.Response.Cookies.Add(new System.Web.HttpCookie("ASP.NET_SessionId",
string.Empty)
                {
                    Expires = DateTime.Now.AddDays(-1D),
                    Secure = true,
                    HttpOnly = true
                });

            // Signs out at the specified security token service (STS) by using the WS-Federation protocol.
            Uri signOutUrl = new Uri(FederatedAuthentication.WSFederationAuthenticationModule.Issuer);
            Uri replyUrl = new Uri(FederatedAuthentication.WSFederationAuthenticationModule.Realm);
            if (!string.IsNullOrEmpty(redirectUrl))
            {
                replyUrl = new Uri(FederatedAuthentication.WSFederationAuthenticationModule.Realm +
redirectUrl);
            }
           //     Signs out of the current session and raises the appropriate events.
            var authModule = FederatedAuthentication.WSFederationAuthenticationModule;
            authModule.SignOut(false);
        //     Signs out at the specified security token service (STS) by using the WS-Federation
        //     protocol.
            WSFederationAuthenticationModule.FederatedSignOut(signOutUrl, replyUrl);
            return new RedirectResult(redirectUrl);
        }
```

Implement proper logout when using Identity Server

TITLE	DETAILS
Component	Identity Server
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	IdentityServer3-Federated sign out
Steps	IdentityServer supports the ability to federate with external identity providers. When a user signs out of an upstream identity provider, depending upon the protocol used, it might be possible to receive a notification when the user signs out. It allows IdentityServer to notify its clients so they can also sign the user out. Check the documentation in the references section for the implementation details.

Applications available over HTTPS must use secure cookies

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	EnvironmentType - OnPrem
References	httpCookies Element (ASP.NET Settings Schema), HttpCookie.Secure Property
Steps	Cookies are normally only accessible to the domain for which they were scoped. Unfortunately, the definition of "domain" does not include the protocol so cookies that are created over HTTPS are accessible over HTTP. The "secure" attribute indicates to the browser that the cookie should only be made available over HTTPS. Ensure that all cookies set over HTTPS use the **secure** attribute. The requirement can be enforced in the web.config file by setting the requireSSL attribute to true. It is the preferred approach because it will enforce the **secure** attribute for all current and future cookies without the need to make any additional code changes.

Example

```
<configuration>
  <system.web>
    <httpCookies requireSSL="true"/>
  </system.web>
</configuration>
```

The setting is enforced even if HTTP is used to access the application. If HTTP is used to access the application, the setting breaks the application because the cookies are set with the secure attribute and the browser will not send them back to the application.

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Web Forms, MVC5
Attributes	EnvironmentType - OnPrem
References	N/A
Steps	When the web application is the Relying Party, and the IdP is ADFS server, the FedAuth token's secure attribute can be configured by setting requireSSL to True in `system.identityModel.services` section of web.config:

TITLE	DETAILS

Example

```
  <system.identityModel.services>
    <federationConfiguration>
      <!-- Set requireSsl=true; domain=application domain name used by FedAuth cookies (Ex: .gdinfra.com); -
->
      <cookieHandler requireSsl="true" persistentSessionLifetime="0.0:20:0" />
    ....
    </federationConfiguration>
  </system.identityModel.services>
```

All http based application should specify http only for cookie definition

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	Secure Cookie Attribute
Steps	To mitigate the risk of information disclosure with a cross-site scripting (XSS) attack, a new attribute - httpOnly - was introduced to cookies and is supported by all major browsers. The attribute specifies that a cookie is not accessible through script. By using HttpOnly cookies, a web application reduces the possibility that sensitive information contained in the cookie can be stolen via script and sent to an attacker's website.

Example

All HTTP-based applications that use cookies should specify HttpOnly in the cookie definition, by implementing following configuration in web.config:

```
<system.web>
.
.
   <httpCookies requireSSL="false" httpOnlyCookies="true"/>
.
.
</system.web>
```

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Web Forms
Attributes	N/A
References	FormsAuthentication.RequireSSL Property
Steps	The RequireSSL property value is set in the configuration file for an ASP.NET application by using the requireSSL attribute of the configuration element. You can specify in the Web.config file for your ASP.NET application whether Transport Layer Security (TLS), previously known as SSL (Secure Sockets Layer), is required to return the forms-authentication cookie to the server by setting the requireSSL attribute.

Example

The following code example sets the requireSSL attribute in the Web.config file.

```
<authentication mode="Forms">
  <forms loginUrl="member_login.aspx" cookieless="UseCookies" requireSSL="true"/>
</authentication>
```

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	MVC5
Attributes	EnvironmentType - OnPrem
References	Windows Identity Foundation (WIF) Configuration – Part II
Steps	To set httpOnly attribute for FedAuth cookies, hideFromCsript attribute value should be set to True.

Example

Following configuration shows the correct configuration:

```
<federatedAuthentication>
<cookieHandler mode="Custom"
                       hideFromScript="true"
                       name="FedAuth"
                       path="/"
                       requireSsl="true"
                       persistentSessionLifetime="25">
</cookieHandler>
</federatedAuthentication>
```

Mitigate against Cross-Site Request Forgery (CSRF) attacks on ASP.NET web pages

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Cross-site request forgery (CSRF or XSRF) is a type of attack in which an attacker can carry out actions in the security context of a different user's established session on a web site. The goal is to modify or delete content, if the targeted web site relies exclusively on session cookies to authenticate received request. An attacker could exploit this vulnerability by getting a different user's browser to load a URL with a command from a vulnerable site on which the user is already logged in. There are many ways for an attacker to do that, such as by hosting a different web site that loads a resource from the vulnerable server, or getting the user to click a link. The attack can be prevented if the server sends an additional token to the client, requires the client to include that token in all future requests, and verifies that all future requests include a token that pertains to the current session, such as by using the ASP.NET AntiForgeryToken or ViewState.

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	MVC5, MVC6
Attributes	N/A
References	XSRF/CSRF Prevention in ASP.NET MVC and Web Pages

TITLE	DETAILS
Steps	Anti-CSRF and ASP.NET MVC forms - Use the `AntiForgeryToken` helper method on Views; put an `Html.AntiForgeryToken()` into the form, for example,

Example

```
@using (Html.BeginForm("UserProfile", "SubmitUpdate")) {
    @Html.ValidationSummary(true)
    @Html.AntiForgeryToken()
    <fieldset>
```

Example

```
<form action="/UserProfile/SubmitUpdate" method="post">
    <input name="__RequestVerificationToken" type="hidden"
value="saTFWpkKN0BYazFtN6c4YbZAmsEwG0srqlUqqloi/fVgeV2ciIFVmelvzwRZpArs" />
    <!-- rest of form goes here -->
</form>
```

Example

At the same time, Html.AntiForgeryToken() gives the visitor a cookie called __RequestVerificationToken, with the same value as the random hidden value shown above. Next, to validate an incoming form post, add the [ValidateAntiForgeryToken] filter to the target action method. For example:

```
[ValidateAntiForgeryToken]
public ViewResult SubmitUpdate()
{
// ... etc.
}
```

Authorization filter that checks that:

- The incoming request has a cookie called __RequestVerificationToken
- The incoming request has a `Request.Form` entry called __RequestVerificationToken
- These cookie and `Request.Form` values match Assuming all is well, the request goes through as normal. But if not, then an authorization failure with message “A required anti-forgery token was not supplied or was invalid”.

Example

Anti-CSRF and AJAX: The form token can be a problem for AJAX requests, because an AJAX request might send JSON data, not HTML form data. One solution is to send the tokens in a custom HTTP header. The following code uses Razor syntax to generate the tokens, and then adds the tokens to an AJAX request.

```
<script>
    @functions{
        public string TokenHeaderValue()
        {
            string cookieToken, formToken;
            AntiForgery.GetTokens(null, out cookieToken, out formToken);
            return cookieToken + ":" + formToken;
        }
    }

    $.ajax("api/values", {
        type: "post",
        contentType: "application/json",
        data: {  }, // JSON data goes here
        dataType: "json",
        headers: {
            'RequestVerificationToken': '@TokenHeaderValue()'
        }
    });
</script>
```

Example

When you process the request, extract the tokens from the request header. Then call the AntiForgery.Validate method to validate the tokens. The Validate method throws an exception if the tokens are not valid.

```
void ValidateRequestHeader(HttpRequestMessage request)
{
    string cookieToken = "";
    string formToken = "";

    IEnumerable<string> tokenHeaders;
    if (request.Headers.TryGetValues("RequestVerificationToken", out tokenHeaders))
    {
        string[] tokens = tokenHeaders.First().Split(':');
        if (tokens.Length == 2)
        {
            cookieToken = tokens[0].Trim();
            formToken = tokens[1].Trim();
        }
    }
    AntiForgery.Validate(cookieToken, formToken);
}
```

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Web Forms
Attributes	N/A
References	Take Advantage of ASP.NET Built-in Features to Fend Off Web Attacks

TITLE	DETAILS
Steps	CSRF attacks in WebForm based applications can be mitigated by setting ViewStateUserKey to a random string that varies for each user - user ID or, better yet, session ID. For a number of technical and social reasons, session ID is a much better fit because a session ID is unpredictable, times out, and varies on a per-user basis.

Example

Here's the code you need to have in all of your pages:

```
void Page_Init (object sender, EventArgs e) {
   ViewStateUserKey = Session.SessionID;
   :
}
```

Set up session for inactivity lifetime

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	HttpSessionState.Timeout Property
Steps	Session timeout represents the event occurring when a user does not perform any action on a web site during an interval (defined by web server). The event, on server side, change the status of the user session to 'invalid' (for example "not used anymore") and instruct the web server to destroy it (deleting all data contained into it). The following code example sets the timeout session attribute to 15 minutes in the Web.config file.

Example

```
<configuration>
  <system.web>
    <sessionState mode="InProc" cookieless="true" timeout="15" />
  </system.web>
</configuration>
```

Enable Threat detection on Azure SQL

TITLE	DETAILS
Component	Web Application

TITLE	DETAILS
SDL Phase	Build
Applicable Technologies	Web Forms
Attributes	N/A
References	Forms element for authentication (ASP.NET Settings Schema)
Steps	Set the Forms Authentication Ticket cookie timeout to 15 minutes

Example

```
<forms  name=".ASPXAUTH" loginUrl="login.aspx"  defaultUrl="default.aspx" protection="All" timeout="15"
path="/" requireSSL="true" slidingExpiration="true"/>
</forms>
```

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Web Forms, MVC5
Attributes	EnvironmentType - OnPrem
References	asdeqa
Steps	When the web application is Relying Party and ADFS is the STS, the lifetime of the authentication cookies - FedAuth tokens - can be set by the following configuration in web.config:

Example

```
  <system.identityModel.services>
    <federationConfiguration>
      <!-- Set requireSsl=true; domain=application domain name used by FedAuth cookies (Ex: .gdinfra.com); -
->
      <cookieHandler requireSsl="true" persistentSessionLifetime="0.0:15:0" />
      <!-- Set requireHttps=true; -->
      <wsFederation passiveRedirectEnabled="true" issuer="http://localhost:39529/"
realm="https://localhost:44302/" reply="https://localhost:44302/" requireHttps="true"/>
      <!--
      Use the code below to enable encryption-decryption of claims received from ADFS. Thumbprint value
varies based on the certificate being used.
      <serviceCertificate>
        <certificateReference findValue="4FBBBA33A1D11A9022A5BF3492FF83320007686A"
storeLocation="LocalMachine" storeName="My" x509FindType="FindByThumbprint" />
      </serviceCertificate>
      -->
    </federationConfiguration>
  </system.identityModel.services>
```

Example

Also the ADFS issued SAML claim token's lifetime should be set to 15 minutes, by executing the following powershell command on the ADFS server:

```
Set-ADFSRelyingPartyTrust -TargetName "<RelyingPartyWebApp>" -ClaimsProviderName @("Active Directory") -
TokenLifetime 15 -AlwaysRequireAuthentication $true
```

Implement proper logout from the application

TITLE	DETAILS
Component	Web Application
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A
Steps	Perform proper Sign Out from the application, when user presses log out button. Upon logout, application should destroy user's session, and also reset and nullify session cookie value, along with resetting and nullifying authentication cookie value. Also, when multiple sessions are tied to a single user identity, they must be collectively terminated on the server side at timeout or logout. Lastly, ensure that Logout functionality is available on every page.

Mitigate against Cross-Site Request Forgery (CSRF) attacks on ASP.NET Web APIs

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	Generic
Attributes	N/A
References	N/A

TITLE	DETAILS
Steps	Cross-site request forgery (CSRF or XSRF) is a type of attack in which an attacker can carry out actions in the security context of a different user's established session on a web site. The goal is to modify or delete content, if the targeted web site relies exclusively on session cookies to authenticate received request. An attacker could exploit this vulnerability by getting a different user's browser to load a URL with a command from a vulnerable site on which the user is already logged in. There are many ways for an attacker to do that, such as by hosting a different web site that loads a resource from the vulnerable server, or getting the user to click a link. The attack can be prevented if the server sends an additional token to the client, requires the client to include that token in all future requests, and verifies that all future requests include a token that pertains to the current session, such as by using the ASP.NET AntiForgeryToken or ViewState.

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	MVC5, MVC6
Attributes	N/A
References	Preventing Cross-Site Request Forgery (CSRF) Attacks in ASP.NET Web API
Steps	Anti-CSRF and AJAX: The form token can be a problem for AJAX requests, because an AJAX request might send JSON data, not HTML form data. One solution is to send the tokens in a custom HTTP header. The following code uses Razor syntax to generate the tokens, and then adds the tokens to an AJAX request.

Example

```
<script>
    @functions{
        public string TokenHeaderValue()
        {
            string cookieToken, formToken;
            AntiForgery.GetTokens(null, out cookieToken, out formToken);
            return cookieToken + ":" + formToken;
        }
    }
    $.ajax("api/values", {
        type: "post",
        contentType: "application/json",
        data: {  }, // JSON data goes here
        dataType: "json",
        headers: {
            'RequestVerificationToken': '@TokenHeaderValue()'
        }
    });
</script>
```

Example

When you process the request, extract the tokens from the request header. Then call the AntiForgery.Validate method to validate the tokens. The Validate method throws an exception if the tokens are not valid.

```
void ValidateRequestHeader(HttpRequestMessage request)
{
    string cookieToken = "";
    string formToken = "";

    IEnumerable<string> tokenHeaders;
    if (request.Headers.TryGetValues("RequestVerificationToken", out tokenHeaders))
    {
        string[] tokens = tokenHeaders.First().Split(':');
        if (tokens.Length == 2)
        {
            cookieToken = tokens[0].Trim();
            formToken = tokens[1].Trim();
        }
    }
    AntiForgery.Validate(cookieToken, formToken);
}
```

Example

Anti-CSRF and ASP.NET MVC forms - Use the AntiForgeryToken helper method on Views; put an Html.AntiForgeryToken() into the form, for example,

```
@using (Html.BeginForm("UserProfile", "SubmitUpdate")) {
    @Html.ValidationSummary(true)
    @Html.AntiForgeryToken()
    <fieldset>
}
```

Example

The example above will output something like the following:

```
<form action="/UserProfile/SubmitUpdate" method="post">
    <input name="__RequestVerificationToken" type="hidden"
value="saTFWpkKN0BYazFtN6c4YbZAmsEwG0srqlUqqloi/fVgeV2ciIFVmelvzwRZpArs" />
    <!-- rest of form goes here -->
</form>
```

Example

At the same time, Html.AntiForgeryToken() gives the visitor a cookie called __RequestVerificationToken, with the same value as the random hidden value shown above. Next, to validate an incoming form post, add the [ValidateAntiForgeryToken] filter to the target action method. For example:

```
[ValidateAntiForgeryToken]
public ViewResult SubmitUpdate()
{
// ... etc.
}
```

Authorization filter that checks that:

- The incoming request has a cookie called __RequestVerificationToken
- The incoming request has a `Request.Form` entry called __RequestVerificationToken
- These cookie and `Request.Form` values match Assuming all is well, the request goes through as normal. But if not, then an authorization failure with message "A required anti-forgery token was not supplied or was invalid".

TITLE	DETAILS
Component	Web API
SDL Phase	Build
Applicable Technologies	MVC5, MVC6
Attributes	Identity Provider - ADFS, Identity Provider - Azure AD
References	Secure a Web API with Individual Accounts and Local Login in ASP.NET Web API 2.2
Steps	If the Web API is secured using OAuth 2.0, then it expects a bearer token in Authorization request header and grants access to the request only if the token is valid. Unlike cookie based authentication, browsers do not attach the bearer tokens to requests. The requesting client needs to explicitly attach the bearer token in the request header. Therefore, for ASP.NET Web APIs protected using OAuth 2.0, bearer tokens are considered as a defense against CSRF attacks. Please note that if the MVC portion of the application uses forms authentication (i.e., uses cookies), anti-forgery tokens have to be used by the MVC web app.

Example

The Web API has to be informed to rely ONLY on bearer tokens and not on cookies. It can be done by the following configuration in `WebApiConfig.Register` method:

```
config.SuppressDefaultHostAuthentication();
config.Filters.Add(new HostAuthenticationFilter(OAuthDefaults.AuthenticationType));
```

The SuppressDefaultHostAuthentication method tells Web API to ignore any authentication that happens before the request reaches the Web API pipeline, either by IIS or by OWIN middleware. That way, we can restrict Web API to authenticate only using bearer tokens.